

PERANCANGAN SISTEM KENDALI **TERDESENTRALISASI** UNTUK **FORMASI** KAWANAN **QUADCOPTER** BERBASIS **IMPROVED ARTIFICIAL POTENTIAL FIELD & EVENT-BASED** **RECONFIGURATION CONTROL**

Oleh:

Izma Alhazmi Herdian – 13321027

Dosen Pembimbing:

Ir. Estiyanti Ekawati, M.T., Ph.D.

Faqihza Mukhlis, S.T., M.T., Ph.D

Dosen Penguji:

Prof. Dr. Suprijanto, S.T., M.T

Dr. Ing. Thomas Budiarto, S.T., M.T.

09.00 – 10.00
Selasa, 8 Juli 2025

Overview

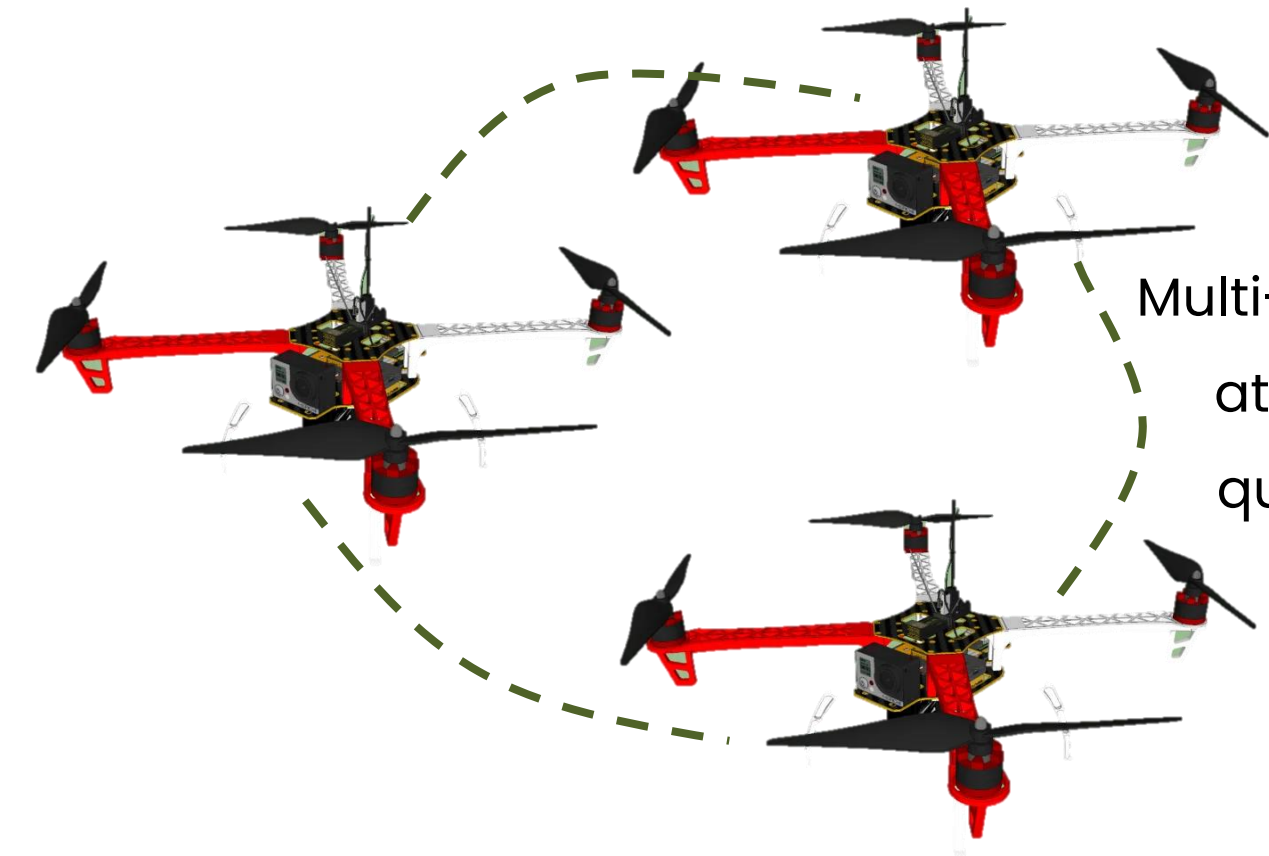
- 1 Pendahuluan Latar Belakang, Rumusan Masalah, Tujuan Penelitian
- 2 Metodologi Arsitektur Sistem, Metrik Kinerja, Implementasi *Software*
- 3 Hasil & Analisis Parameter, Verifikasi Simulator, Pengujian Skenario
- 4 Kesimpulan & Saran Kesimpulan penelitian ini dan saran penelitian berikutnya
- 5 Lampiran Grafik Tambahan

Swarm of Micro Flying Robots in the Wild [All] – Fei Gao [*]



[*] <https://www.youtube.com/watch?v=L0fJ0EHHfOA>

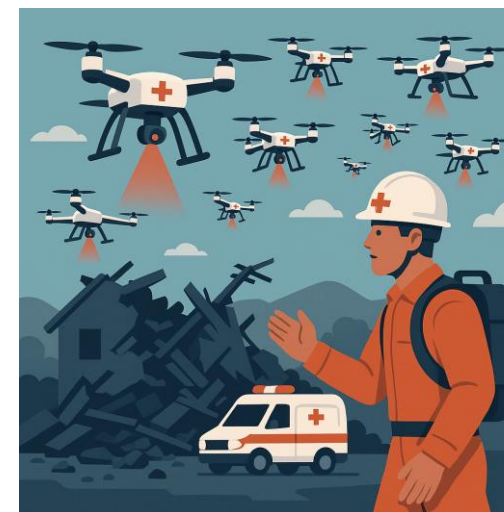
Latar Belakang



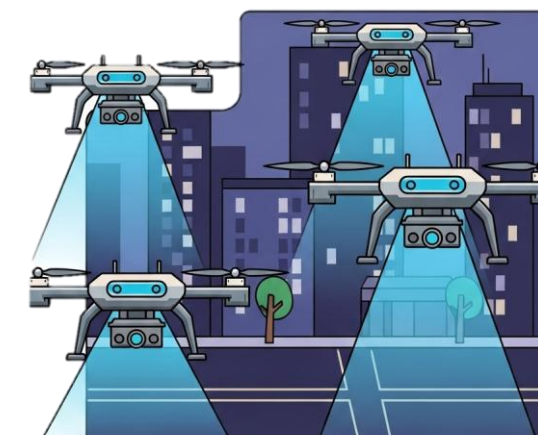
Multi-Quadcopter
atau swarm
quadcopter

Quadcopter atau quadrotor adalah salah satu jenis *unmanned aerial vehicle* (UAV) yang menggunakan 4 rotor untuk bermanuver di udara

Pencarian korban
di area luas [5]



Patroli Keamanan
Otomatis [6]



Inspeksi Infrastruktur [7]



Rumusan Masalah

Tujuan

Batasan & Asumsi

[5] F. A. De Alcantara Andrade et al., "Autonomous Unmanned Aerial Vehicles in Search and Rescue Missions Using Real-Time Cooperative Model Predictive Control," *Sensors*, vol. 19, no. 19, p. 4067, Sep. 2019, doi:10.3390/s19194067.

[6] A. V. Borkar, S. Hangal, H. Arya, A. Sinha, and L. Vachhani, "Reconfigurable formations of quadrotors on Lissajous curves for surveillance applications," *Eur. J. Control*, vol. 56, pp. 274–288, Nov. 2020, doi:10.1016/j.ejcon.2020.03.005.

[7] P. J. Sanchez-Cuevas et al., "Fully-Actuated Aerial Manipulator for Infrastructure Contact Inspection: Design, Modeling, Localization, and Control," *Sensors*, vol. 20, no. 17, p. 4708, Aug. 2020, doi:10.3390/s20174708.

Militer [*]



Show [**]



Terdapat tantangan utama untuk kawanan quadcopter, seperti:

1. Navigasi ke titik tujuan
2. Menghindari rintangan
3. Mencegah tabrakan antar-agen
4. Mempertahankan formasi
5. Adaptasi di ruang sempit
6. Stabilitas melawan gangguan angin

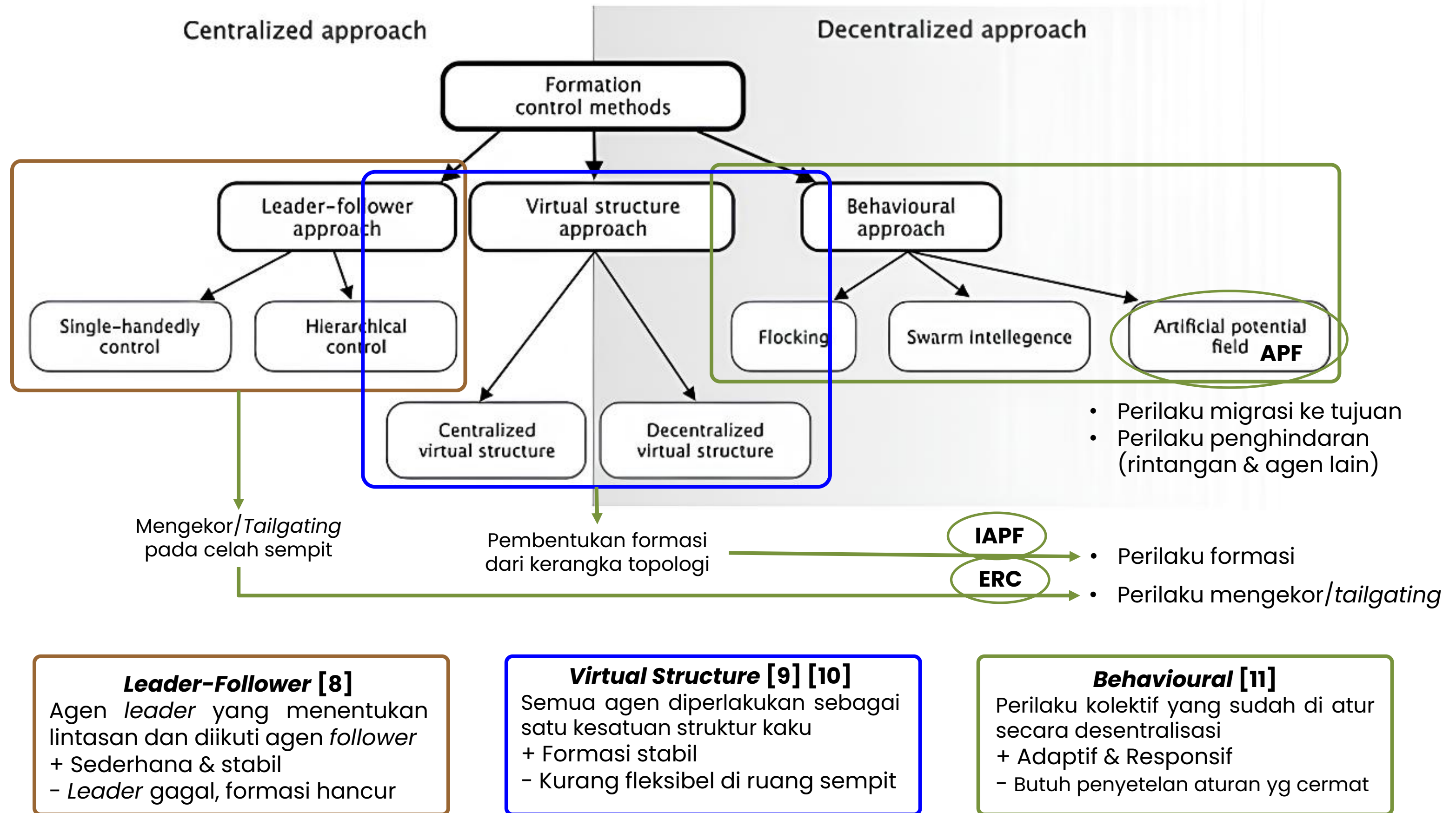


Sistem **kontrol formasi** yang adaptif

[*] https://youtube.com/shorts/LsCN49kkp_A?si=dUdn6l9qml9gghp9

[**] <https://www.youtube.com/watch?v=UgVH7u3j-mg>

Kontrol Formasi [4]



[4] S. Yu. Kurochkin and A. A. Tachkov, "Methods of Formation Control for a Group of Mobile Robots (a Review)," *Mekhatronika Avtom. Upr.*, vol. 22, no. 6, pp. 304–312, Jun. 2021, doi: 10.17587/mau.22.304–312.

[8] G. E. M. Abro, A. M. Abdallah, F. Zahid, and S. Ahmed, "A Comprehensive Review of Next-Gen UAV Swarm Robotics: Optimisation Techniques and Control Strategies for Dynamic Environments," *Intell. Autom. Soft Comput.*, vol. 40, no. 1, pp. 99–123, 2025, doi: 10.32604/iasc.2025.060364.

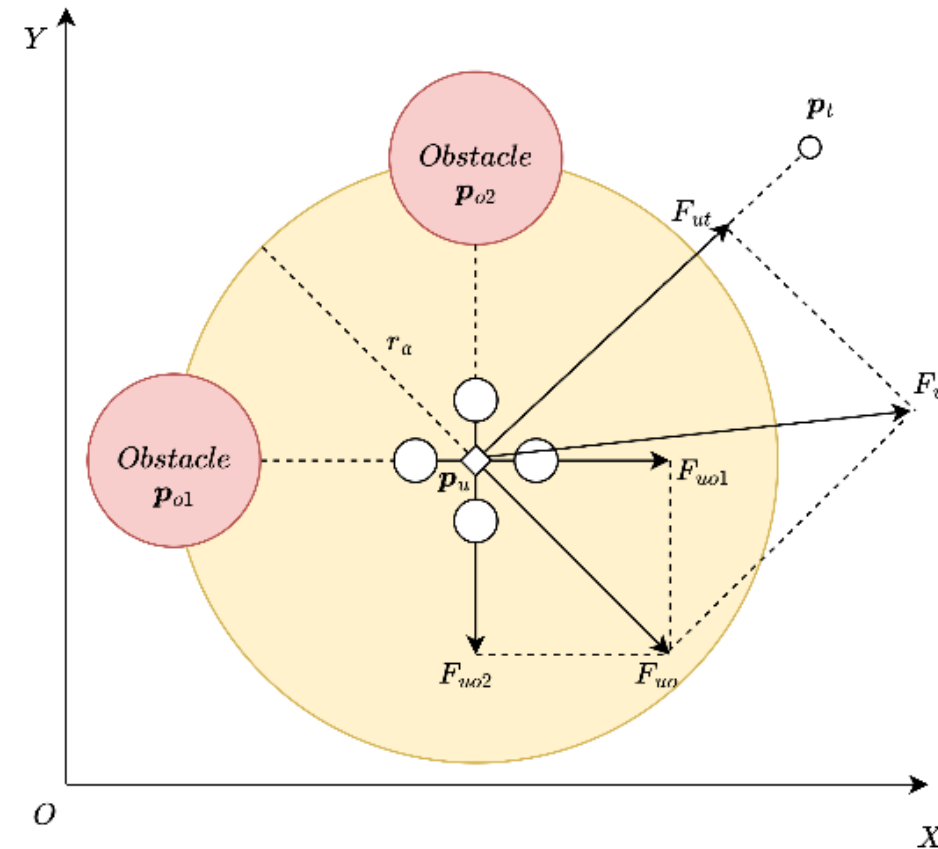
[9] W. Ren and R. W. Beard, "Formation feedback control for multiple spacecraft via virtual structures," *IEE Proc. - Control Theory Appl.*, vol. 151, no. 3, pp. 357–368, May 2004, doi: 10.1049/ip-cta:20040484.

[10] Y. Liu, J. Gao, X. Shi, and C. Jiang, "Decentralization of Virtual Linkage in Formation Control of Multi-Agents via Consensus Strategies," *Appl. Sci.*, vol. 8, no. 11, p. 2020, Oct. 2018, doi: 10.3390/app8112020.

[11] T. Balch and R. C. Arkin, "Behavior-based formation control for multirobot teams," *IEEE Trans. Robot. Autom.*, vol. 14, no. 6, pp. 926–939, Dec. 1998, doi: 10.1109/70.736776.

Artificial potential field

APF [12] [20]



$$F_u(p_u) = F_{ut}(p_u, p_t) + \sum_k F_{uo,k}(p_u, p_{o,k}) + \sum_{j=1, j \neq i}^N F_{ij}(p_i, p_j)$$

Improved Artificial Potential Field (IAPF) [13]

$$F_{form}(p_i) = k_{form} \sum_{j=1, j \neq i}^N [(p_j - p_i) - \kappa(\delta_j^* - \delta_i^*)]$$

di mana κ adalah faktor skala formasi. $\delta_j^* - \delta_i^*$ adalah vektor posisi relatif yang ditentukan oleh formasi yang diinginkan antara agen j dan i .

$$F_u(p_u) = F_{ut}(p_u, p_t) + \sum_k F_{uo,k}(p_u, p_{o,k}) + \sum_{j=1, j \neq i}^N F_{ij}(p_i, p_j) + F_{form}(p_i)$$

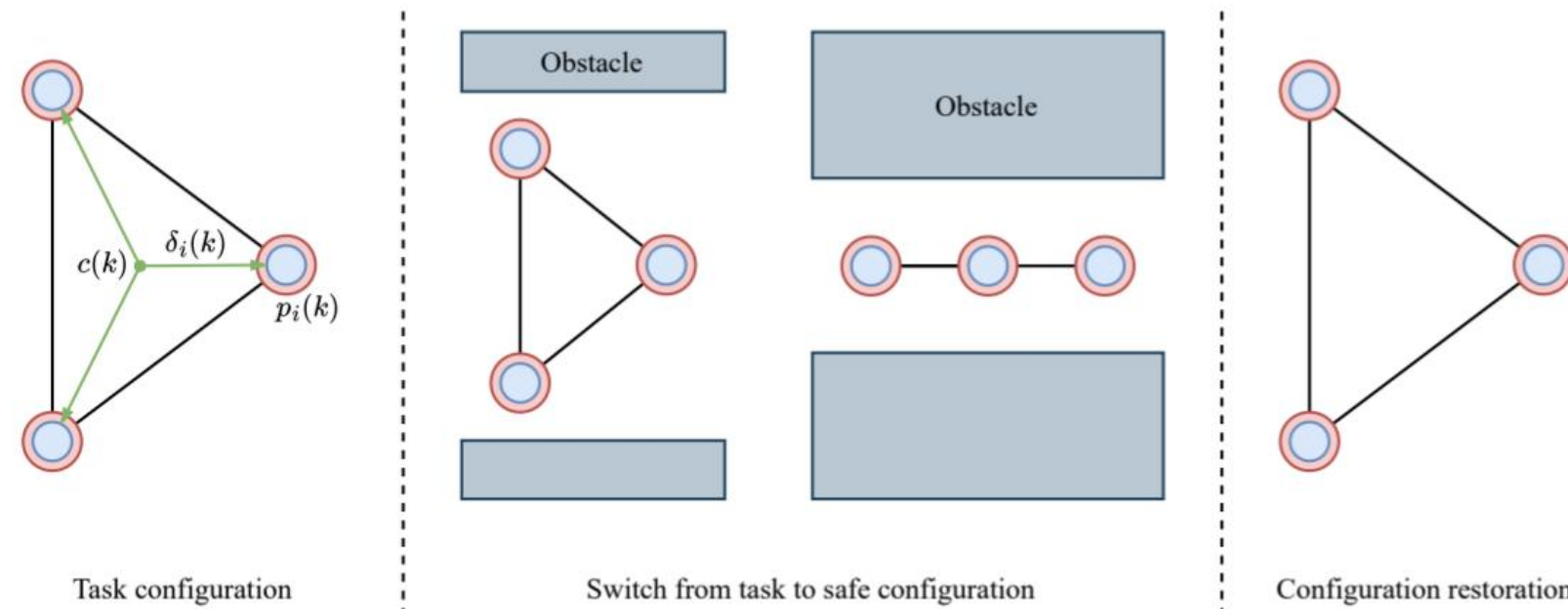
[12] N. Wang, J. Dai, and J. Ying, "UAV Formation Obstacle Avoidance Control Algorithm Based on Improved Artificial Potential Field and Consensus," Int.J. Aeronaut. Space Sci., vol. 22, no. 6, pp. 1413–1427, Dec. 2021, doi:10.1007/s42405-021-00407-6.

[13] D.-N. Bui, M. D. Phung, and H. P. Duy, "Event-based Reconfiguration Control for Time-varying Formation of Robot Swarms in Narrow Spaces," Intell. Serv. Robot., May 2025, doi: 10.1007/s11370-025-00607-z.

[20] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," 1986.

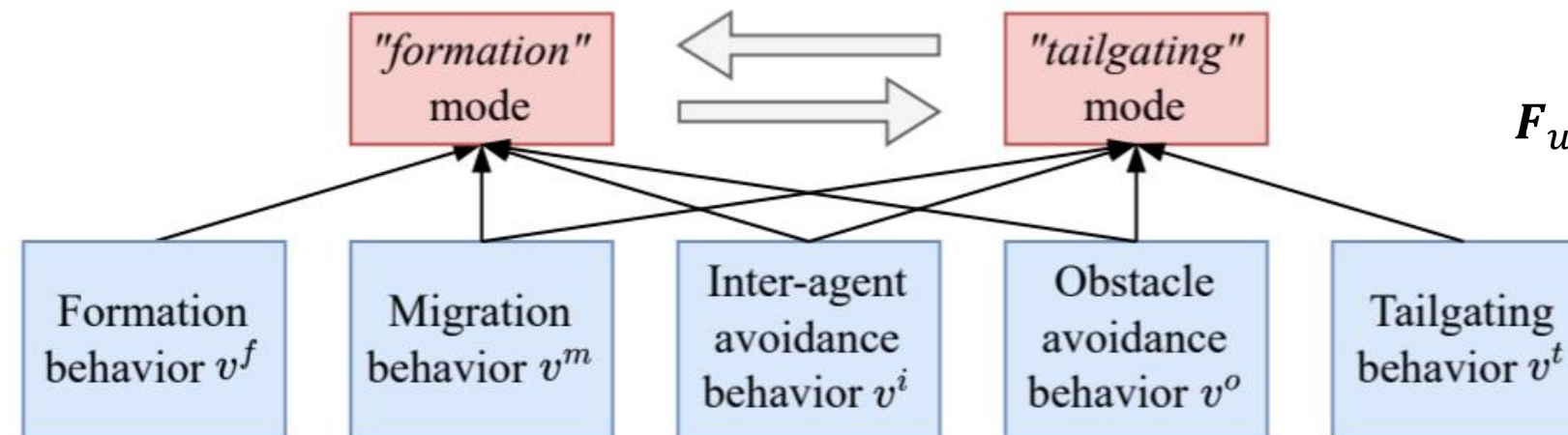
Event-Based Reconfiguration Control (ERC) [13]

...



$$F_{tail}(\mathbf{p}_i) = k_{tail} \left[(\mathbf{p}_{li} - \mathbf{p}_i) - d_{ref} \frac{\mathbf{v}_{li}}{\|\mathbf{v}_{li}\|} \right] + \mathbf{v}_{li}$$

di mana $\mathbf{p}_{li}$ dan $\mathbf{v}_{li}$ adalah posisi dan kecepatan agen pemimpin, k_{tail} adalah gain untuk kontrol mengekor, dan d_{ref} adalah jarak referensi yang diinginkan antara agen pengikut dan pemimpin.



$$F_u(\mathbf{p}_u) = F_{ut}(\mathbf{p}_u, \mathbf{p}_t) + \sum_k F_{uo,k}(\mathbf{p}_u, \mathbf{p}_{o,k}) + \sum_{j=1, j \neq i}^N F_{ij}(\mathbf{p}_i, \mathbf{p}_j) + \sigma_i F_{form}(\mathbf{p}_i) + (1 - \sigma_i) F_{tail}(\mathbf{p}_i)$$

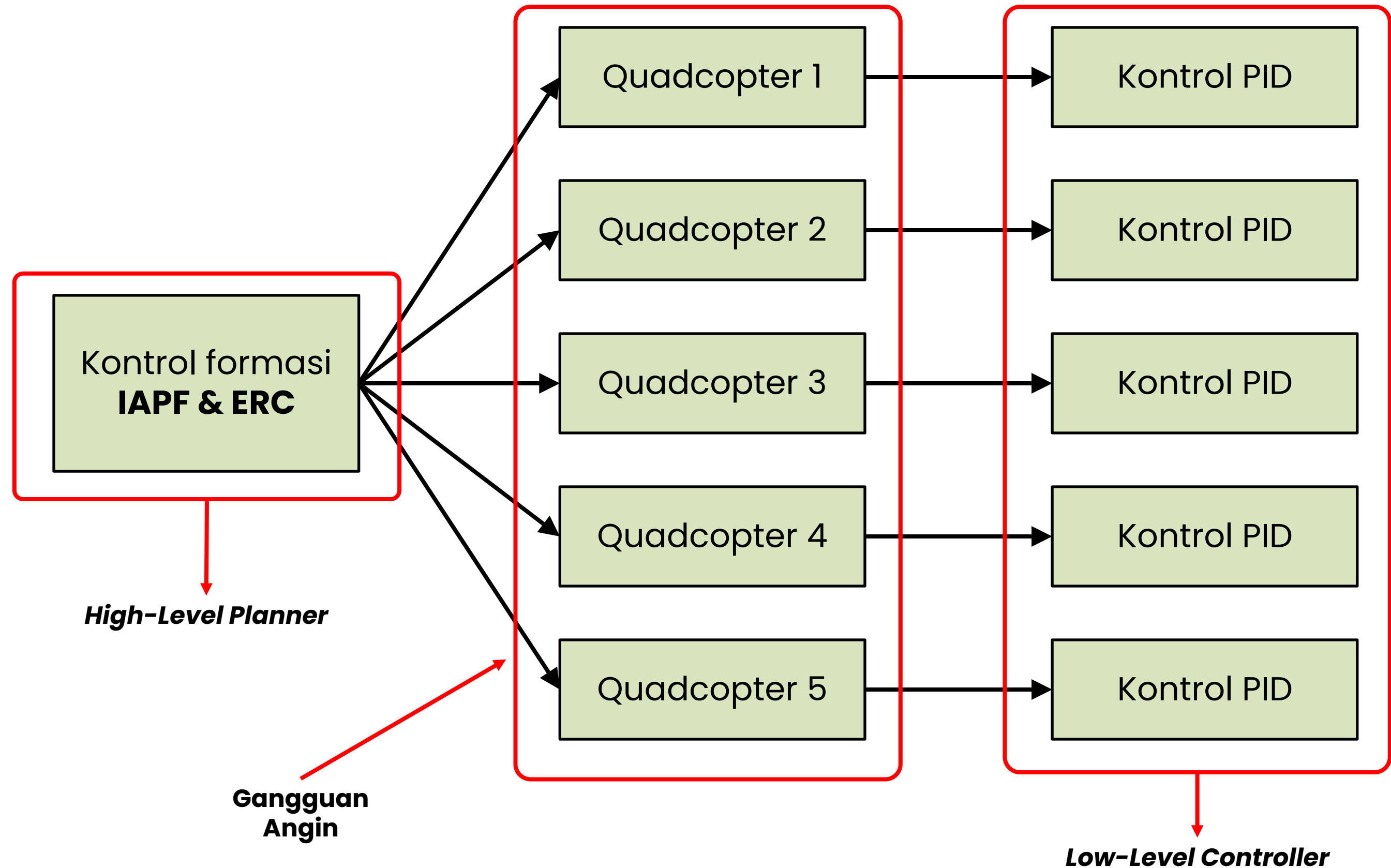
jika ruang bebas cukup luas, $\sigma_i = 1$ (mode formasi) dan jika ruang terbatas, $\sigma_i = 0$ (mode mengekor)

Latar Belakang

Rumusan Masalah

Tujuan

Batasan & Asumsi



Bagaimana memodelkan **dinamika** quadcopternya?

Bagaimana merancang **kontrol PID** quadcopternya?

Bagaimana mengoordinasikan kawanan quadcopter agar:

- bergerak bersama menuju **tujuan**,
- membentuk dan menjaga **formasi**,
- menghindari **tabrakan** dengan quadcopter lain dan rintangan
- **serta menyesuaikan formasi saat melewati ruang sempit?**

Bagaimana membangun **lingkungan simulasinya** dengan berbagai skenario rintangan dan gangguan angin, serta mengevaluasi kinerjanya?

Mengadaptasi platform simulasi *single-agent* menjadi sistem simulasi 3D untuk lima quadcopter yang mampu merepresentasikan **dinamika fisik** dari sebuah kawanan.

Mengimplementasikan dan mengembangkan kerangka kerja **high-level planner** berbasis perilaku dari studi literatur untuk menghasilkan sistem navigasi formasi yang fungsional, yang mampu: (a) mengikuti rangkaian *waypoint*, (b) melacak lintasan non-linear secara dinamis, serta (c) menjaga orientasi formasi agar selalu selaras dengan arah gerak kawanan.

Mengintegrasikan **high-level planner** dengan **low-level controller** pada setiap agen, sehingga perintah navigasi kolektif dapat diterjemahkan menjadi pergerakan quadcopter yang presisi dan terkoordinasi.

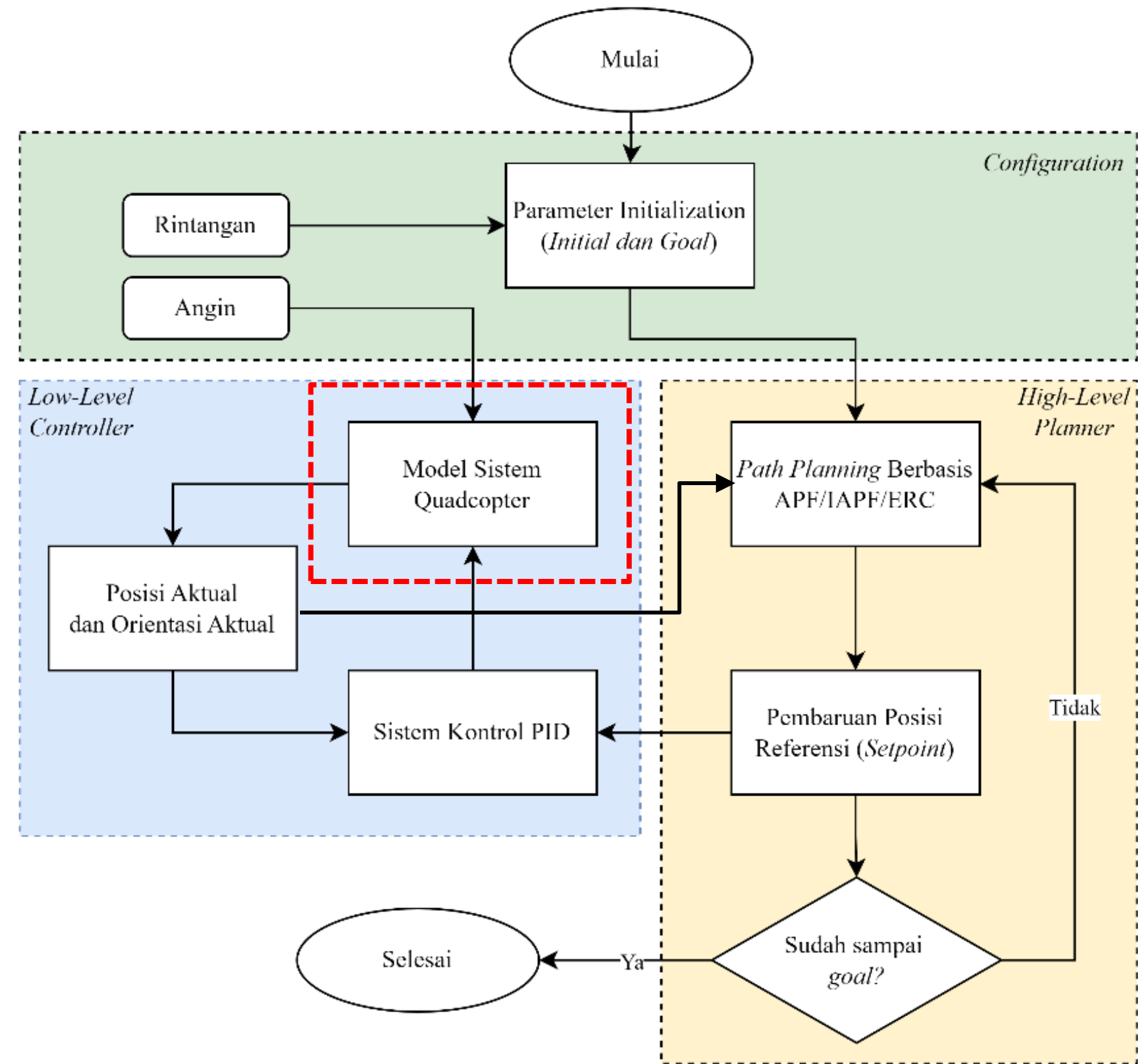
Mengevaluasi performa sistem terintegrasi dengan **menguji** secara kuantitatif kemampuan adaptasi formasi dalam melewati rintangan sempit dan ketahanannya dalam mempertahankan stabilitas saat menghadapi gangguan angin.

Batasan

1. **Simulasi** menggunakan python
2. Tiap quadcopter menggunakan **kontrol PID**
3. Penghindaran hanya pada bidang **xy**
4. Hanya **5 agen** quadcopter
5. *Planner* bersifat **reaktif**, bukan pemetaan
6. Parameter kontrol **PID statis**

Asumsi

1. Tiap quadcopter mempunyai parameter fisik yang **identik**
2. Komunikasi dan “sensor” **ideal**
3. Rintangan **statis**
4. **Gangguan angin** bersifat deterministik
5. Model **aerodinamika** quadcopter disederhanakan
6. Deteksi lingkungan dan *high-level planner* **ideal** dan akurat pada **sumbu-y** untuk memicu rekonfigurasi formasi.

Arsitektur
SistemMetrik
KinerjaImplementasi
Software

Arsitektur
Sistem

Metrik
Kinerja

Implementasi
Software

Model Sistem
Quadcopter

Arsitektur Sistem

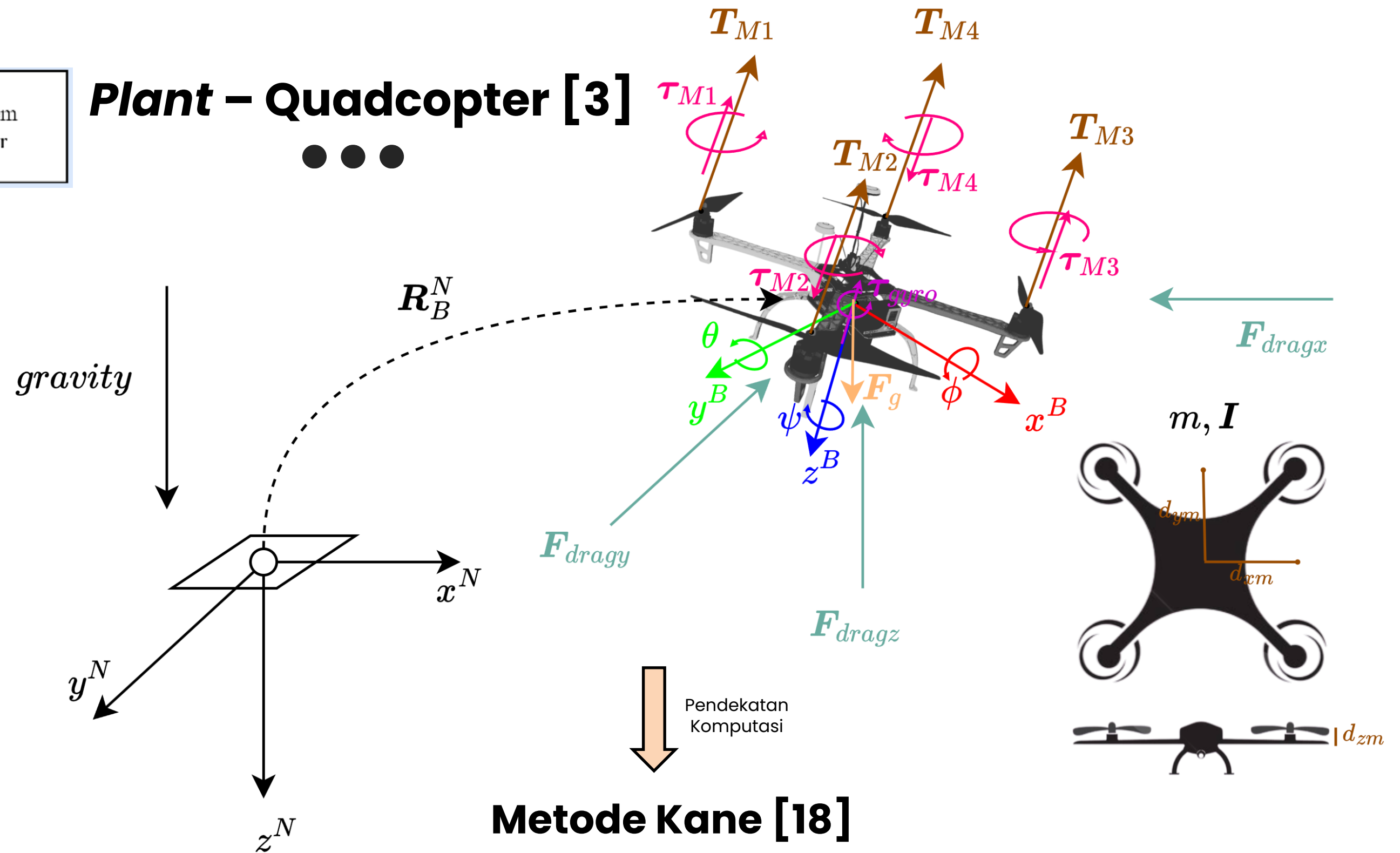
Metrik Kinerja

Implementasi Software

Model Sistem
Quadcopter

Plant – Quadcopter [3]

...



Metode Kane [18]

[3] John, bobzvik/Quadcopter_SimCon. (Jun. 10, 2025). Python. Accessed: Jun.13, 2025. [Online]. Available: https://github.com/bobzvik/Quadcopter_SimCon

[18] T. R. Kane and D. A. Levinson, "Dynamics Theory and Applications," 1985.

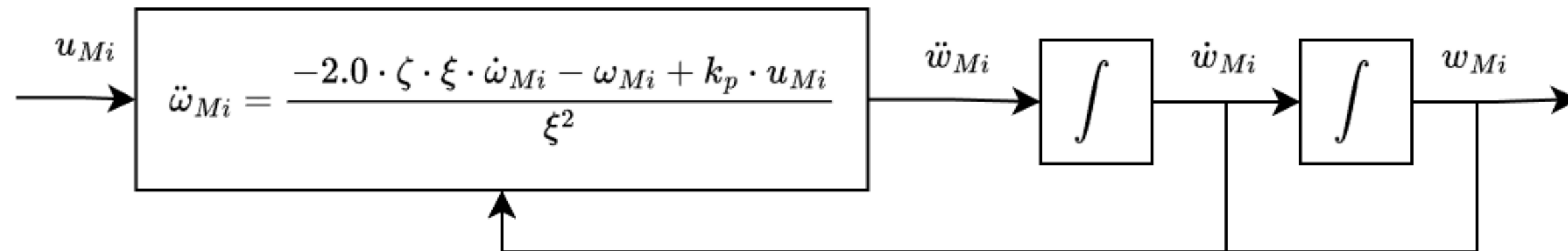
$$\dot{\mathbf{s}}_{\text{plant}}(t) = \frac{d}{dt} \begin{bmatrix} x \\ y \\ z \\ q_w \\ q_x \\ q_y \\ q_z \\ \dot{x} \\ \dot{y} \\ \dot{z} \\ \omega_\phi \\ \omega_\theta \\ \omega_\psi \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ -0.5(\omega_\phi q_x + \omega_\theta q_y + \omega_\psi q_z) \\ 0.5(\omega_\phi q_w - \omega_\theta q_z + \omega_\psi q_y) \\ 0.5(\omega_\phi q_z + \omega_\theta q_w - \omega_\psi q_x) \\ -0.5(\omega_\phi q_y - \omega_\theta q_x - \omega_\psi q_w) \\ \frac{C_d \text{sign}(v_w \cos q_{w1} \cos q_{w2} - \dot{x})(v_w \cos q_{w1} \cos q_{w2} - \dot{x})^2 - 2(q_w q_y + q_x q_z)(T_{M1} + T_{M2} + T_{M3} + T_{M4})}{m_B} \\ \frac{C_d \text{sign}(v_w \sin q_{w1} \cos q_{w2} - \dot{y})(v_w \sin q_{w1} \cos q_{w2} - \dot{y})^2 + 2(q_w q_x - q_y q_z)(T_{M1} + T_{M2} + T_{M3} + T_{M4})}{m_B} \\ \frac{-C_d \text{sign}(v_w \sin q_{w2} + \dot{z})(v_w \sin q_{w2} + \dot{z})^2 - (T_{M1} + T_{M2} + T_{M3} + T_{M4})(q_w^2 - q_x^2 - q_y^2 + q_z^2) + g m_B}{m_B} \\ \frac{(I_{Byy} - I_{Bzz})\omega_\theta \omega_\psi - u_p I_{Rzz}(\omega_{M1} - \omega_{M2} + \omega_{M3} - \omega_{M4})\omega_\theta + (T_{M1} - T_{M2} - T_{M3} + T_{M4})d_{ym}}{I_{Bxx}} \\ \frac{(I_{Bzz} - I_{Bxx})\omega_\phi \omega_\psi + u_p I_{Rzz}(\omega_{M1} - \omega_{M2} + \omega_{M3} - \omega_{M4})\omega_\phi + (T_{M1} + T_{M2} - T_{M3} - T_{M4})d_{xm}}{I_{Byy}} \\ \frac{(I_{Bxx} - I_{Byy})\omega_\phi \omega_\theta + \tau_{M1} - \tau_{M2} + \tau_{M3} - \tau_{M4}}{I_{Bzz}} \end{bmatrix}$$

Actuator – Motor [3]



Aktuator (Dinamika Motor)

$$\mathbf{s}_{akt}(t) = [w_{M1}, \dot{w}_{M1}, w_{M2}, \dot{w}_{M2}, w_{M3}, \dot{w}_{M3}, w_{M4}, \dot{w}_{M4}]^T$$



$$\dot{\mathbf{s}} = \frac{d}{dt}$$

$$\begin{bmatrix} x \\ y \\ z \\ q_w \\ q_x \\ q_y \\ q_z \\ \dot{x} \\ \dot{y} \\ \dot{z} \\ \omega_\phi \\ \omega_\theta \\ \omega_\psi \\ \omega_{M1} \\ \dot{\omega}_{M1} \\ \omega_{M2} \\ \dot{\omega}_{M2} \\ \omega_{M3} \\ \dot{\omega}_{M3} \\ \omega_{M4} \\ \dot{\omega}_{M4} \end{bmatrix}$$

=

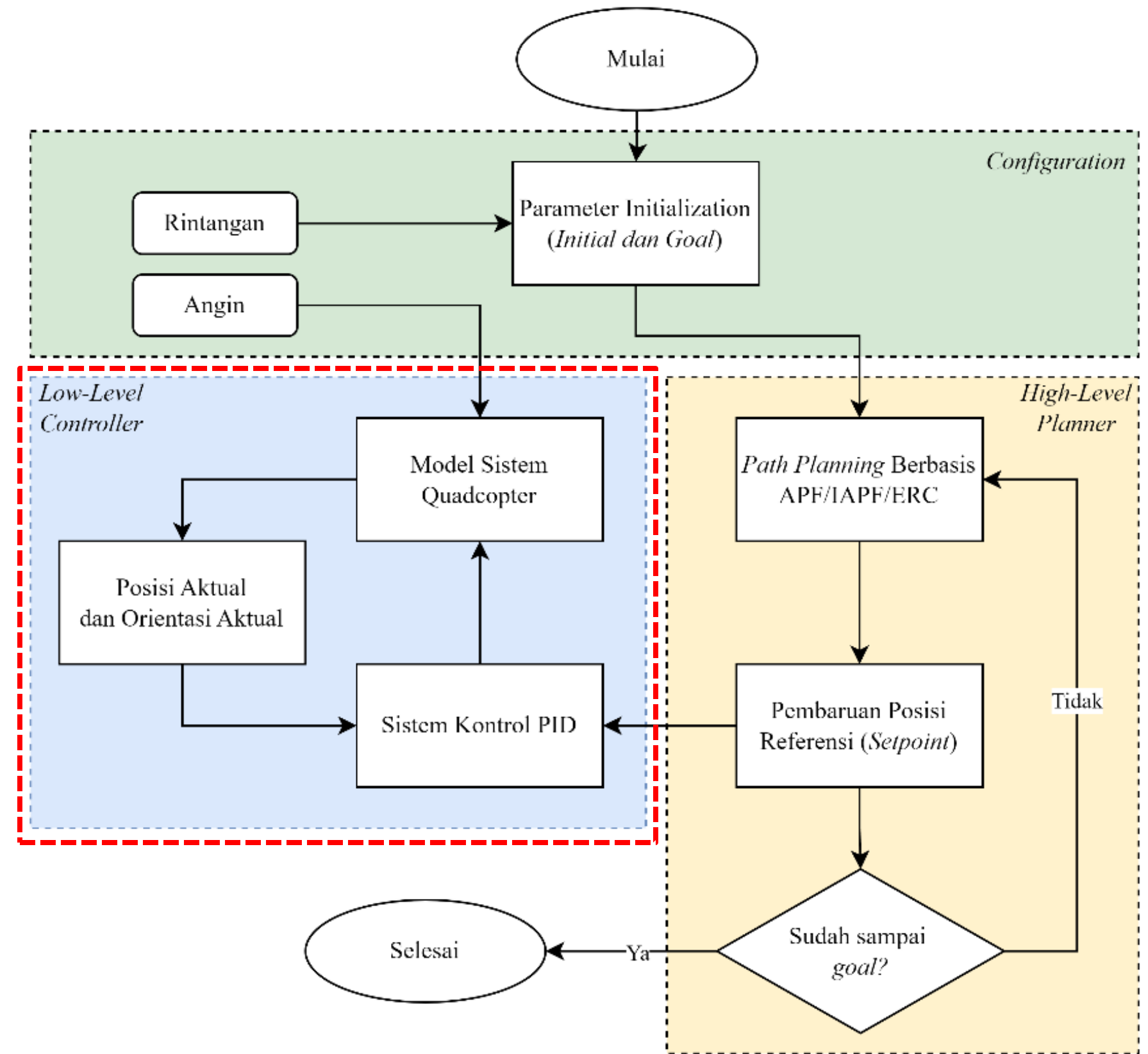
$$\begin{aligned} & \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ -0.5(\omega_\phi q_x + \omega_\theta q_y + \omega_\psi q_z) \\ 0.5(\omega_\phi q_w - \omega_\theta q_z + \omega_\psi q_y) \\ 0.5(\omega_\phi q_z + \omega_\theta q_w - \omega_\psi q_x) \\ -0.5(\omega_\phi q_y - \omega_\theta q_x - \omega_\psi q_w) \\ \frac{C_d \text{sign}(v_w \cos q_{w1} \cos q_{w2} - \dot{x}) (v_w \cos q_{w1} \cos q_{w2} - \dot{x})^2 - 2(q_w q_y + q_x q_z)(T_{M1} + T_{M2} + T_{M3} + T_{M4})}{m_B} \\ \frac{C_d \text{sign}(v_w \sin q_{w1} \cos q_{w2} - \dot{y}) (v_w \sin q_{w1} \cos q_{w2} - \dot{y})^2 + 2(q_w q_x - q_y q_z)(T_{M1} + T_{M2} + T_{M3} + T_{M4})}{m_B} \\ \frac{-C_d \text{sign}(v_w \sin q_{w2} + \dot{z}) (v_w \sin q_{w2} + \dot{z})^2 - (T_{M1} + T_{M2} + T_{M3} + T_{M4})(q_w^2 - q_x^2 - q_y^2 + q_z^2) + g m_B}{m_B} \\ \frac{(I_{Byy} - I_{Bzz})\omega_\theta \omega_\psi - u_p I_{Rzz}(\omega_{M1} - \omega_{M2} + \omega_{M3} - \omega_{M4})\omega_\theta + (T_{M1} - T_{M2} - T_{M3} + T_{M4})d_{ym}}{I_{Bxx}} \\ \frac{(I_{Bzz} - I_{Bxx})\omega_\phi \omega_\psi + u_p I_{Rzz}(\omega_{M1} - \omega_{M2} + \omega_{M3} - \omega_{M4})\omega_\phi + (T_{M1} + T_{M2} - T_{M3} - T_{M4})d_{xm}}{I_{Byy}} \\ \frac{(I_{Bxx} - I_{Byy})\omega_\phi \omega_\theta + \tau_{M1} - \tau_{M2} + \tau_{M3} - \tau_{M4}}{I_{Bzz}} \\ \dot{\omega}_{M1} \\ \frac{-2\zeta\xi\dot{\omega}_{M1} - \omega_{M1} + k_p u_{M1}}{\xi^2} \\ \dot{\omega}_{M2} \\ \frac{-2\zeta\xi\dot{\omega}_{M2} - \omega_{M2} + k_p u_{M2}}{\xi^2} \\ \dot{\omega}_{M3} \\ \frac{-2\zeta\xi\dot{\omega}_{M3} - \omega_{M3} + k_p u_{M3}}{\xi^2} \\ \dot{\omega}_{M4} \\ \frac{-2\zeta\xi\dot{\omega}_{M4} - \omega_{M4} + k_p u_{M4}}{\xi^2} \end{bmatrix} \end{aligned}$$

[3] John, bobzwik/Quadcopter_SimCon. (Jun. 10, 2025). Python. Accessed: Jun.13, 2025. [Online]. Available:https://github.com/bobzwik/Quadcopter_SimCon

Arsitektur Sistem

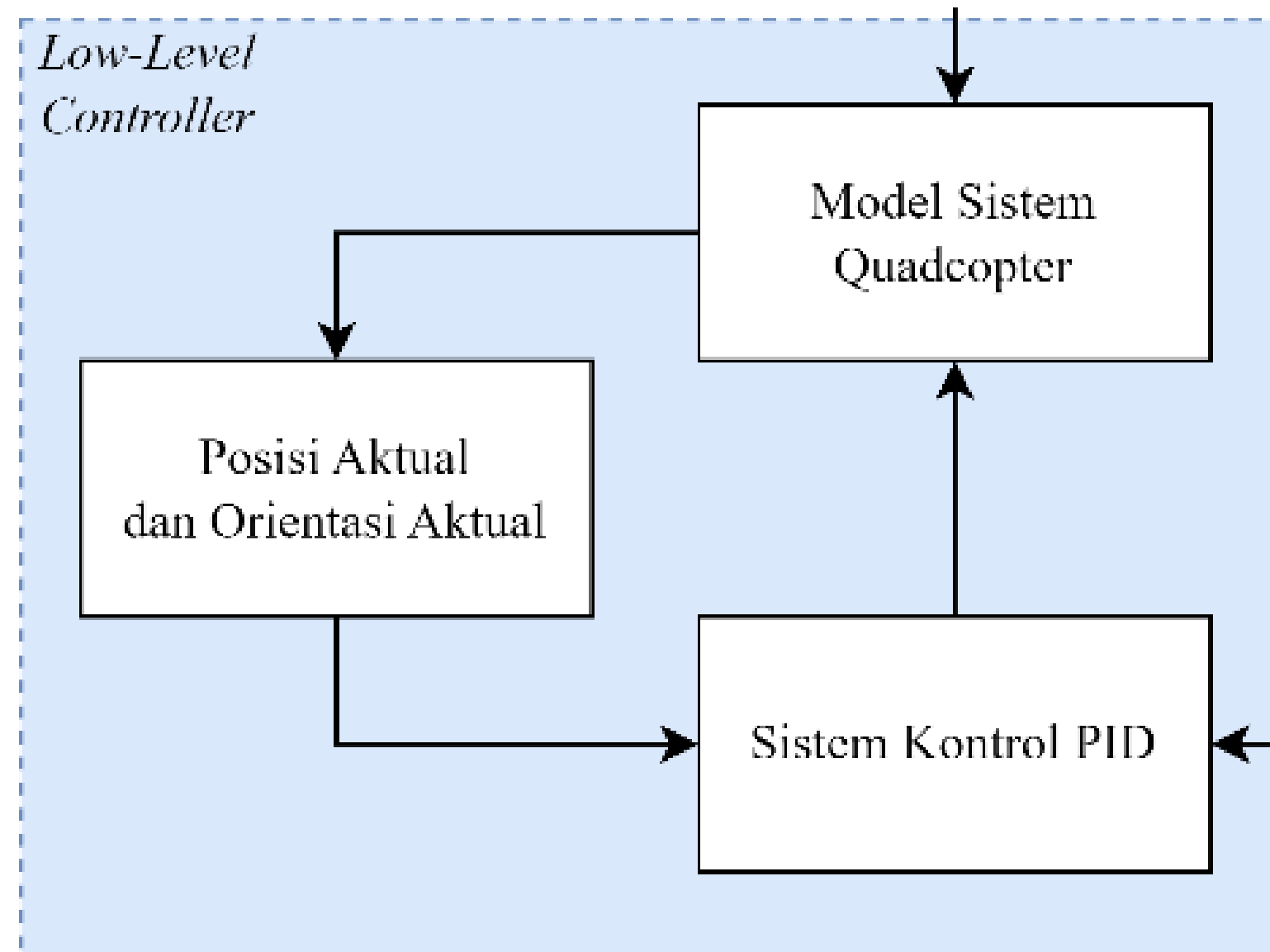
Metrik Kinerja

Implementasi Software

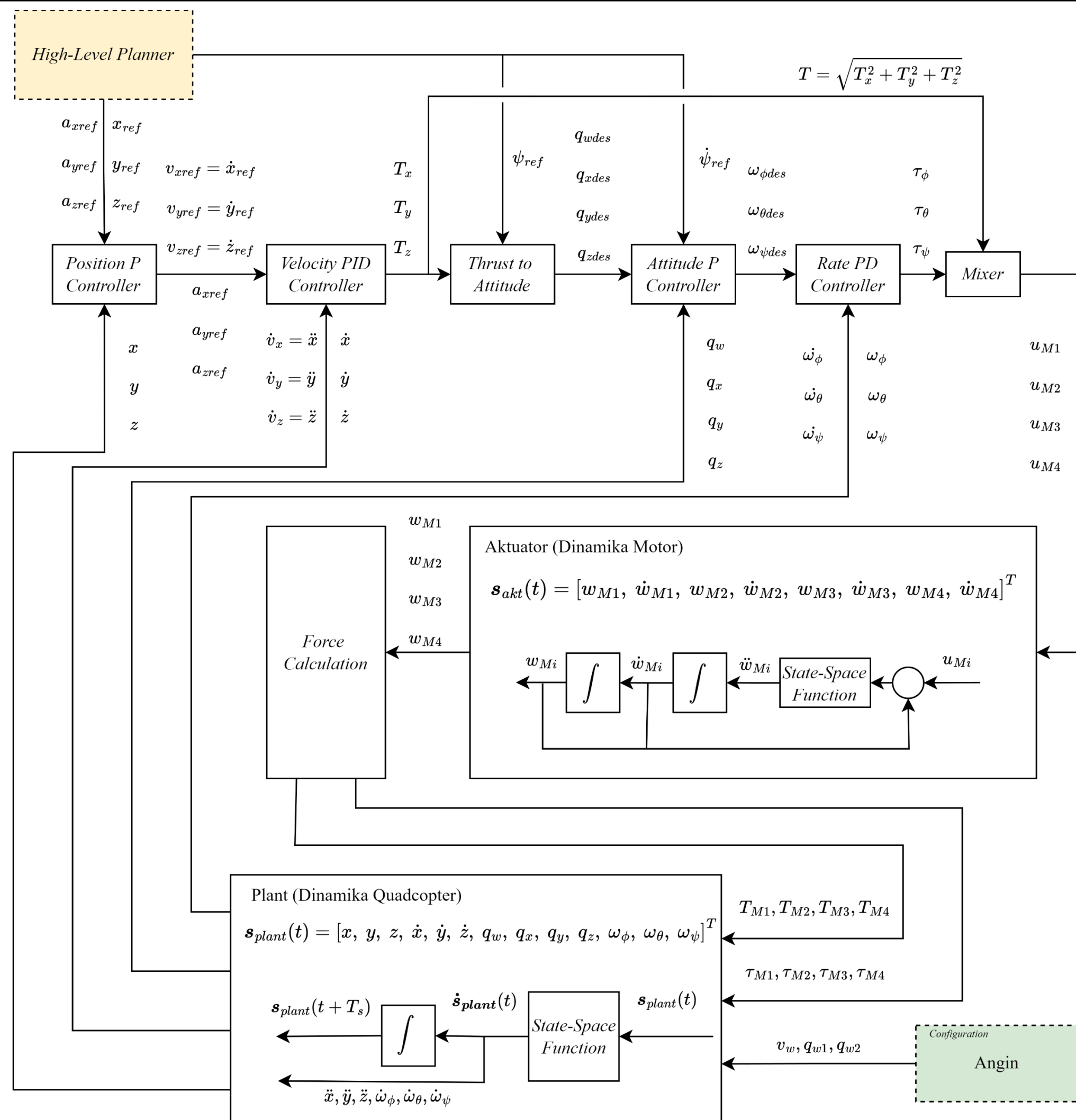
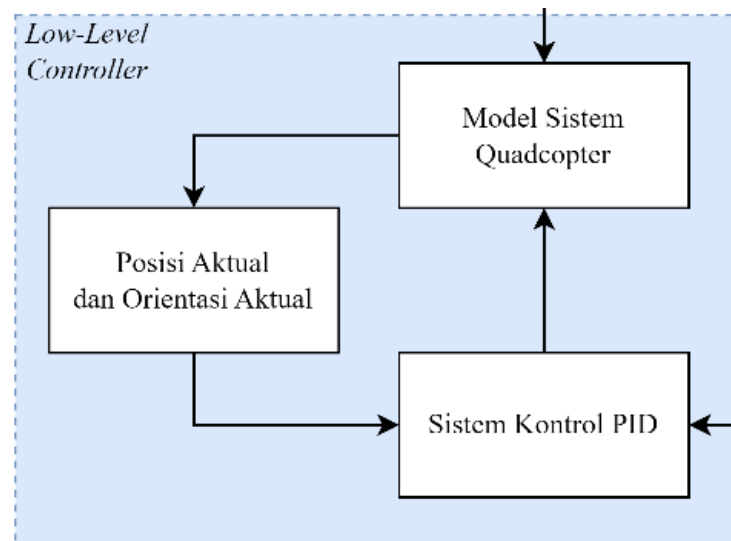


Low-Level Controller [3]

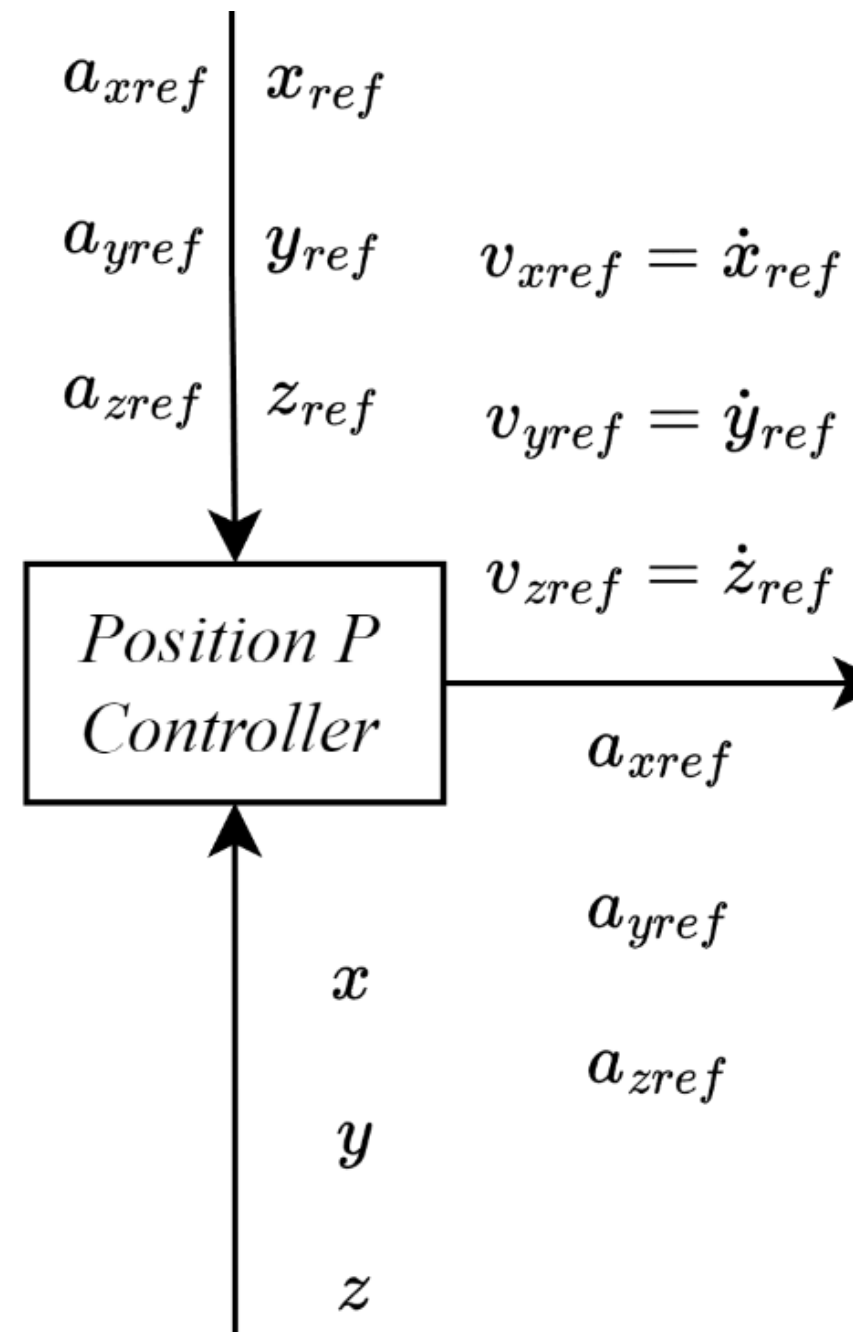
...



Low-Level Controller [3]



[3] John, bobzwik/Quadcopter_SimCon. (Jun. 10, 2025). Python. Accessed: Jun.13, 2025. [Online]. Available: https://github.com/bobzwik/Quadcopter_SimCon

Arsitektur
SistemMetrik
KinerjaImplementasi
Software

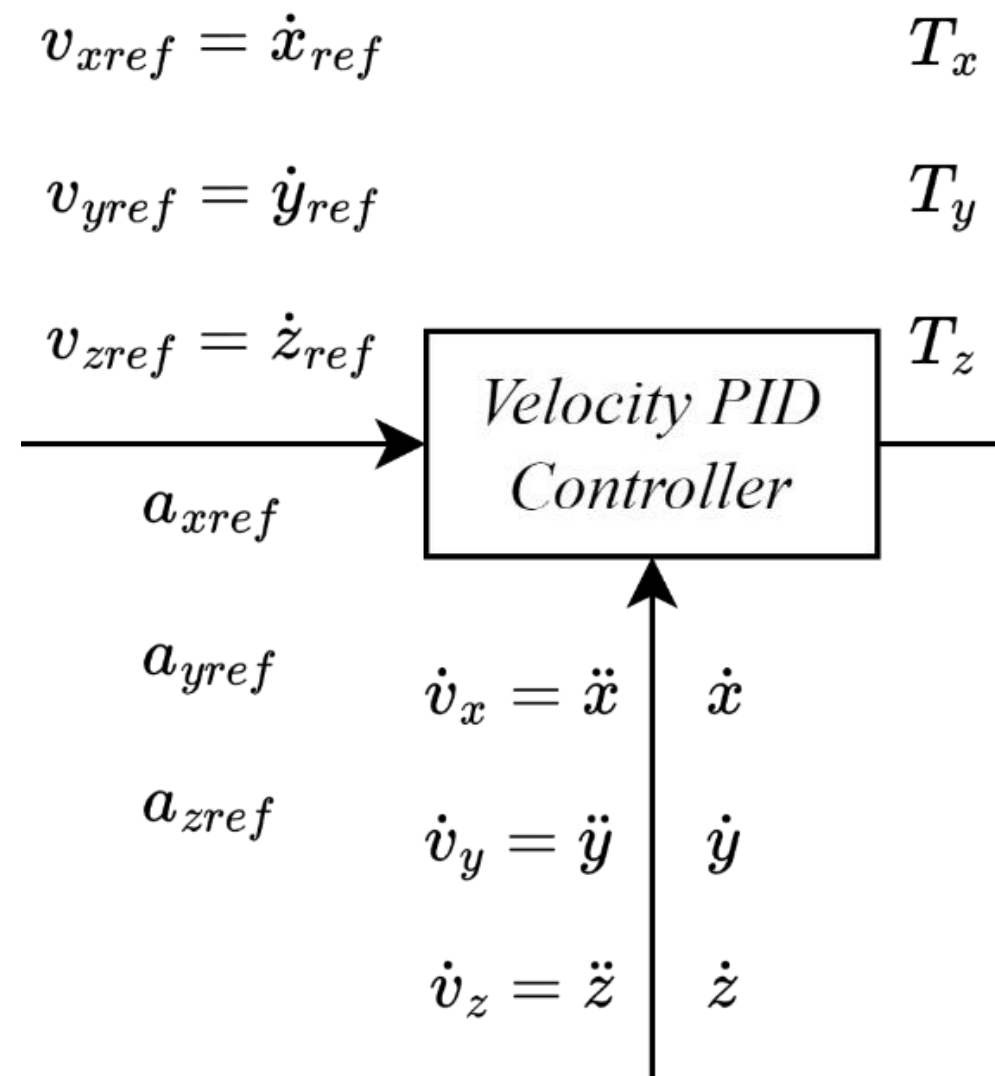
$$\mathbf{e}_p(t) = \mathbf{p}_{ref}(t) - \mathbf{p}(t)$$

$$\mathbf{v}_{ref}(t) = \mathbf{K}_{p,pos} \mathbf{e}_p(t)$$

Arsitektur Sistem

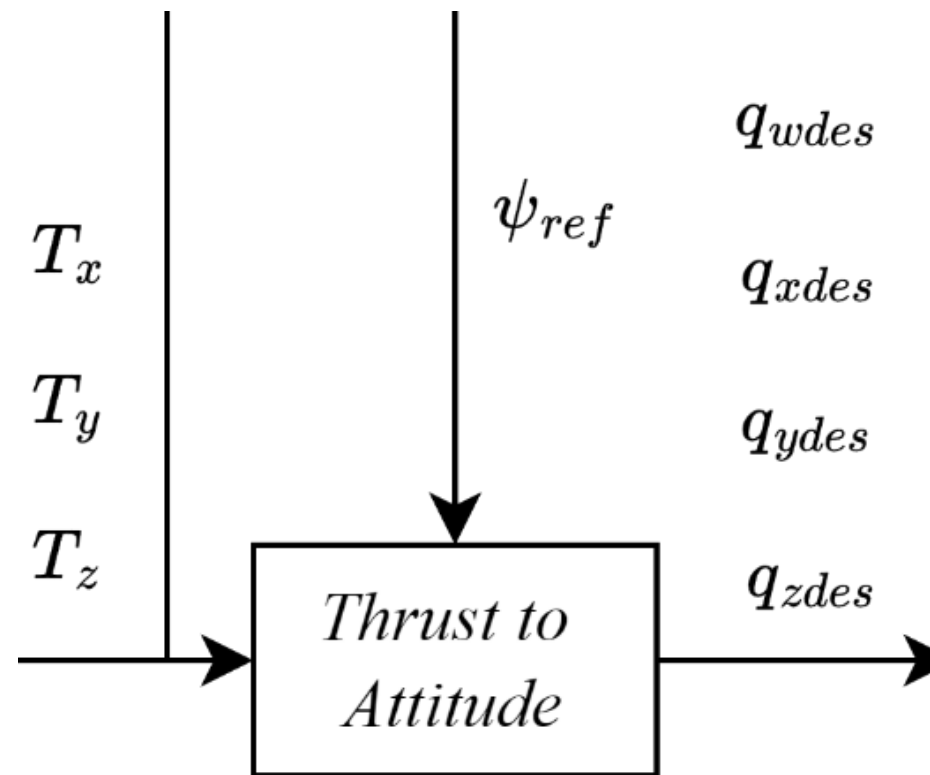
Metrik Kinerja

Implementasi Software



$$\mathbf{e}_v(t) = \mathbf{v}_{ref}(t) - \mathbf{v}(t)$$

$$\mathbf{T}_{sp}(t) = \mathbf{K}_{p,vel} \mathbf{e}_v(t) + \mathbf{K}_{i,vel} \int_0^t \mathbf{e}_v(\tau) d\tau + \mathbf{K}_{d,vel} \dot{\mathbf{v}}(t) + m\mathbf{a}_{ref}(t) - m\mathbf{g}$$

Arsitektur
SistemMetrik
KinerjaImplementasi
Software

$$\mathbf{b}_{z,d} = -\frac{T_{sp}}{\|\mathbf{T}_{sp}\|}$$

$$\mathbf{b}_{x,d} = \frac{\mathbf{y}_C \times \mathbf{b}_{z,d}}{\|\mathbf{y}_C \times \mathbf{b}_{z,d}\|}, \mathbf{y}_C = [-\sin(\psi_{ref}), \cos(\psi_{ref}), 0]^T$$

$$\mathbf{b}_{y,d} = \mathbf{b}_{z,d} \times \mathbf{b}_{x,d}$$

$$\mathbf{R}_d = [\mathbf{b}_{x,d} \quad \mathbf{b}_{y,d} \quad \mathbf{b}_{z,d}] = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix}$$

$$\mathbf{q} = [q_w, q_x, q_y, q_z]^T$$

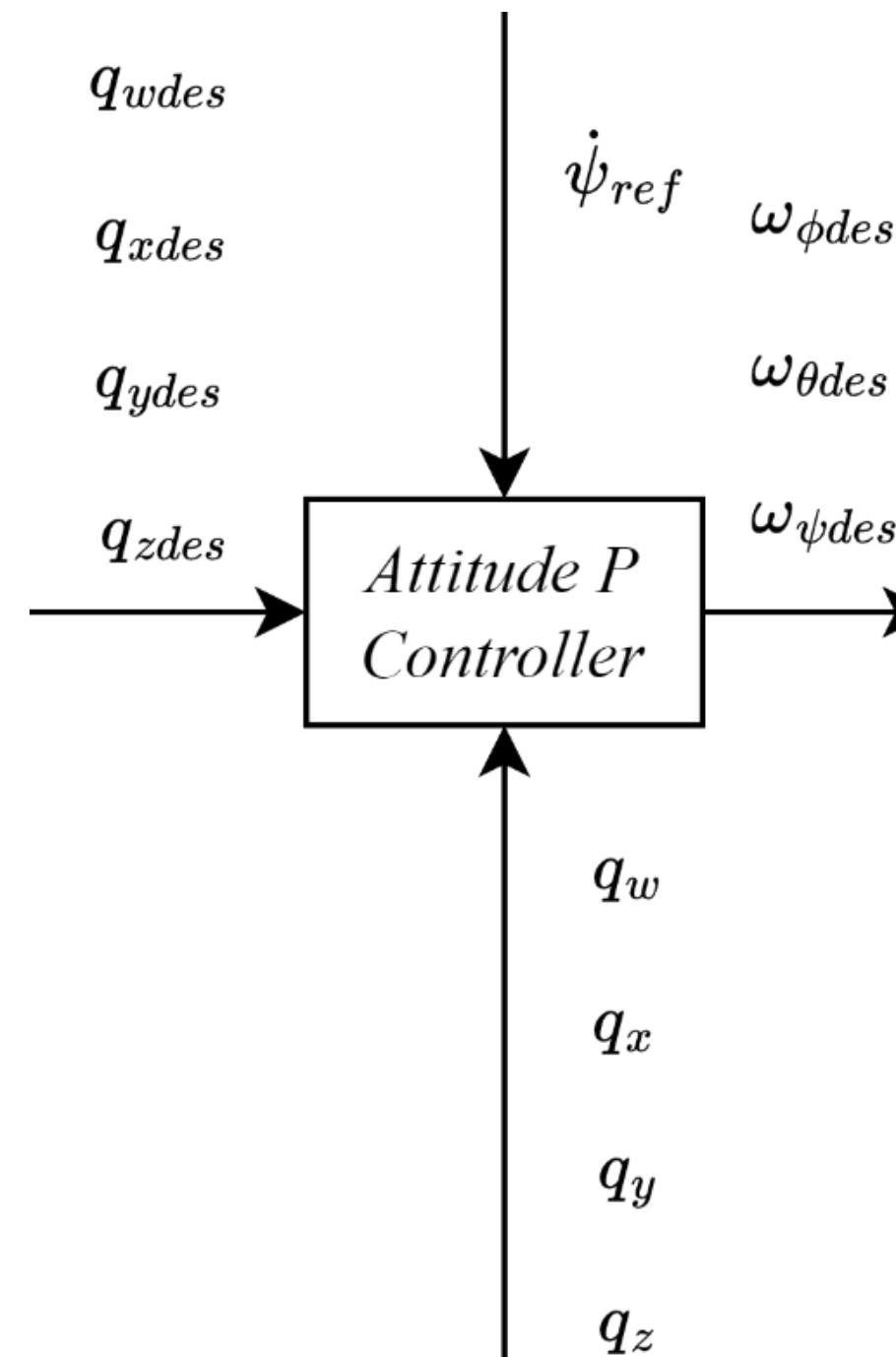
$$q_w = 0.5 \sqrt{1 + R_{11} + R_{22} + R_{33}}$$

$$q_x = \frac{R_{32} - R_{23}}{4 q_w}$$

$$q_y = \frac{R_{13} - R_{31}}{4 q_w}$$

$$q_z = \frac{R_{21} - R_{12}}{4 q_w}$$

$$\mathbf{q}_{des} = \frac{\mathbf{q}}{\|\mathbf{q}\|}$$

Arsitektur
Sistem

$$\mathbf{e}_q(t) = \mathbf{q}(t)^{-1} \otimes \mathbf{q}_{des}(t)$$

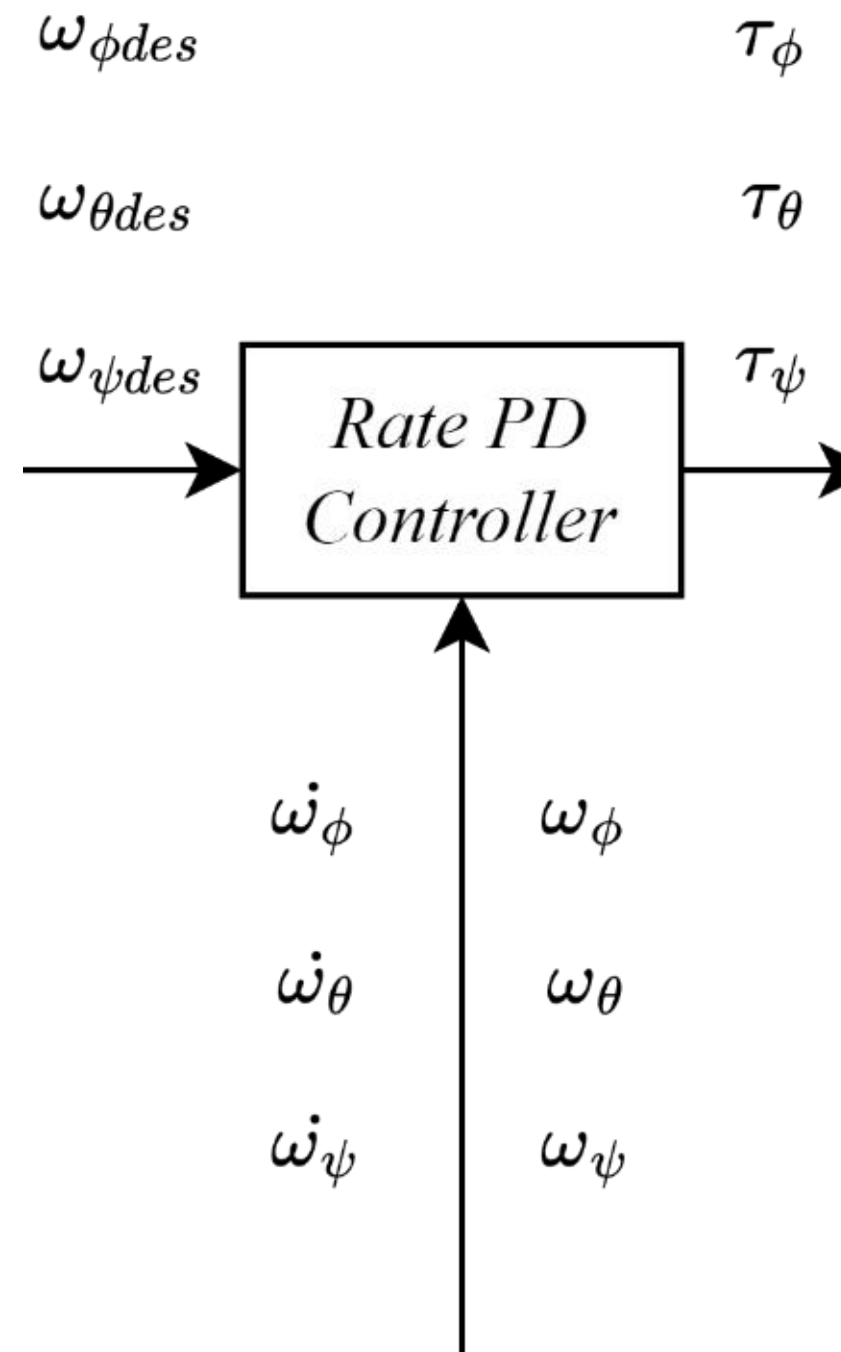
$$\boldsymbol{\omega}_{ref}(t) = \mathbf{K}_{p,att} \cdot \text{vect}(\mathbf{e}_q(t)) + \mathbf{R}(\mathbf{q})^{-1} \begin{bmatrix} 0 \\ 0 \\ \dot{\psi}_{ref} \end{bmatrix}$$

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_y q_z - q_w q_x) \\ 2(q_x q_y - q_w q_z) & 2(q_y q_z + q_w q_x) & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}$$

Metrik
KinerjaImplementasi
Software

Arsitektur
Sistem

Metrik
Kinerja

Implementasi
Software


$$\mathbf{e}_{\omega}(t) = \boldsymbol{\omega}_{ref}(t) - \boldsymbol{\omega}(t)$$

$$\boldsymbol{\tau}_{cmd}(t) = \mathbf{K}_{p,rate} \mathbf{e}_{\omega}(t) - \mathbf{K}_{d,rate} \dot{\boldsymbol{\omega}}(t)$$

$$\boldsymbol{\tau}_{cmd} = [\tau_{\phi}, \tau_{\theta}, \tau_{\psi}]^T$$

Arsitektur Sistem

Metrik Kinerja

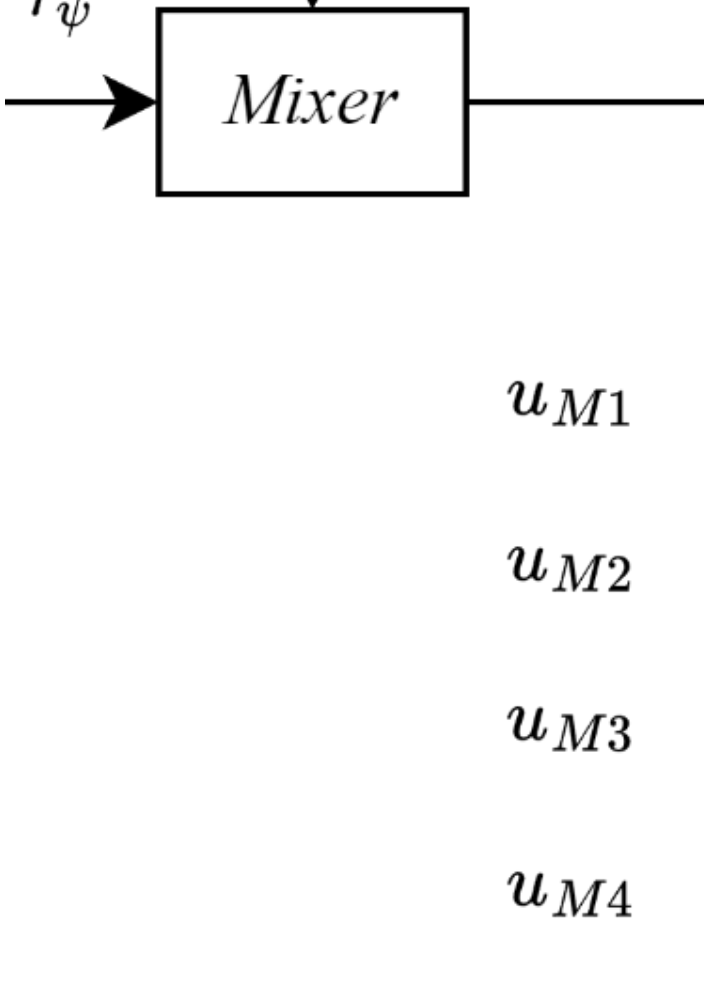
Implementasi Software

$$T = \sqrt{T_x^2 + T_y^2 + T_z^2}$$

τ_ϕ

τ_θ

τ_ψ



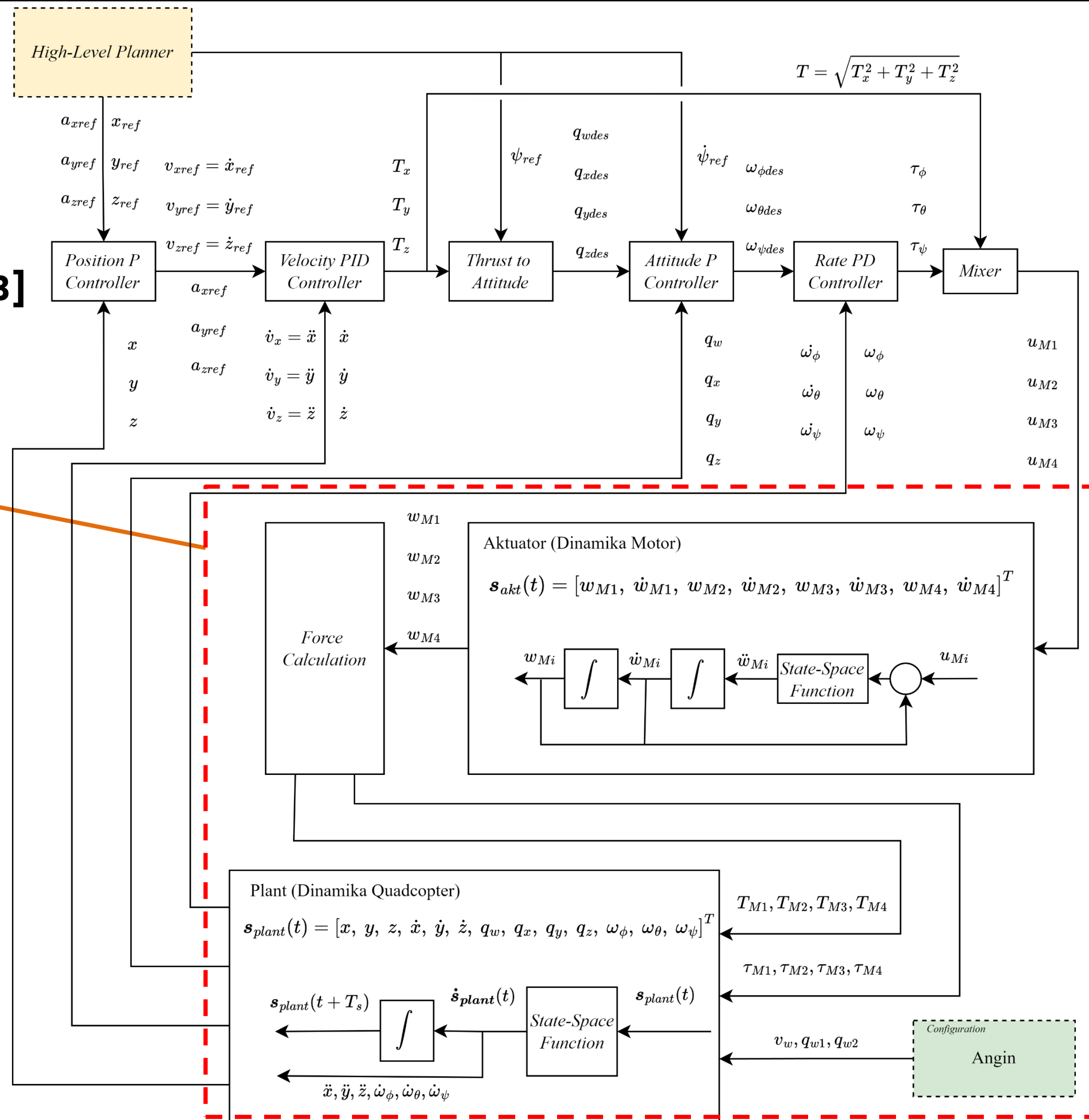
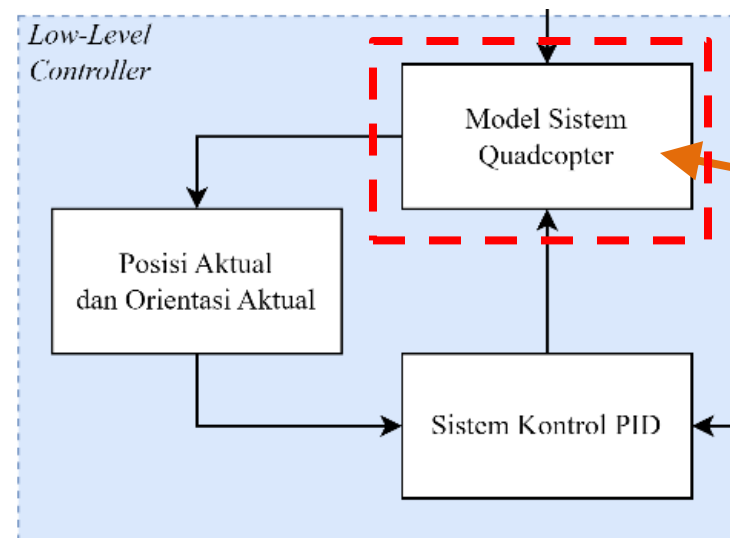
$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \mathbf{M}_{mix}^{-1} \begin{bmatrix} T_{sp} \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix}$$

$$\omega_{cmd,i} = \sqrt{\omega_i^2}$$

$$u_{Mi} = \frac{\omega_{cmd,i} - c_0}{c_1}$$

$$\mathbf{M}_{mix} = \begin{bmatrix} k_{Th} & k_{Th} & k_{Th} & k_{Th} \\ d_{ym}k_{Th} & -d_{ym}k_{Th} & -d_{ym}k_{Th} & d_{ym}k_{Th} \\ d_{xm}k_{Th} & d_{xm}k_{Th} & -d_{xm}k_{Th} & -d_{xm}k_{Th} \\ -k_{To} & k_{To} & -k_{To} & k_{To} \end{bmatrix}$$

Low-Level Controller [3]

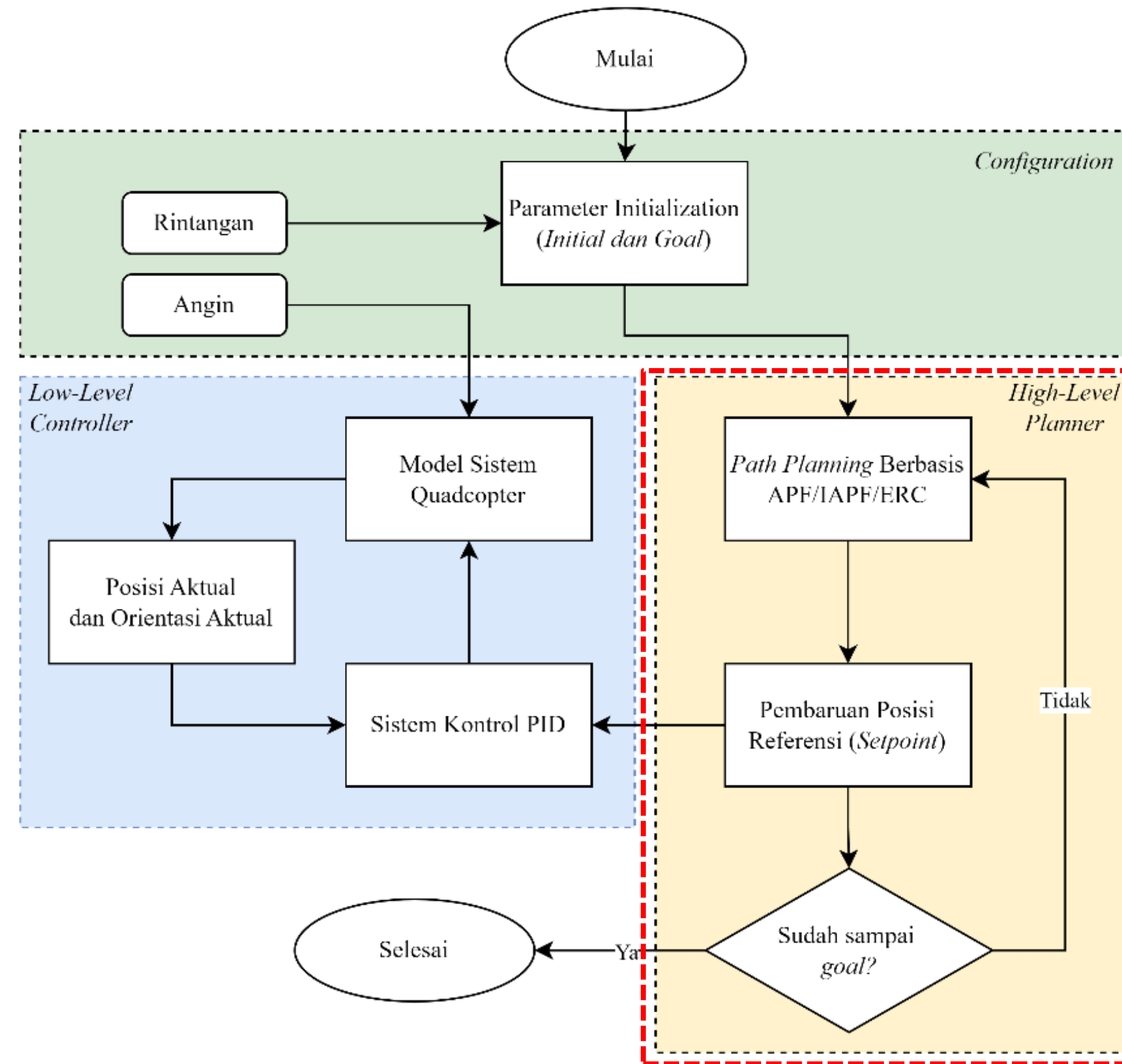


[3] John, bobzwik/Quadcopter_SimCon. (Jun. 10, 2025). Python. Accessed: Jun.13, 2025. [Online]. Available: https://github.com/bobzwik/Quadcopter_SimCon

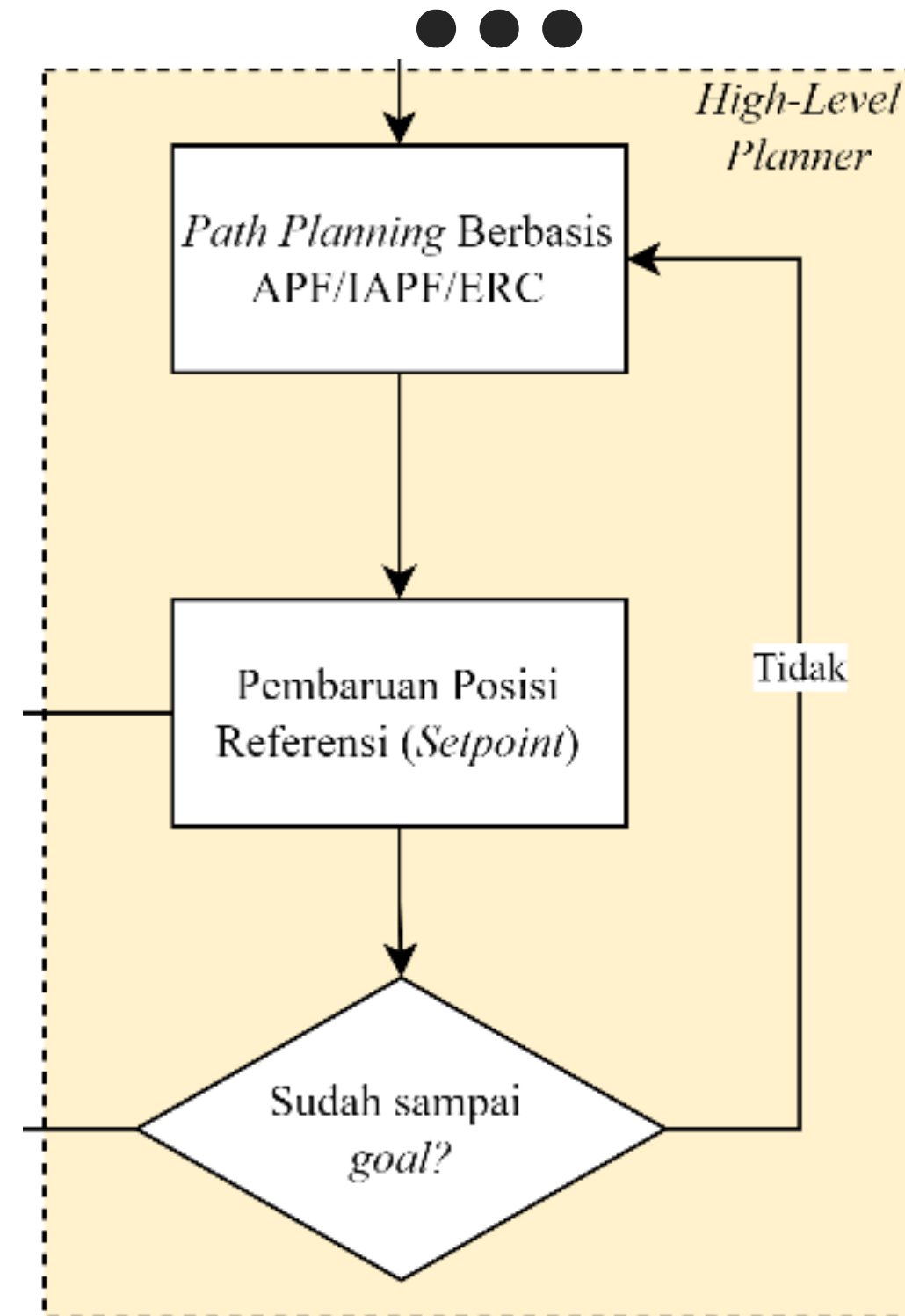
Arsitektur Sistem

Metrik Kinerja

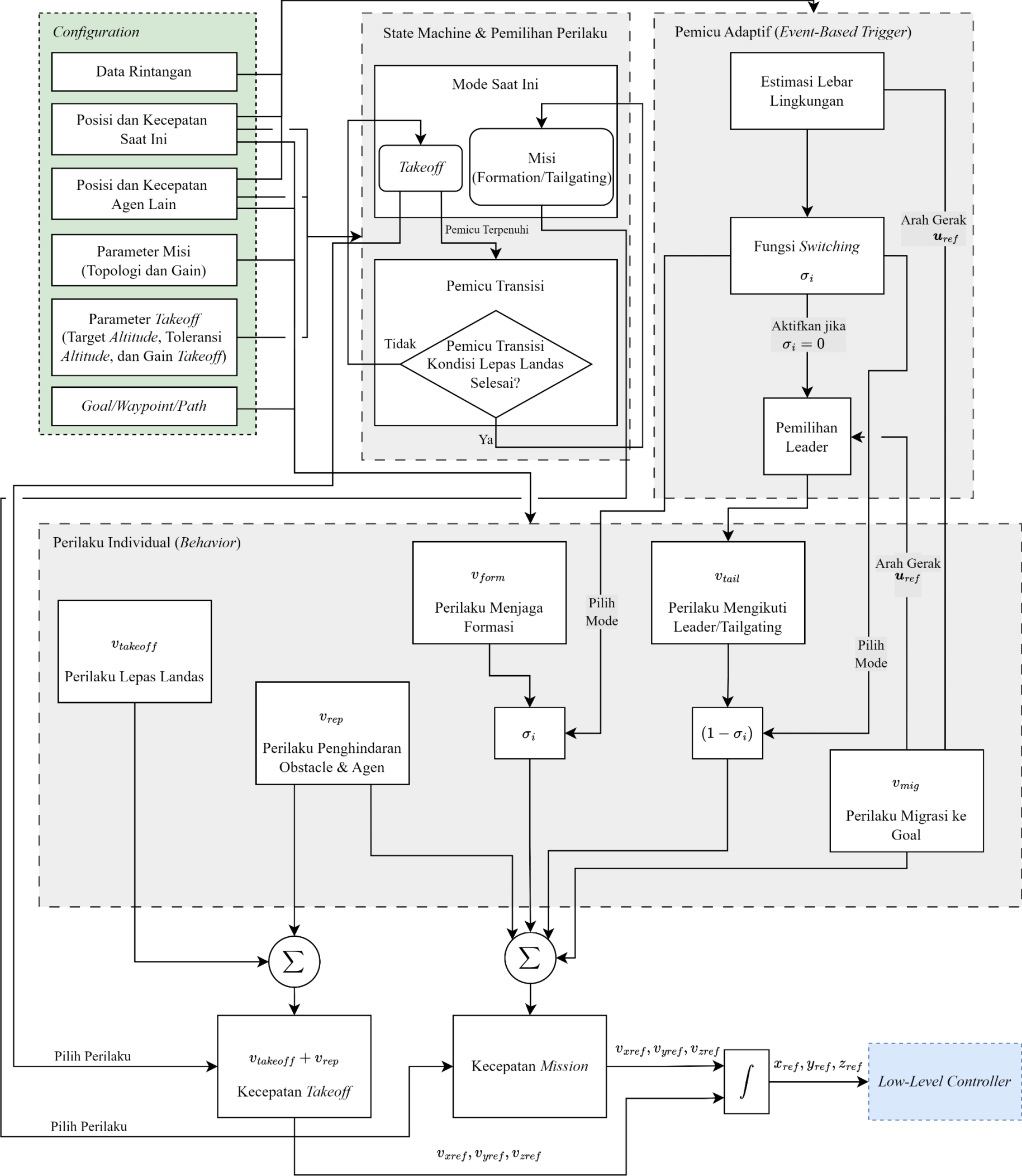
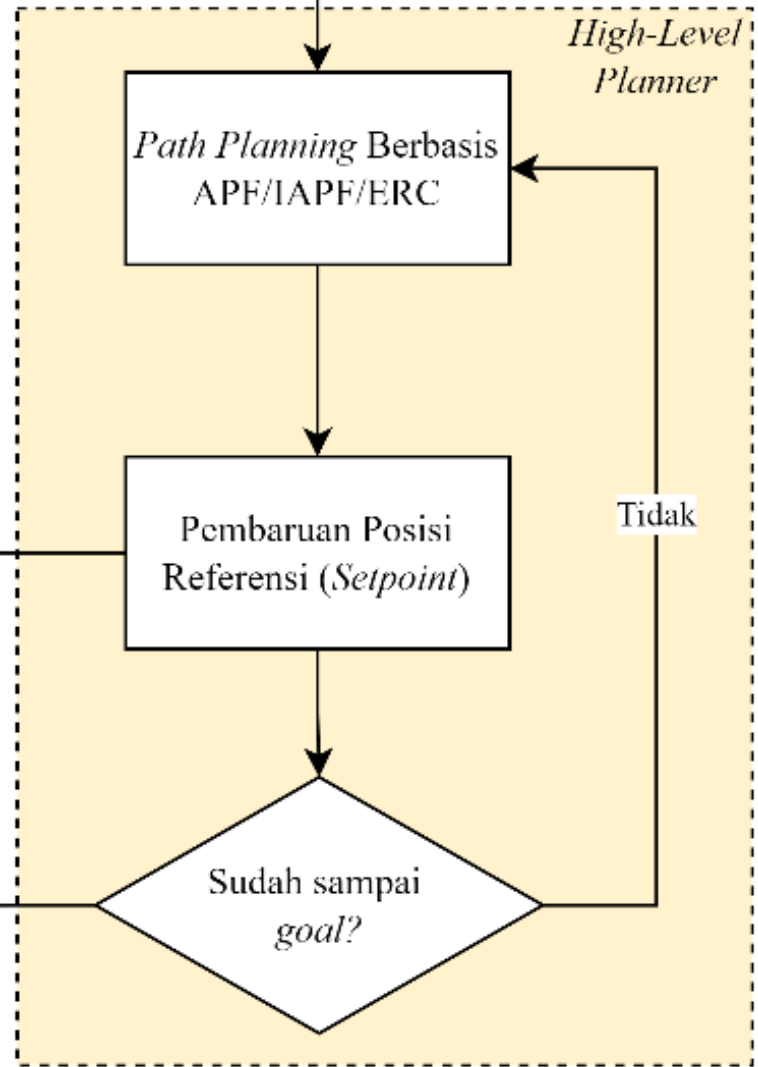
Implementasi Software



High-Level Planner [13]



High-Level Planner[13]



[13] D.-N. Bui, M. D. Phung, and H. P. Duy, "Event-based Reconfiguration Control for Time-varying Formation of Robot Swarms in Narrow Spaces," *Intell. Serv. Robot.*, May 2025, doi: 10.1007/s11370-025-00607-z.

Arsitektur Sistem

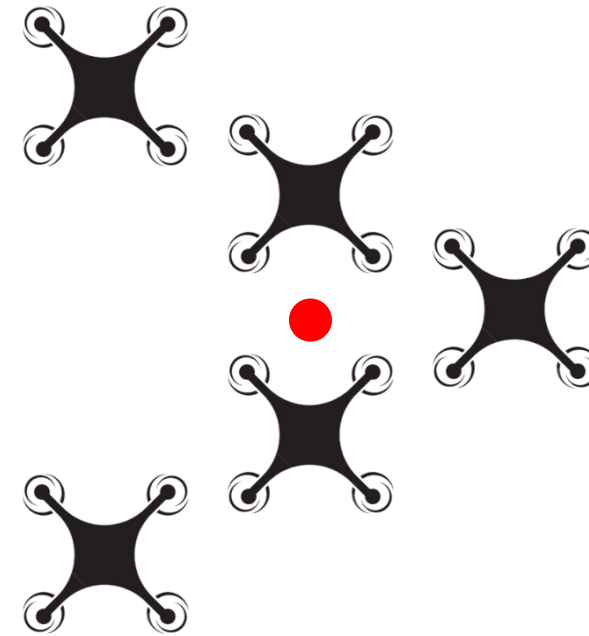
Metrik Kinerja

Implementasi Software

Configuration

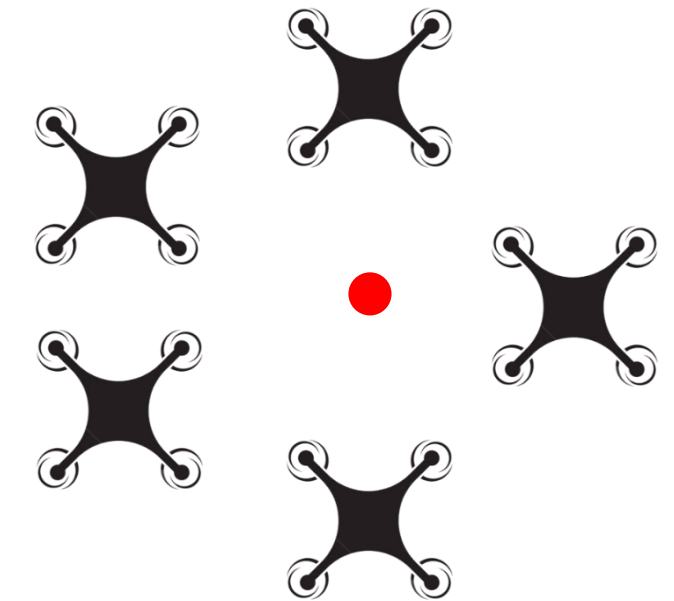
Data Rintangan
Posisi dan Kecepatan Saat Ini
Posisi dan Kecepatan Agen Lain
Parameter Misi (Topologi dan Gain)
Parameter <i>Takeoff</i> (Target <i>Altitude</i> , Toleransi <i>Altitude</i> , dan Gain <i>Takeoff</i>)
Goal/Waypoint/Path

V-shape



$$\begin{bmatrix} \delta_1^* \\ \delta_2^* \\ \delta_3^* \\ \delta_4^* \\ \delta_5^* \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ -1.0 & 1.0 & 0.0 \\ 0.0 & 0.5 & 0.0 \\ 0.0 & -0.5 & 0.0 \\ -1.0 & -1.0 & 0.0 \end{bmatrix}$$

Poligon



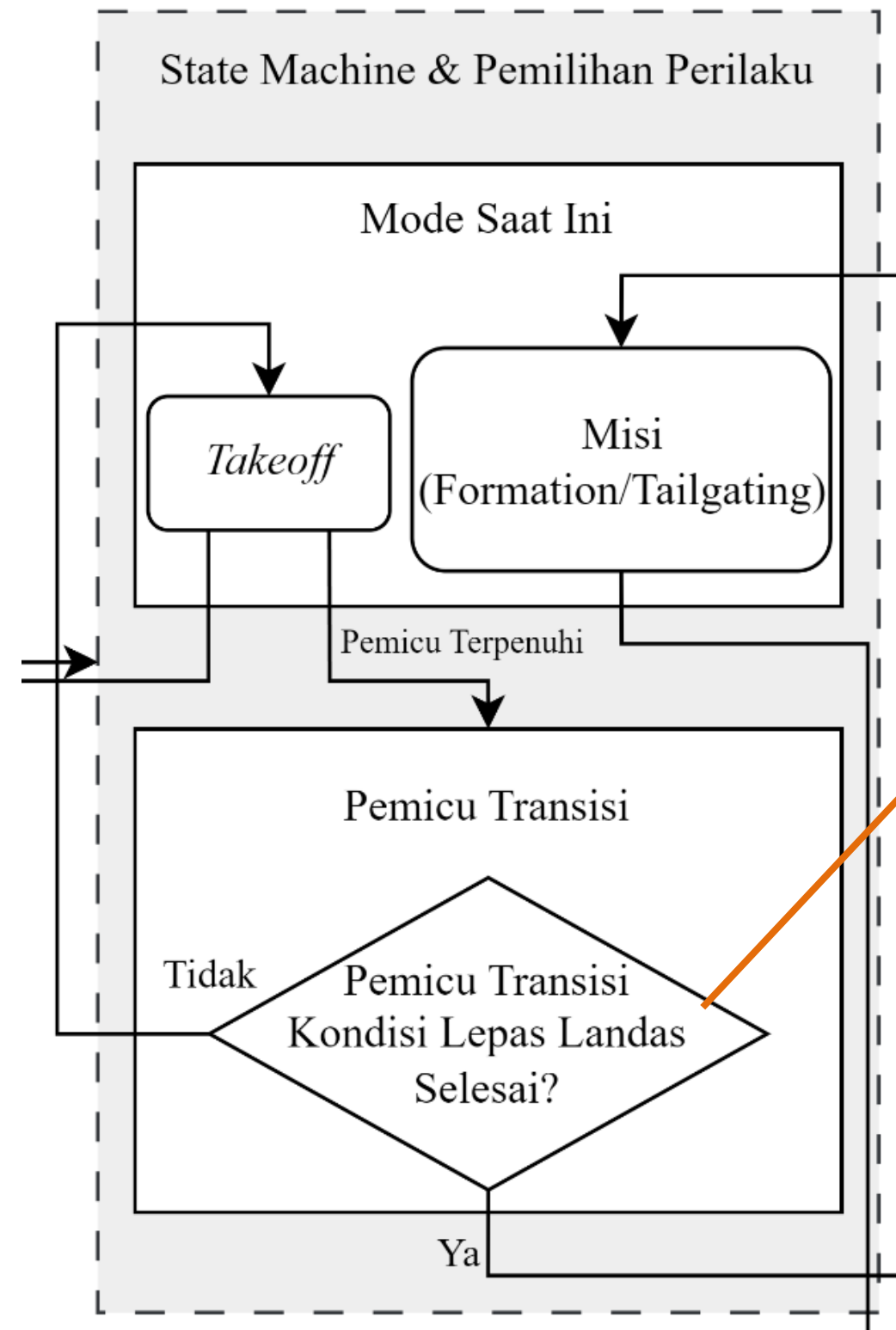
$$N = 5$$

$$\delta_i^* = \left[\cos\left(\frac{2\pi i}{N}\right) \quad \sin\left(\frac{2\pi i}{N}\right) \quad 0 \right]$$

Arsitektur Sistem

Metrik Kinerja

Implementasi
Software

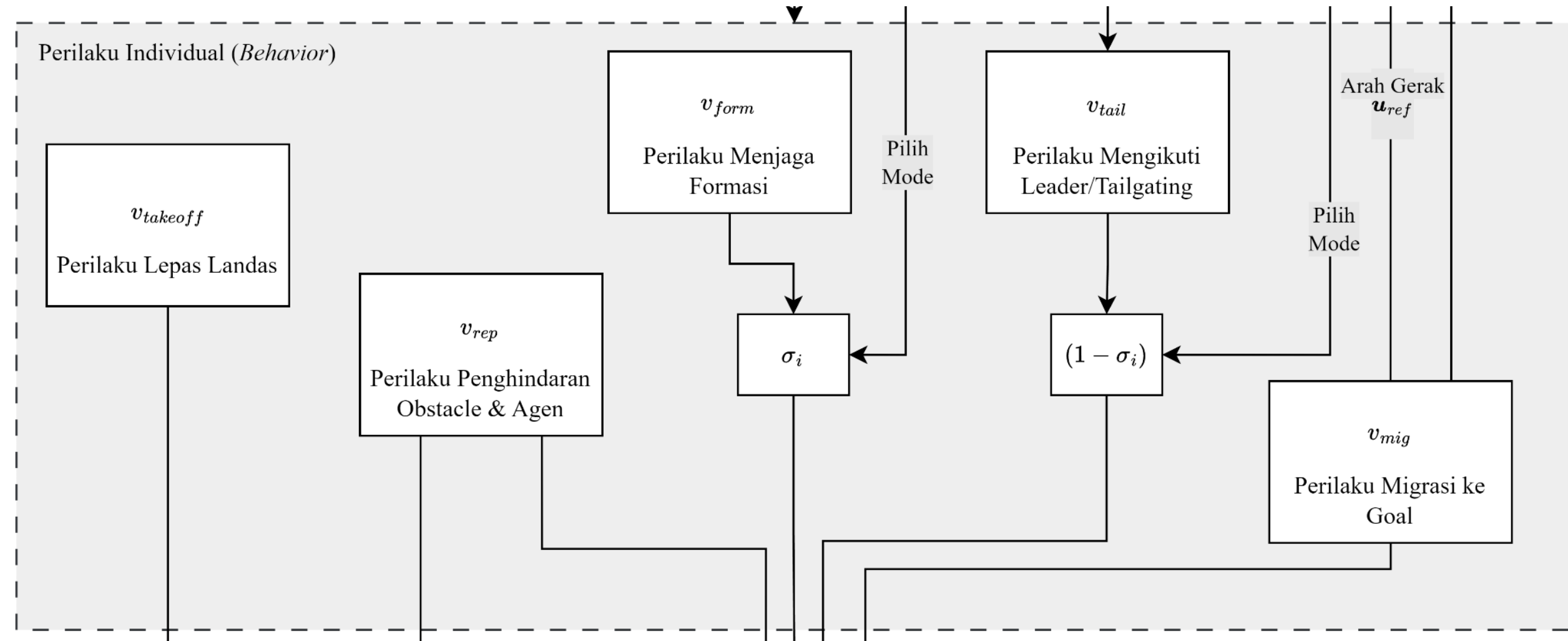


$$|z_i(t) - z_{target}| \leq \epsilon_z, \quad \forall i \in \{1, \dots, 5\}$$

Arsitektur Sistem

Metrik Kinerja

Implementasi Software



Arsitektur Sistem

Metrik Kinerja

Implementasi Software

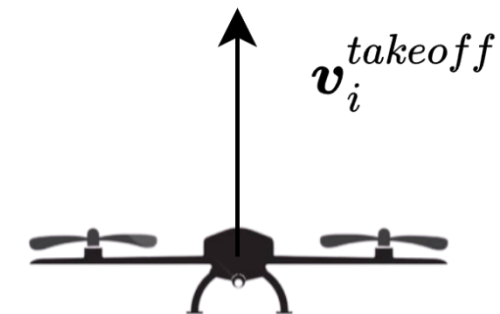
$v_{takeoff}$
Perilaku Lepas Landas

$p_{target,takeoff}(x_i, y_i, z_{target})$



$$v_{z,des}^{takeoff} = k_{p,takeoff} (z_{target} - z_i(t))$$

$$\mathbf{v}_i^{takeoff} = \begin{bmatrix} 0.0 \\ 0.0 \\ v_{z,des}^{takeoff} \end{bmatrix}$$



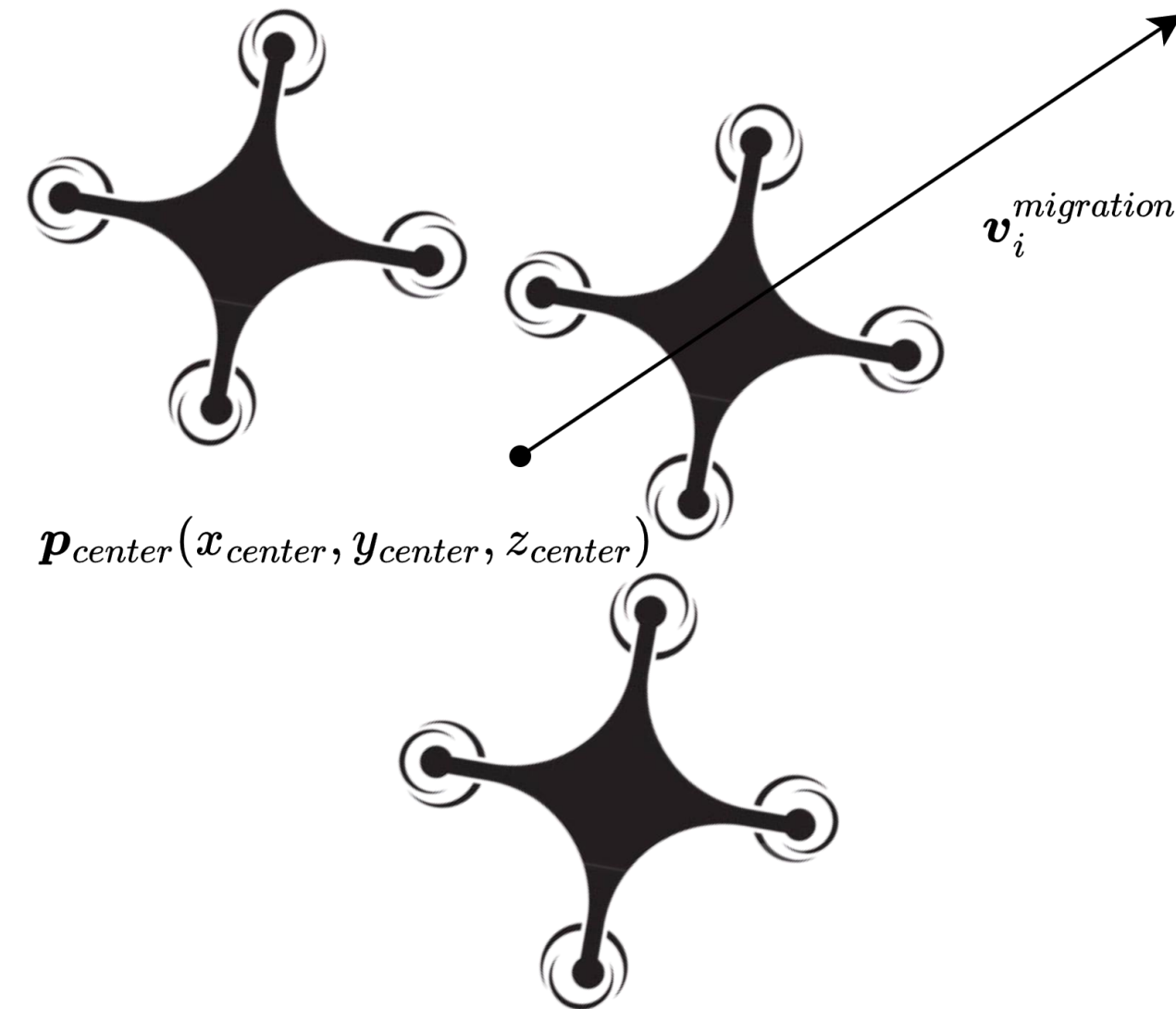
$p_i(x_i, y_i, z_i)$

$$\mathbf{p}_{goal}(x_{goal}, y_{goal}, z_{goal})$$



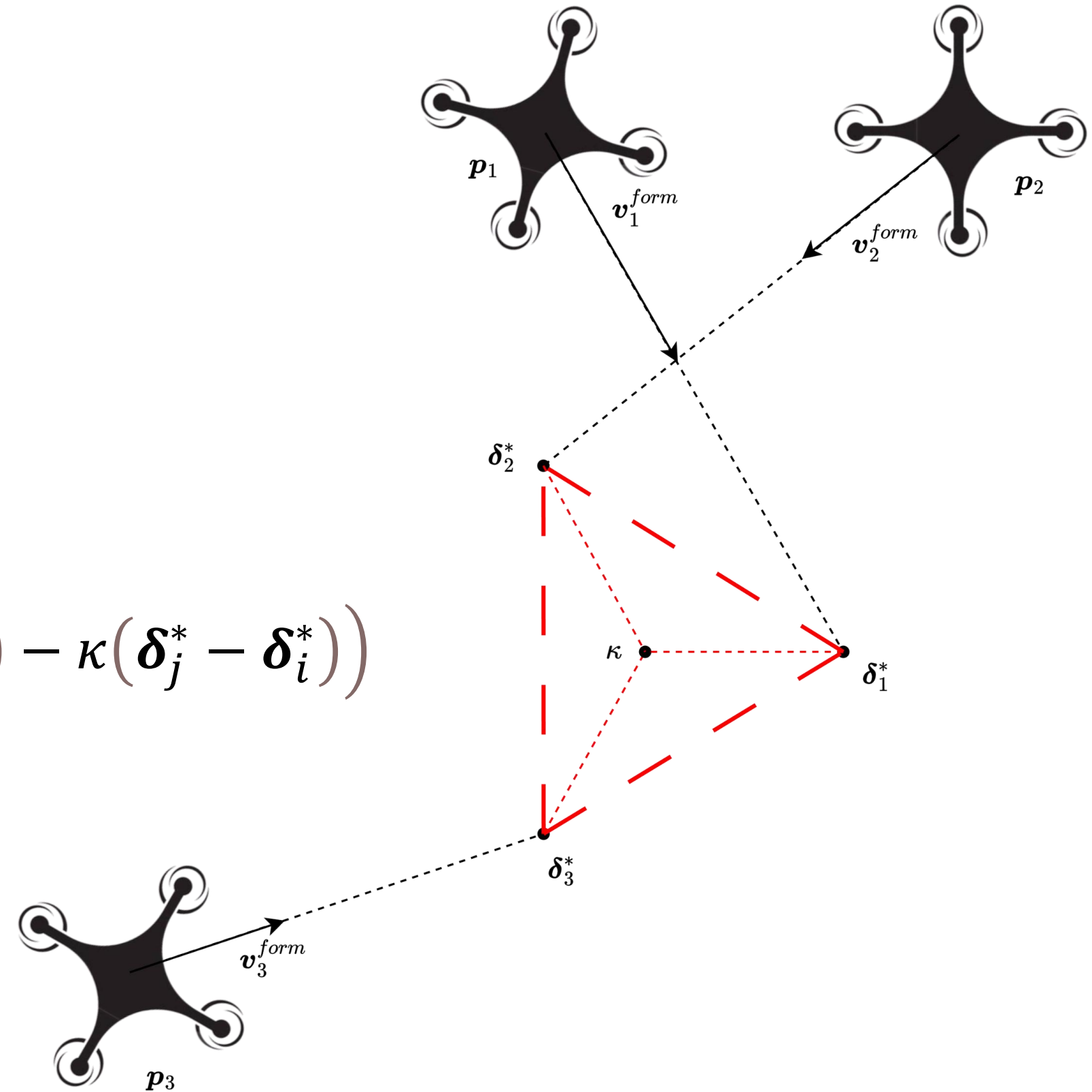
$$\mathbf{v}_i^{mig} = speed \cdot \mathbf{u}_{ref}$$

$$\mathbf{u}_{ref} = \frac{\mathbf{p}_{goal} - \mathbf{p}_{center}}{\|\mathbf{p}_{goal} - \mathbf{p}_{center}\|}$$





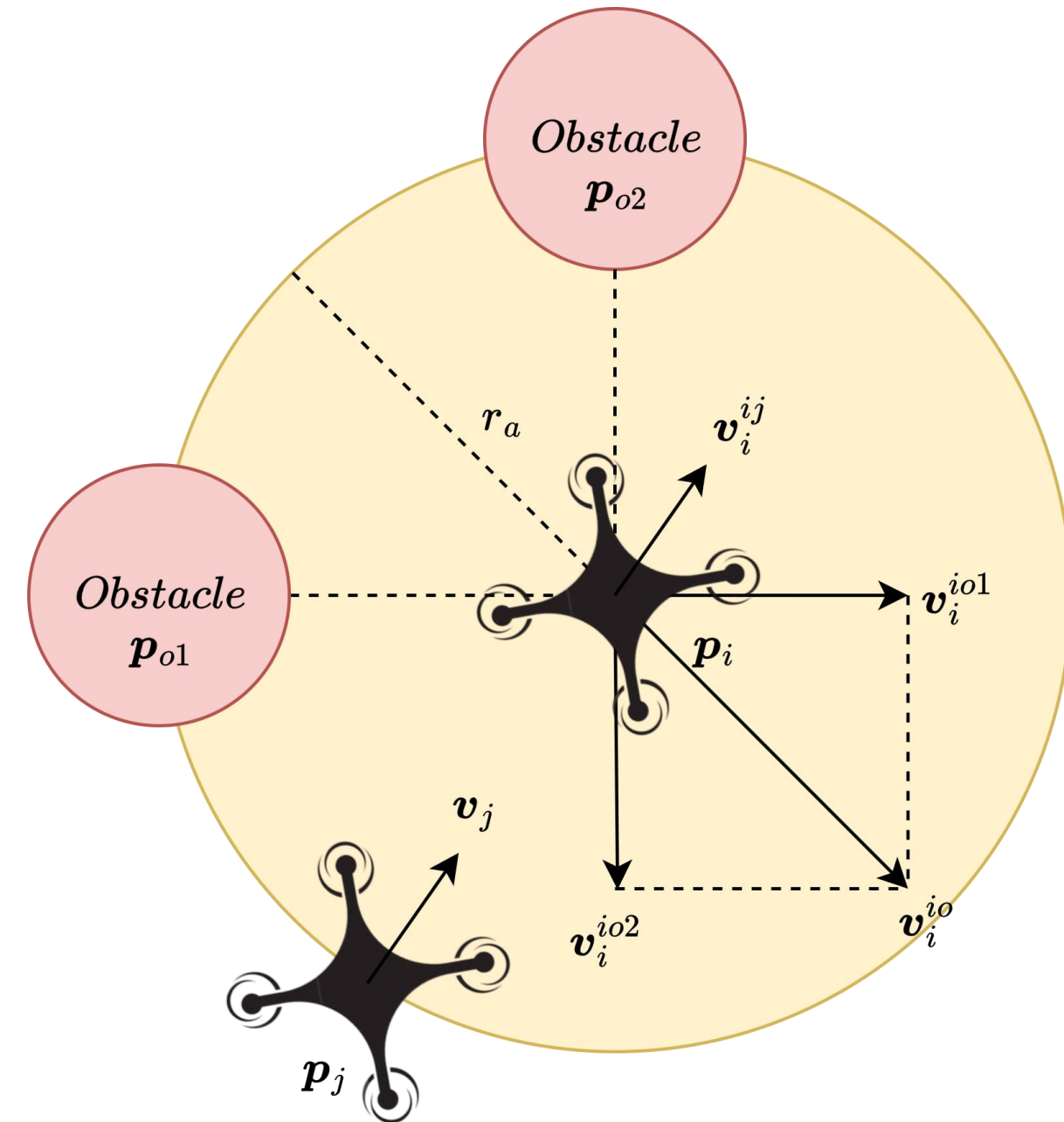
$$v_i^{form} = k_{form} \sum_{j=1, j \neq i}^N \left((p_j - p_i) - \kappa (\delta_j^* - \delta_i^*) \right)$$

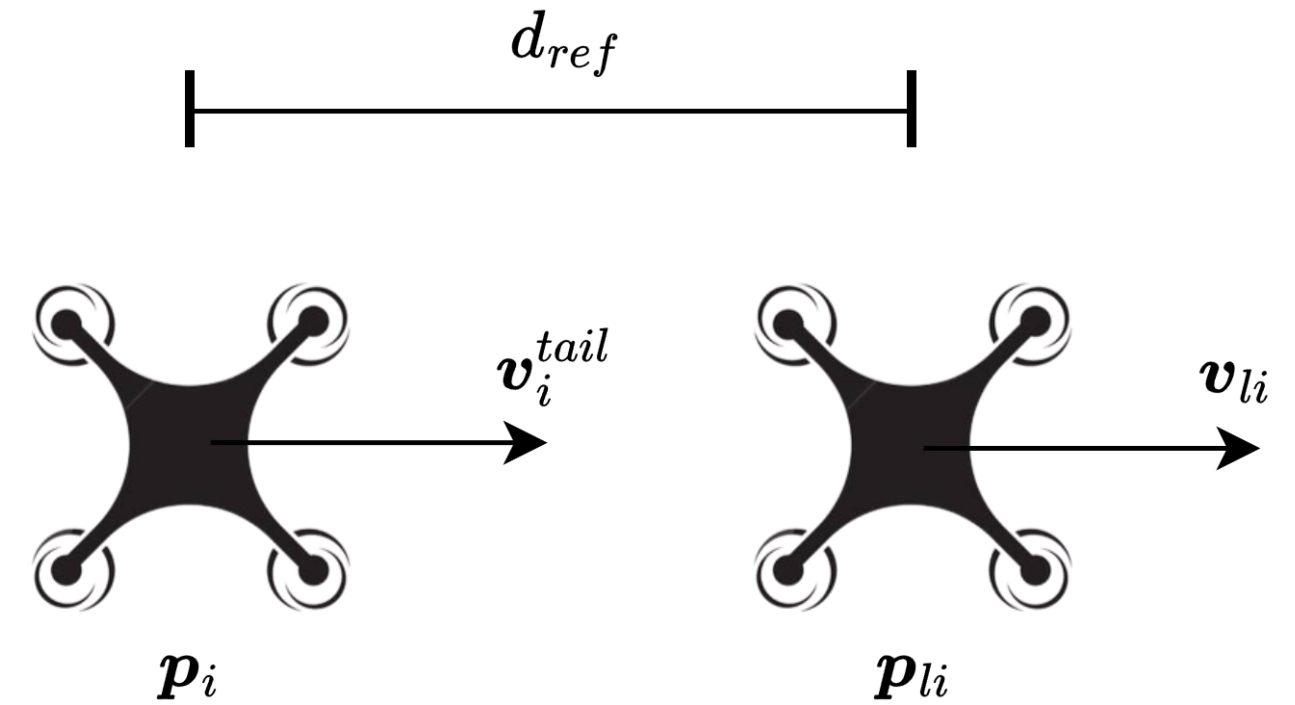
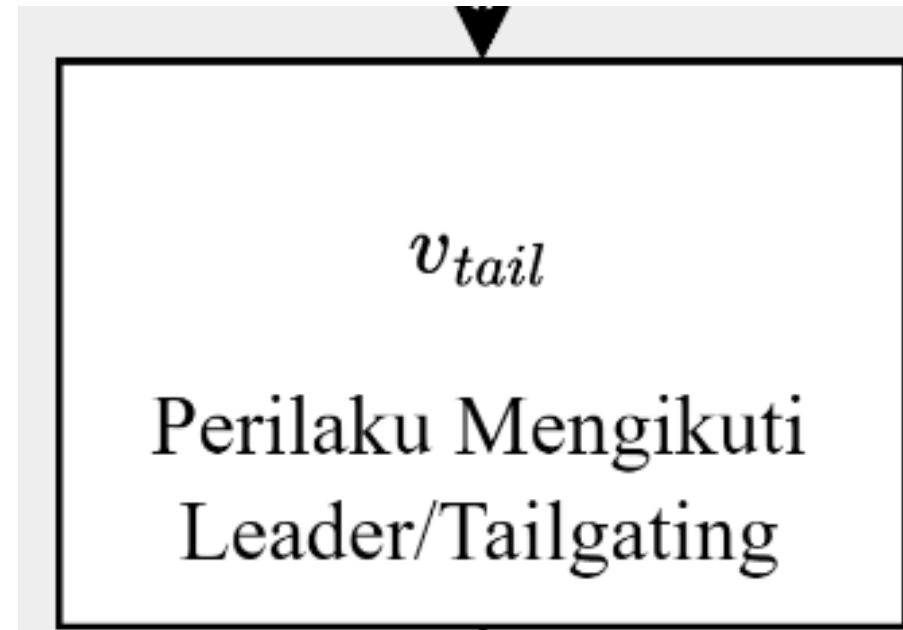


v_{rep} Perilaku Penghindaran
Obstacle & Agen

$$\mathbf{v}_i^{ij} = k_{col} \left(\frac{1}{\|\mathbf{p}_{ij}\|} - \frac{1}{r_a} \right) \frac{1}{\|\mathbf{p}_{ij}\|^2} \hat{\mathbf{p}}_{ij} + \mathbf{v}_j$$

$$\mathbf{v}_i^{io} = k_{obs} \left(\frac{1}{\|\mathbf{p}_{io}\|} - \frac{1}{r_a} \right) \frac{1}{\|\mathbf{p}_{io}\|^2} \hat{\mathbf{p}}_{io}$$



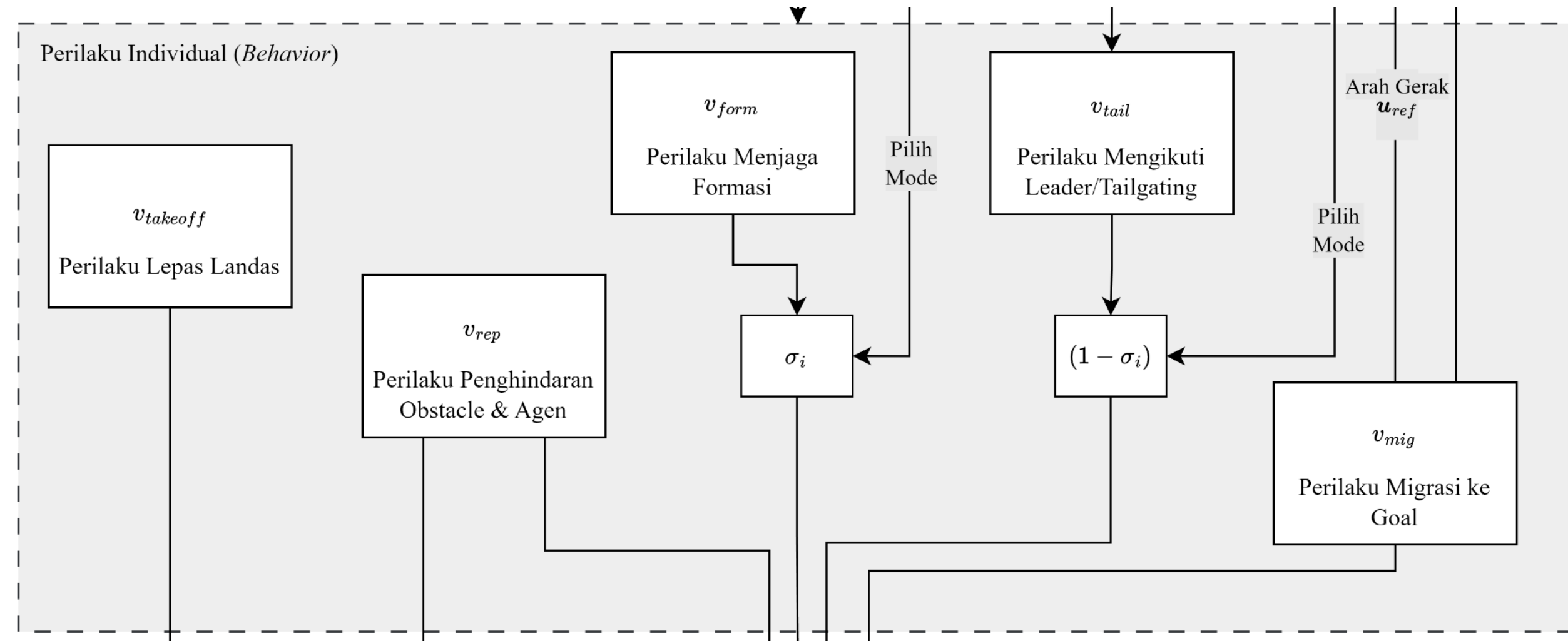


$$v_i^{tail} = \begin{cases} k_{tail} \left((p_{li} - p_i) - d_{ref} \frac{v_{li}}{\|v_{li}\|} \right) + v_{li}, & \text{jika } l_i \neq -1 \\ 0, & \text{jika } l_i = -1 \end{cases}$$

Arsitektur Sistem

Metrik Kinerja

Implementasi Software

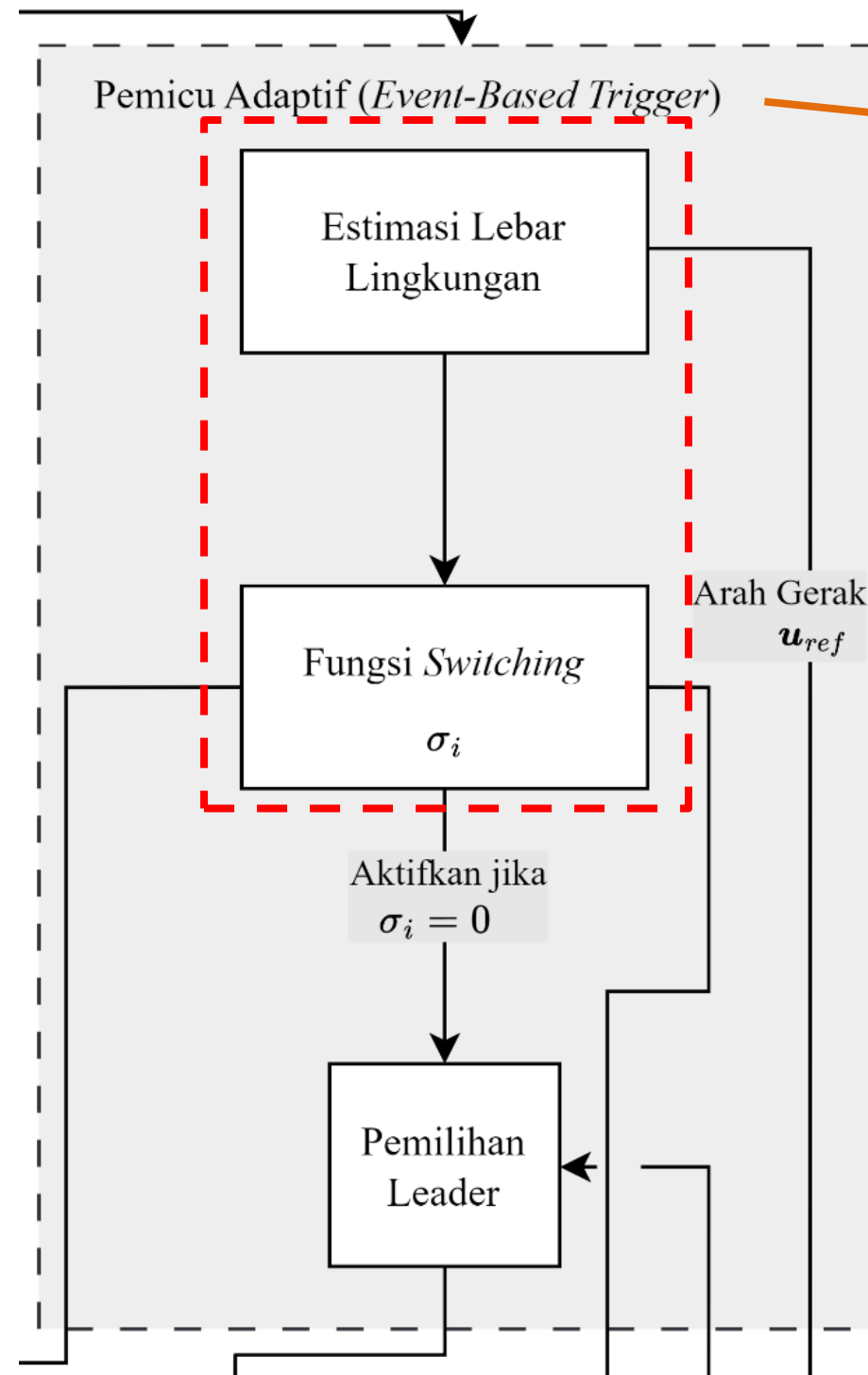


$$v_{ref}^{IAPF} = v_i^{mig} + v_i^{io} + v_i^{ij} + v_i^{form}$$

Arsitektur Sistem

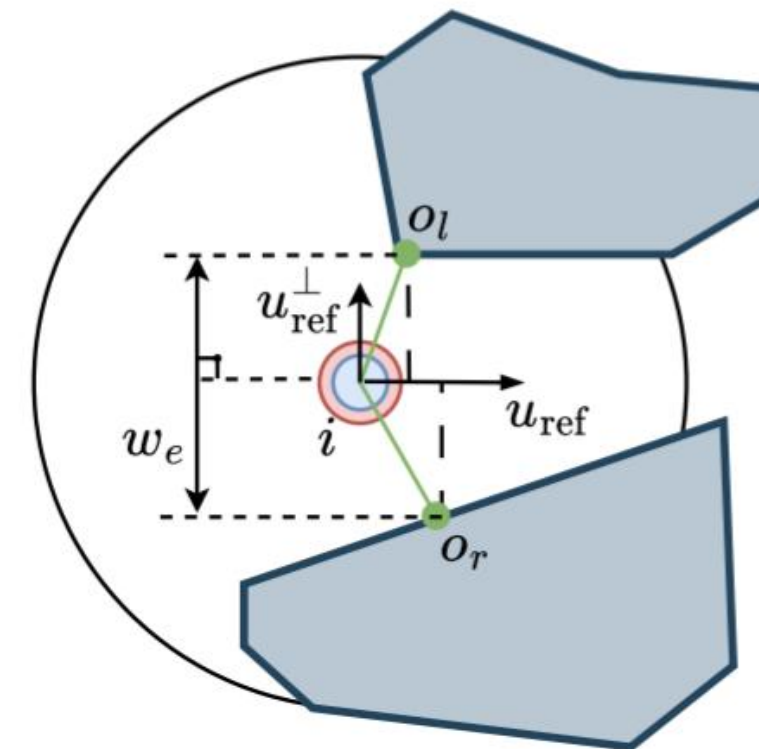
Metrik Kinerja

Implementasi Software



$$\mathbf{v}_{ref}^{ERC} = \mathbf{v}_i^{mig} + \mathbf{v}_i^{io} + \mathbf{v}_i^{ij} + \sigma_i \mathbf{v}_i^{form} + (1 - \sigma_i) \mathbf{v}_i^{tail}$$

$$\sigma_i = 1 \text{ (Mode Formation)}$$



$$w_e = |\langle (\mathbf{o}_r - \mathbf{o}_l), \mathbf{u}_{ref}^\perp \rangle|$$

$$w_e < w_f + 2r$$

$$\kappa = \frac{w_e - 2r}{w_f}$$

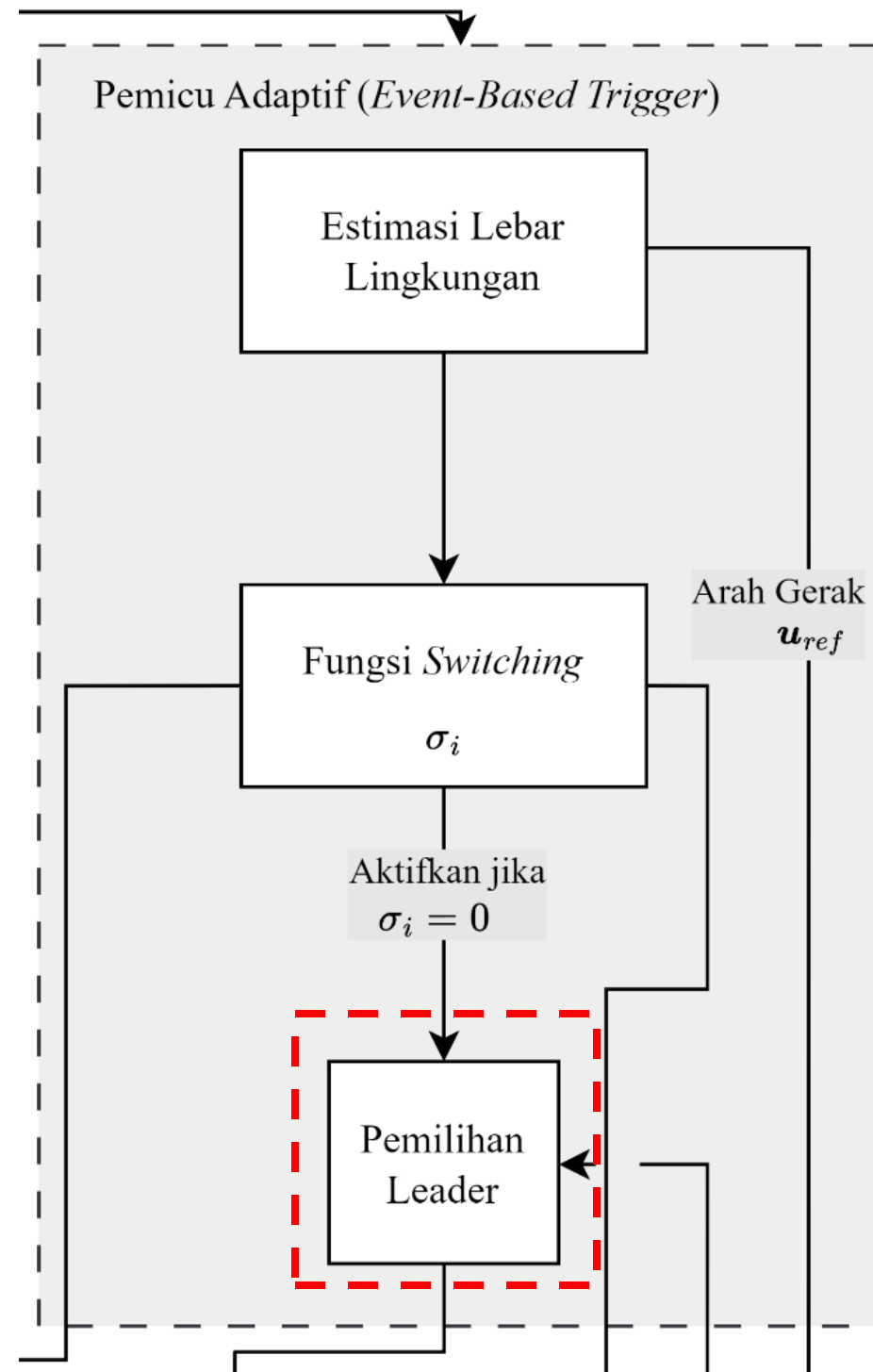
$$w_e \leq \alpha \cdot r$$

$$\sigma_i = 0 \text{ (Mode Tailgating)}$$

Arsitektur Sistem

Metrik Kinerja

Implementasi Software



$$\tilde{p}_{ij} = \langle (\mathbf{p}_j - \mathbf{p}_i), \mathbf{u}_{ref} \rangle$$

Agen j dianggap berada di depan agen i jika nilai $\tilde{p}_{ij} \geq 0$.

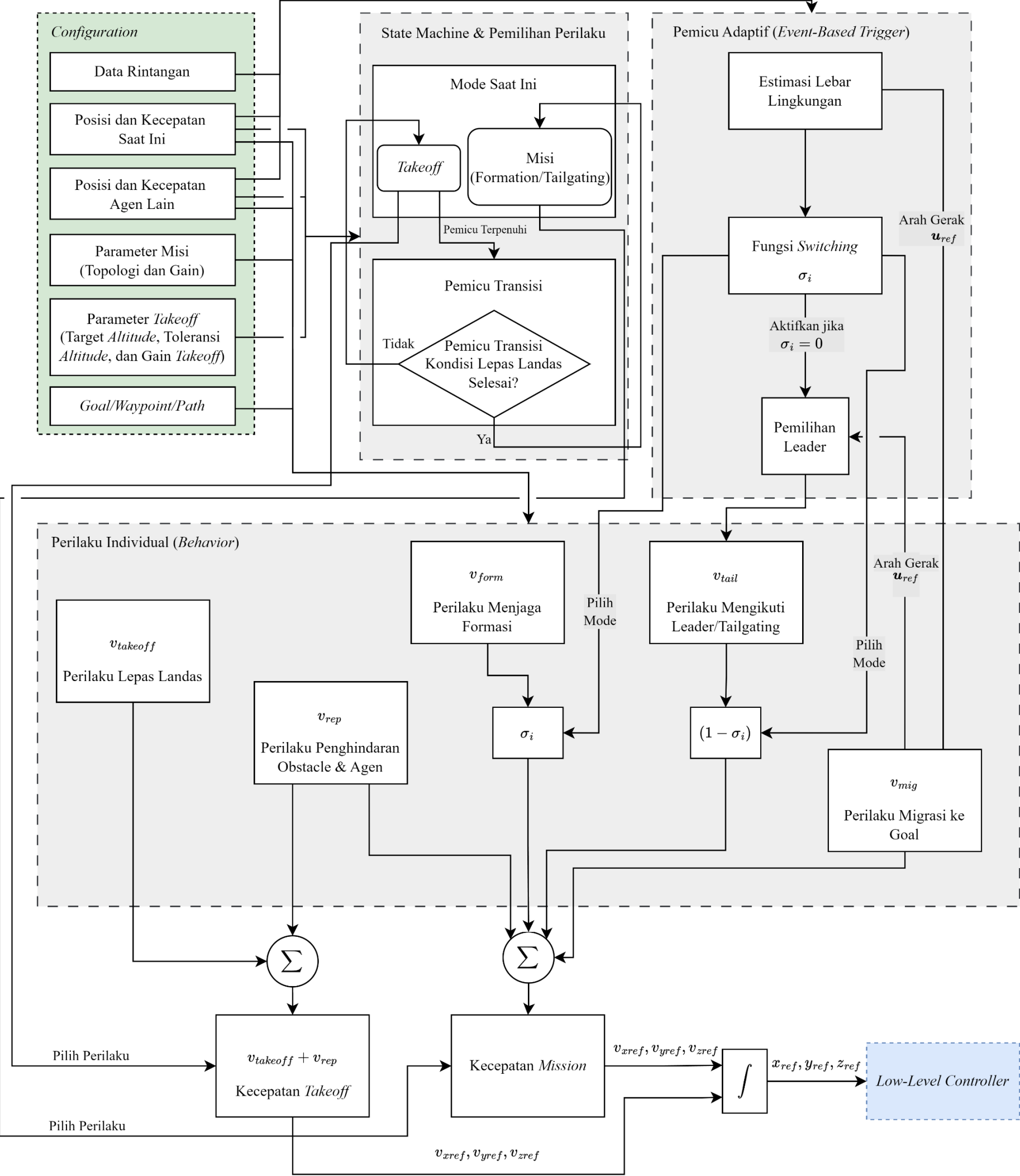
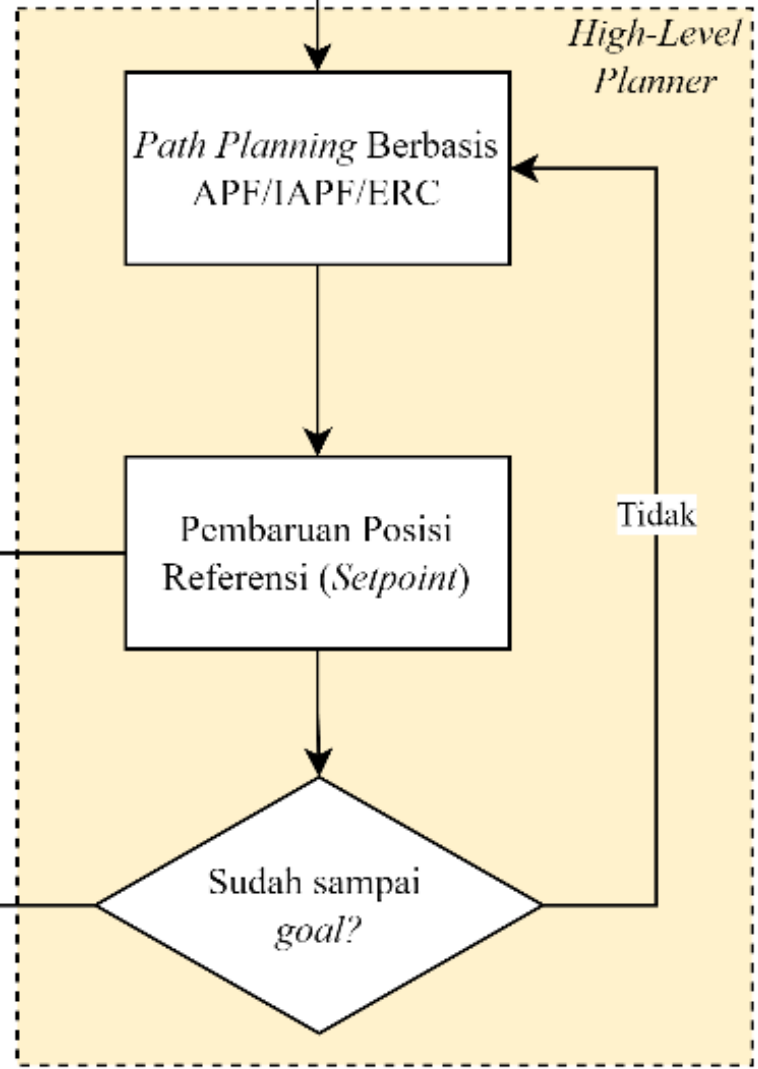
Dari semua agen yang berada di depan, agen yang dipilih sebagai pemimpin (l_i) adalah yang memiliki nilai $\tilde{p}_{ij}$ terkecil.

$$l_i = \begin{cases} \arg \min_{j: \tilde{p}_{ij} > 0} \tilde{p}_{ij}, & \text{jika } \{j | \tilde{p}_{ij} \geq 0\} \neq \{\} \\ -1, & \text{lainnya} \end{cases}$$

$$\|\mathbf{v}_{ref}\| < \varepsilon_v$$

$$\mathbf{v}_{ref} = [0, 0, 0]$$

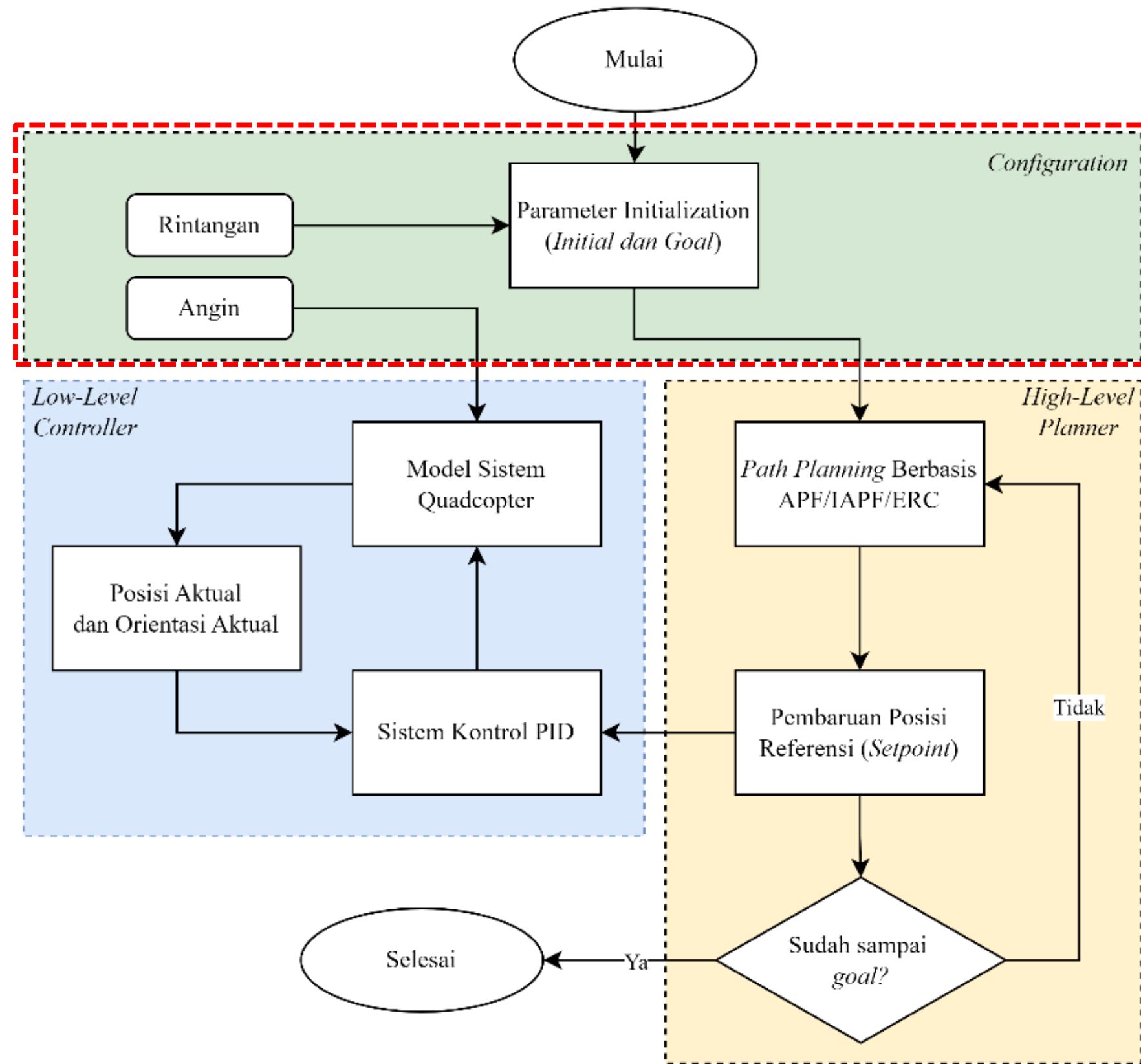
$$\mathbf{p}_{ref} += \mathbf{v}_{ref} \times dt$$



Arsitektur Sistem

Metrik Kinerja

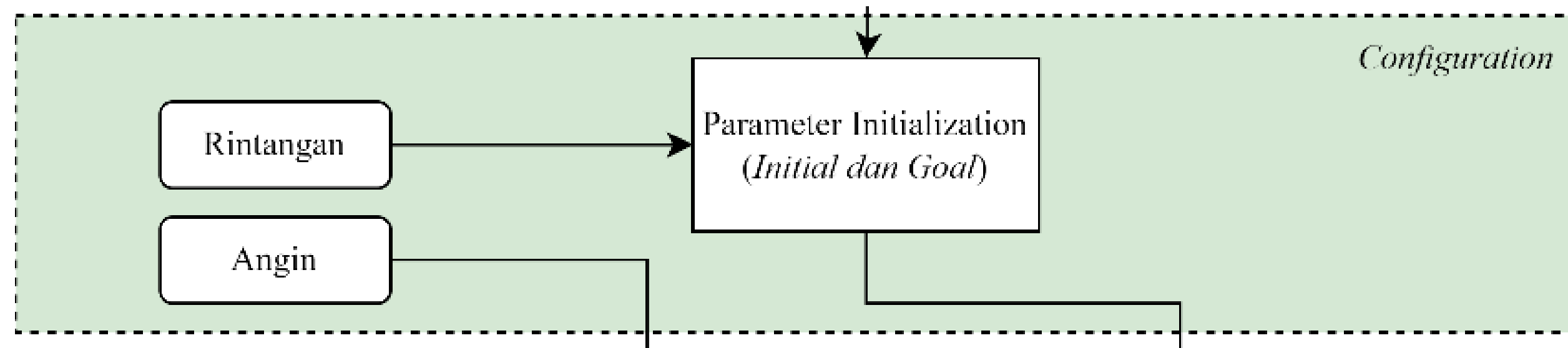
Implementasi Software



Arsitektur Sistem

Metrik Kinerja

Implementasi Software



Arsitektur
SistemMetrik
KinerjaImplementasi
Software

Rintangan

Angin

Uji Penghindaran Dasar, menggunakan rintangan yang tersebar untuk memvalidasi kemampuan manuver dasar.

Uji Celah Sempit, secara spesifik dirancang untuk membandingkan strategi IAPF yang kaku (diprediksi gagal) dengan strategi ERC yang adaptif (diharapkan berhasil) saat melewati ruang gerak terbatas.

Gangguan angin dimodelkan sebagai hembusan angin sesaat (***gust***) yang bersifat periodik dan pulsatif untuk memberikan tantangan dinamis pada sistem kontrol.

Arsitektur
Sistem

Metrik
Kinerja

Implementasi
Software



Dibahas lebih lanjut di bagian **Hasil & Analisis**

Metrik Akurasi Operasional → *Root Mean Squared Error (RMSE)*

$$RMSE_{takeoff}(t) = \sqrt{\frac{1}{N} \sum_{i=1}^N (z_i(t) - z_{target})^2}$$

$$RMSE_{formation}(t) = \sqrt{\frac{1}{M} \sum_{pasangan} \left(\| \mathbf{p}_{ij}(t) \| - \| \boldsymbol{\delta}_{ij}^* \| \right)^2}$$

$$RMSE_{tailgating}(t) = \sqrt{\frac{1}{N_f} \sum_{i \in followers} \left(\| \mathbf{p}_i(t) - \mathbf{p}_{li}(t) \| - d_{ref} \right)^2}$$

$$\overline{RMSE}_{total} = \frac{1}{K} \sum_{k=0}^K \left(RMSE_{takeoff}(t_k) + RMSE_{formation}(t_k) + RMSE_{tailgating}(t_k) \right)$$

Metrik Kekohesifan Gerak → Indeks Keteraturan (Φ)

$$\Phi(t) = \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=1, j \neq i}^N \frac{\langle \mathbf{v}_i(t), \mathbf{v}_j(t) \rangle}{\| \mathbf{v}_i(t) \| \| \mathbf{v}_j(t) \|}$$

$$\bar{\Phi} = \frac{1}{K} \sum_{k=1}^K \left(\Phi(t_k) \right)$$

```
graph TD; A[Pendahuluan] --> B[Arsitektur Sistem]; B --> C[Metrik Kinerja]; C --> D[Implementasi Software];
```

Pendahuluan

Arsitektur Sistem

Metrik Kinerja

Implementasi *Software*

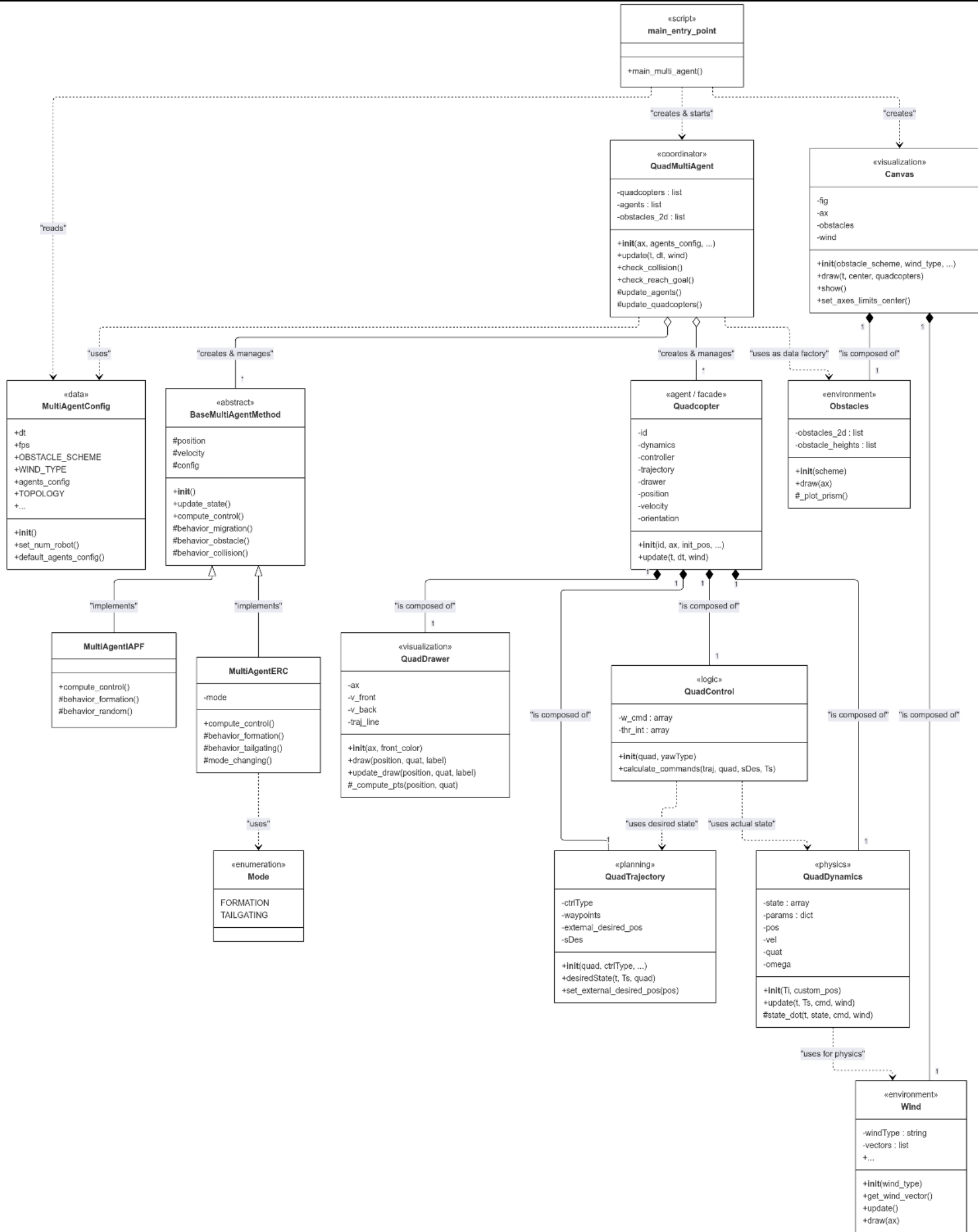
Metodologi

Arsitektur
Sistem

Metrik Kinerja

Implementasi *Software*

- ▼ MULTIAGENTSIM
 - > __pycache__
 - ▼ Agent
 - > __pycache__
 - ▼ ModelSystemQuad
 - 🔗 Quad_3D_flu_ENU_Quat.py
 - 🔗 Quad_3D_frd_NED_Quat.py
 - ▼ QuadUtils
 - > __pycache__
 - 🔗 __init__.py
 - 🔗 mixer.py
 - 🔗 quaternionFunctions.py
 - 🔗 rotationConversion.py
 - 🔗 stateConversions.py
 - 🔗 __init__.py
 - 🔗 MultiAgentConfig.py
 - 🔗 MultiAgentERC.py
 - 🔗 MultiAgentIAPF.py
 - 🔗 MultiAgentUtils.py
 - 🔗 QuadControl.py
 - 🔗 Quadcopter.py
 - 🔗 QuadDrawer.py
 - 🔗 QuadDynamics.py
 - 🔗 QuadMultiAgent.py
 - 🔗 QuadTrajectory.py
 - ▼ Environment
 - > __pycache__
 - 🔗 __init__.py
 - 🔗 Canvas.py
 - 🔗 Obstacles.py
 - 🔗 Wind.py
 - ▼ Visualization
 - 🔗 __init__.py
 - 🔗 analysis_agent.py
 - 🔗 analysis_for_report.py
 - 🔗 analysis_quadcopter_single.py
 - 🔗 analysis_quadcopter.py
 - 🔗 CobraCobra.py
 - 🔗 main.py



Parameter

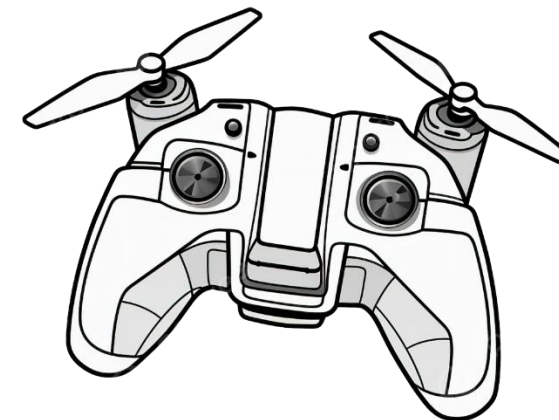
Verifikasi
Simulator

Pengujian
Skenario

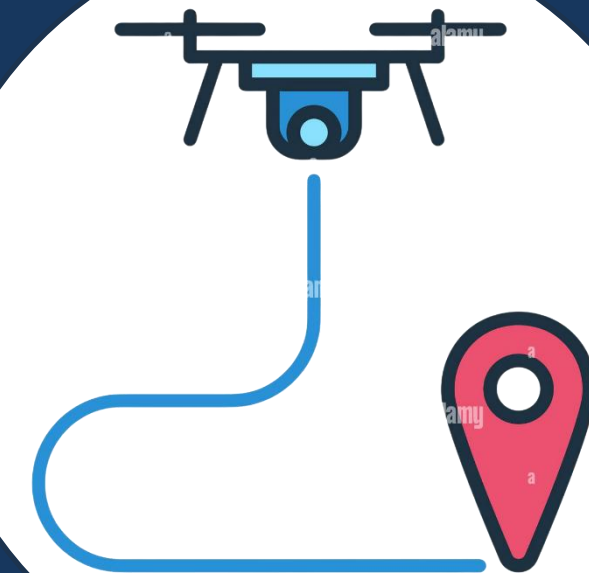
Parameter Fisik Quadcopter



Parameter *Low-Level Controller*



Parameter *High-Level Planner*



Parameter Fisik Quadcopter

Kerangka DJI F450 [3]



Parameter	Simbol	Nilai	Satuan
Massa Bodi Quadcopter	m_B	1.2	kg
Gravitasi	g	9.81	m/s^2
Momen Inersia Sb-x	I_{Bxx}	0.0123	$kg.m^2$
Momen Inersia Sb-y	I_{Byy}	0.0123	$kg.m^2$
Momen Inersia Sb-z	I_{Bzz}	0.0224	$kg.m^2$
Momen Inersia Rotor	I_{Rzz}	2.7×10^{-5}	$kg.m^2$
Jarak Pusat ke Motor	d_{xm}, d_{ym}, d_{zm}	0.16, 0.16, 0.05	m
Koefisien Gaya Dorong	k_{Th}	1.076×10^{-5}	$N/(rad/s)^2$
Koefisien Torsi	k_{To}	1.632×10^{-7}	$Nm/(rad/s)^2$
Koefisien Gaya Hambat	C_d	0.1	-
Gaya dorong minimum	T_{min}	0.1×4	N
Gaya dorong maksimum	T_{max}	9.18×4	N
Kecepatan angular motor minimum	$\omega_{M,min}$	75	rad/s
Kecepatan angular motor maksimum	$\omega_{M,max}$	925	rad/s
Konstanta waktu motor	ξ	0.015	s
Gain statis motor	k_p	1.0	rad/s
Rasio redaman motor	ζ	1.0	-
Koefisien linear motor	c_1	8.49	-
Offset	c_0	74.7	-

[3] John, bobzwik/Quadcopter_SimCon. (Jun. 10, 2025). Python. Accessed: Jun.13, 2025. [Online]. Available:https://github.com/bobzwik/Quadcopter_SimCon



Parameter

Verifikasi Simulator

Pengujian Skenario

Parameter
Low-Level Controller

Kerangka DJI F450 [3]

Parameter Batasan	Simbol	Nilai
Batas Kecepatan Keamanan Fisik	$v_{max,all}$	5.0 m/s
Sudut Kemiringan Maksimum	$tilt_{max}$	50°
Laju Sudut Roll Maksimum	$\omega_{\phi,max}$	200 deg/s
Laju Sudut Pitch Maksimum	$\omega_{\theta,max}$	200 deg/s
Laju Sudut Yaw Maksimum	$\omega_{\psi,max}$	150 deg/s

Loop Kontrol	Parameter Gain	Nilai
Kontroler Laju Angular (PD)	$K_{p,rate}(\omega_{\phi}, \omega_{\theta}, \omega_{\psi})$	1.5, 1.5, 1.0
	$K_{d,rate}(\omega_{\phi}, \omega_{\theta}, \omega_{\psi})$	0.04, 0.04, 0.1
Kontroler Sikap (P)	$K_{p,att}(\phi, \theta, \psi)$	8.0, 8.0, 1.5
Kontroler Kecepatan (PID)	$K_{p,vel}(v_x, v_y, v_z)$	5.0, 5.0, 4.0
	$K_{i,vel}(v_x, v_y, v_z)$	5.0, 5.0, 5.0
	$K_{d,vel}(v_x, v_y, v_z)$	0.5, 0.5, 0.5
Kontroler Posisi (P)	$K_{p,pos}(x, y, z)$	1.5, 1.5, 1.5

[3] John, bobzwik/Quadcopter_SimCon. (Jun. 10, 2025). Python. Accessed: Jun.13, 2025. [Online]. Available:https://github.com/bobzwik/Quadcopter_SimCon

Parameter

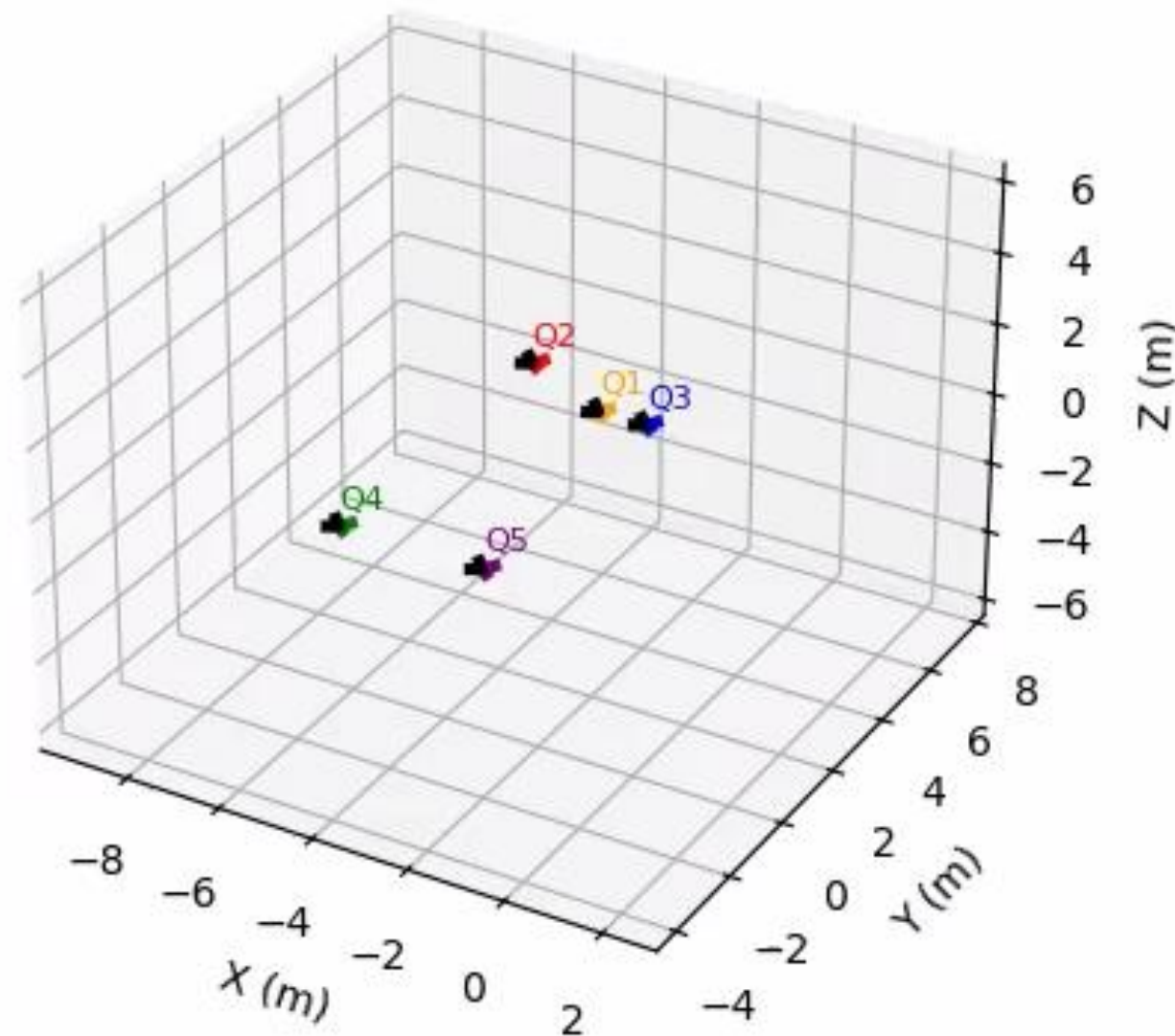
Parameter *High-Level Planner*

Verifikasi Simulator

Pengujian Skenario

Parameter	Simbol	Nilai [13]
Kecepatan Referensi Migrasi	v_{ref}	0.5 m/s
Kecepatan Referensi Maksimum	v_{max}	1.0 m/s
Gain Penghindaran Rintangan	k_{obs}	8.0
Gain Penghindaran Agen	k_{col}	8.0
Gain Perilaku Formasi	k_{form}	1.0
Gain Perilaku Tailgating ERC	k_{tail}	1.2
Gain Perilaku Takeoff	$k_{p,takeoff}$	1.5
Target Ketinggian	z_{target}	-2.0
Toleransi Ketinggian	ϵ_z	0.1
Indikasi kecepatan stagnan	ϵ	0.001 m/s
Kecepatan maksimum	$v_{max,takeoff}$	1.0 m/s
Faktor Ambang Batas ERC	α	8.0
Jarak Referensi ERC	d_{ref}	1.0 m

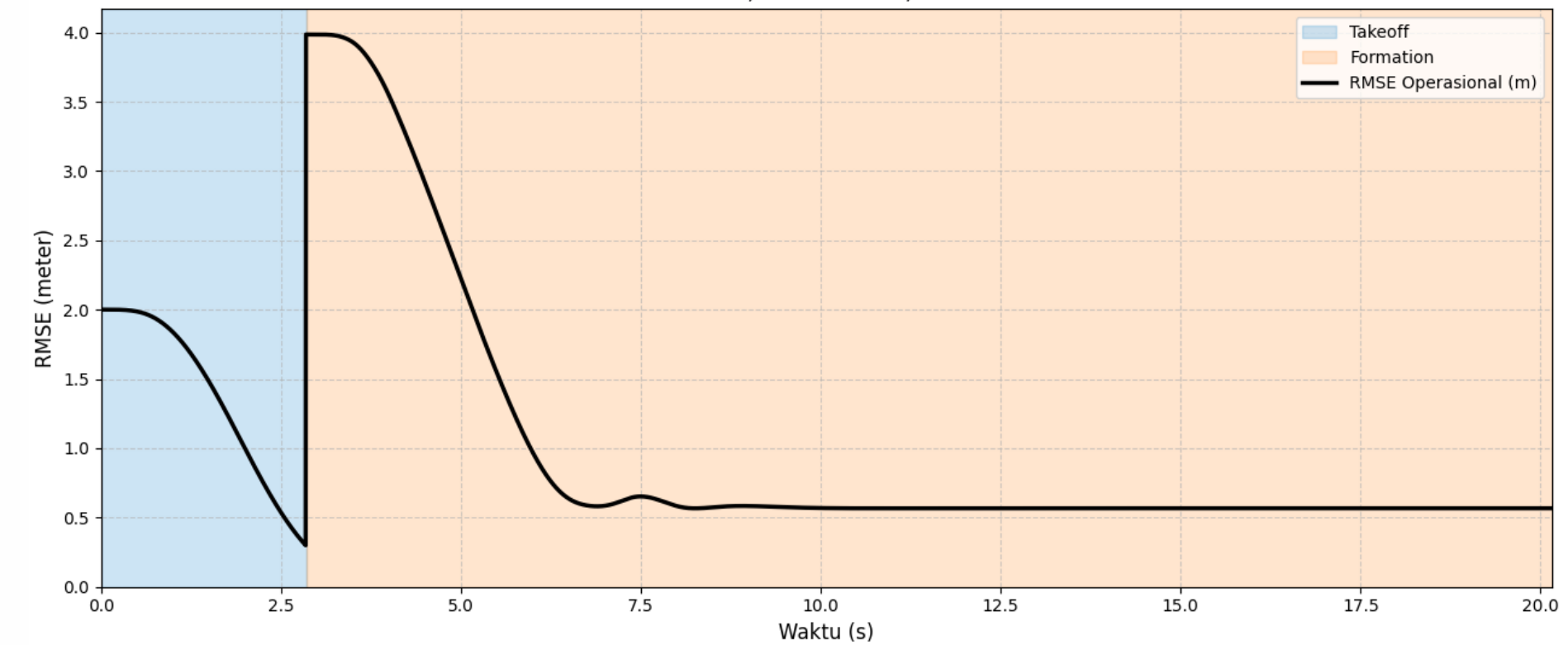
3D Environment
Obs: NONE - Wind: NONE - Contr: erc
Formation: 1 - Orient: NED - Time: 0.10s



RMSE Akurasi Operasional Berdasarkan Mode

RMSE_{total} = 1.047 meter

Obs: NONE, Wind: NONE, Contr: erc



Parameter

Verifikasi
Simulator

Pengujian
Skenario

Skenario 1: Uji Penghindaran Dasar

Skenario 2: Uji Adaptasi Pada Celah Sempit

Skenario 3: Uji Ketahanan Gangguan Angin

Skenario 4: Uji Batas Kemampuan

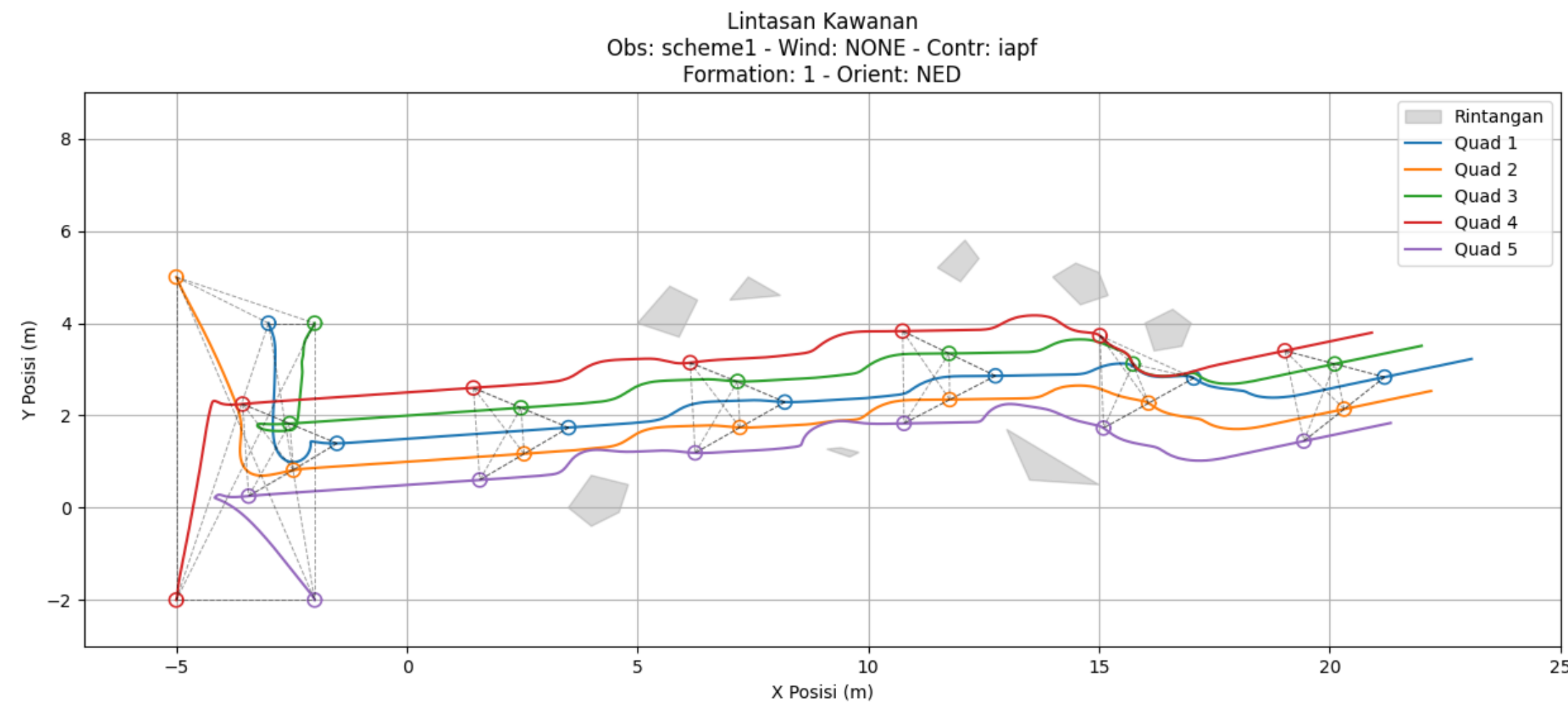
Skenario 5: Uji Navigasi *Waypoint*

Skenario 6: Uji Navigasi Lintasan Melingkar

Skenario 1: Uji Penghindaran Dasar

Posisi awal tersebar
Titik tujuan akhir pada (22.0, 3.0, -5.0)

Kontrol IAPF

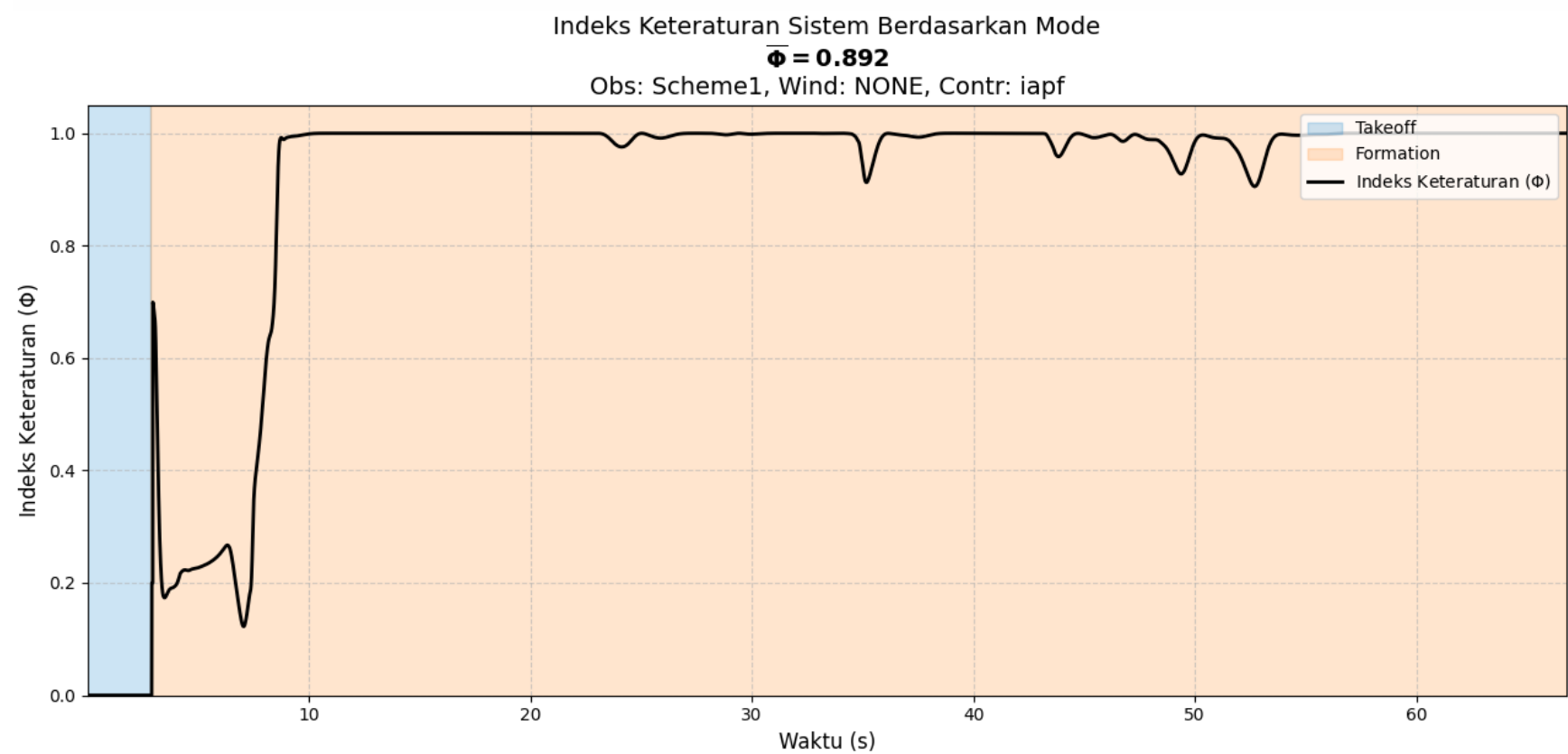
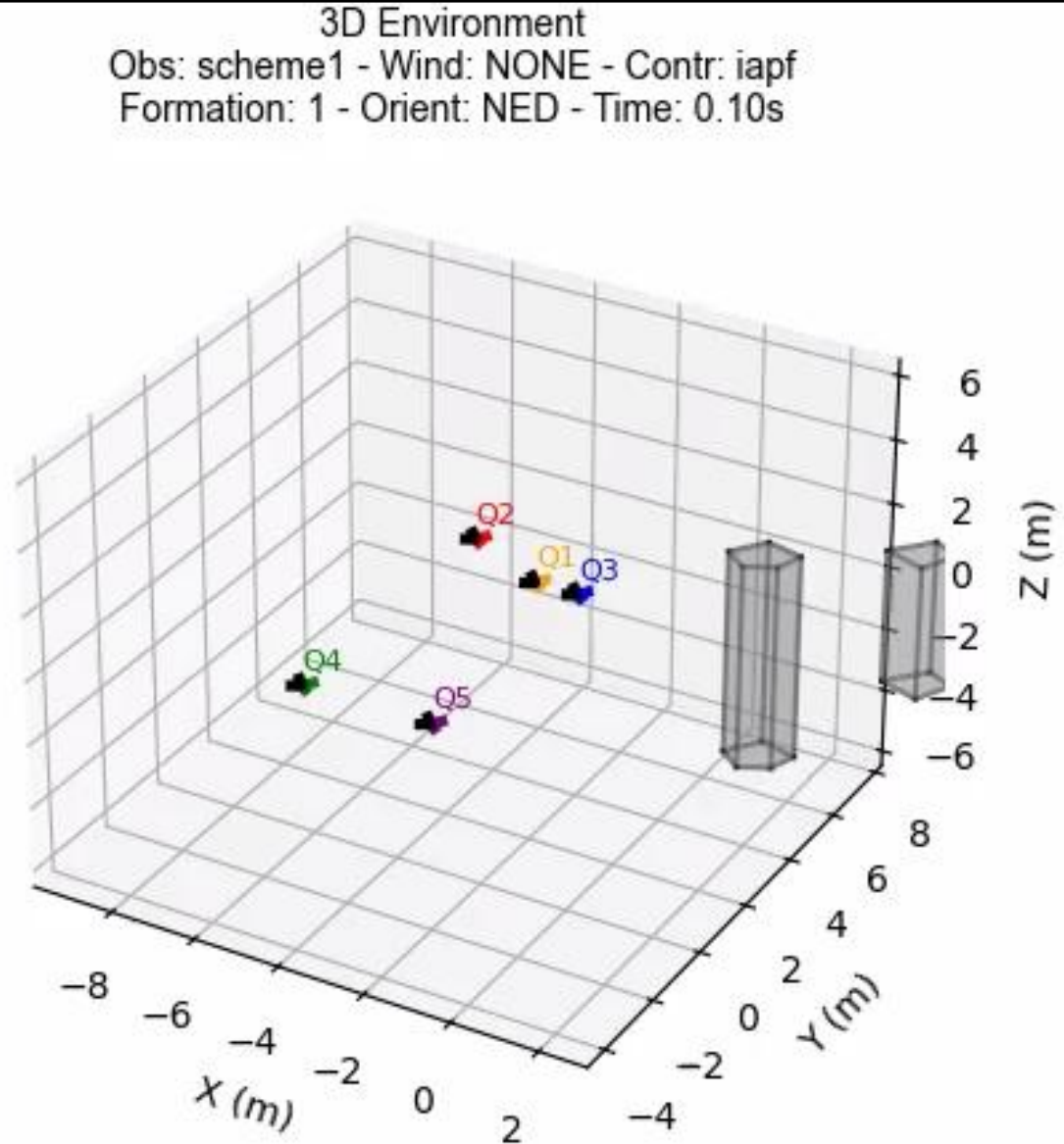
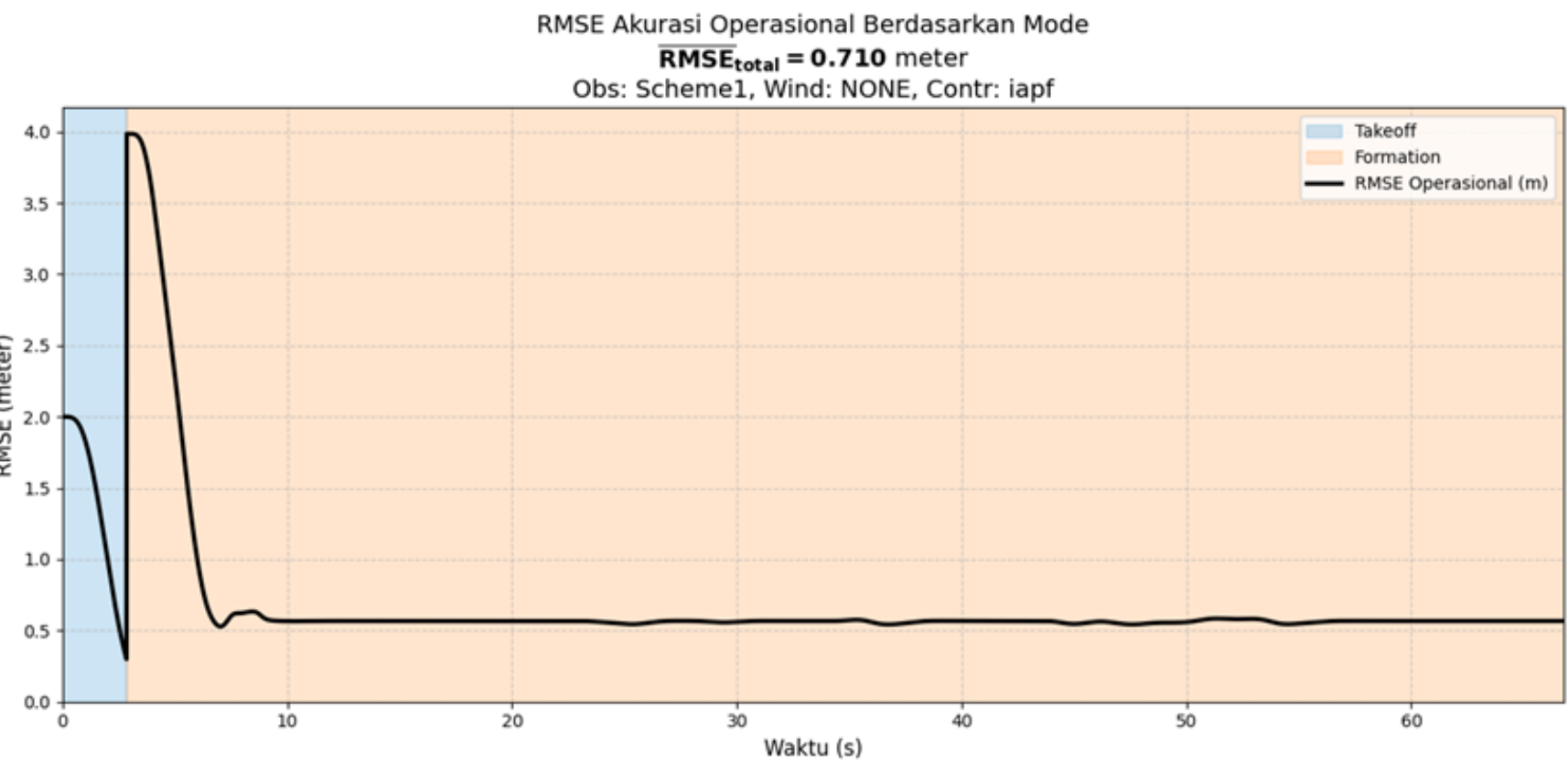
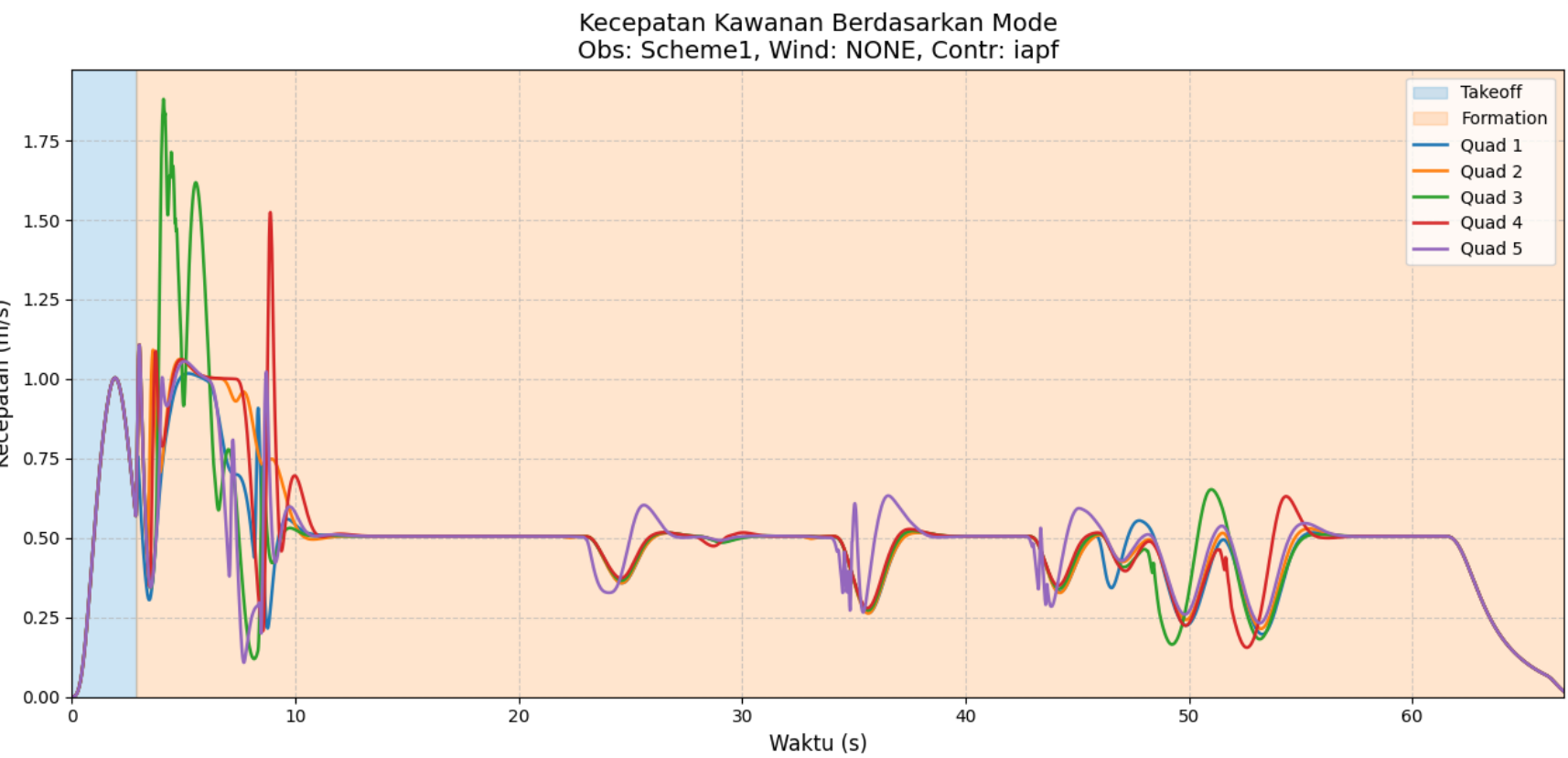


Misi selesai dalam
66.79 detik

Parameter

Verifikasi
SimulatorPengujian
Skenario

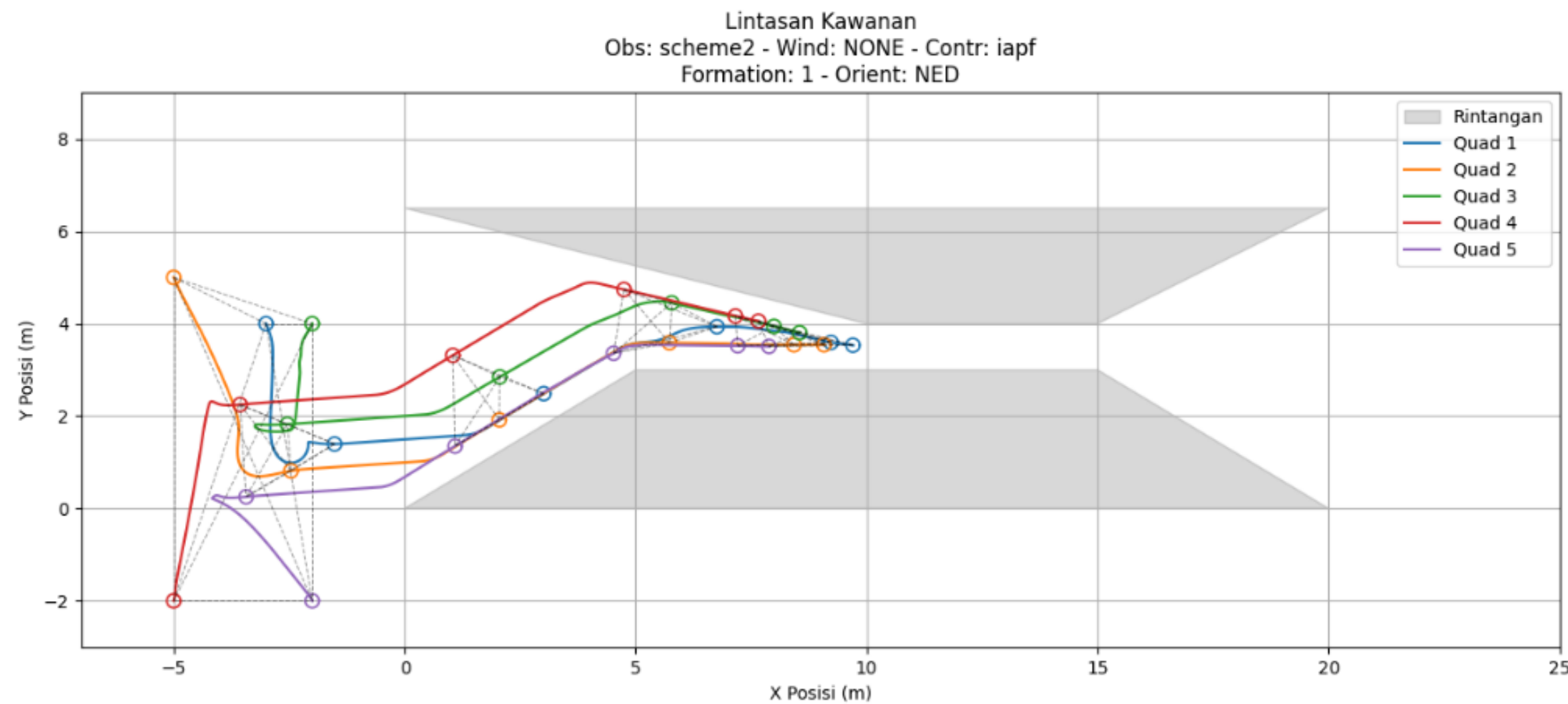
Skenario 1: Uji Penghindaran Dasar



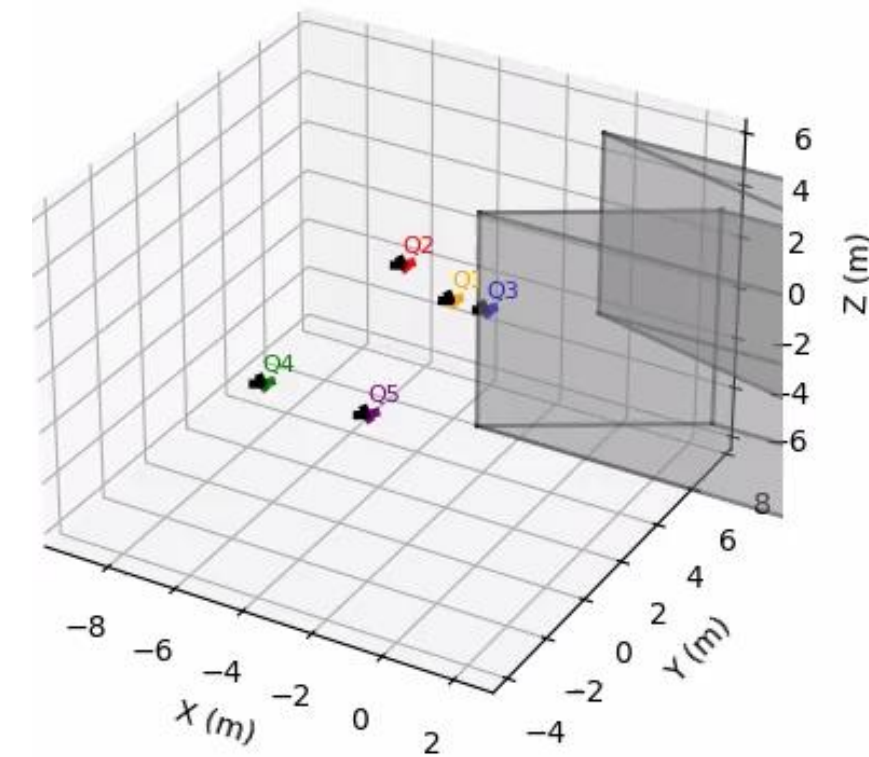
Skenario 2: Uji Adaptasi Pada Celah Sempit

Posisi awal tersebar
Titik tujuan akhir pada (22.0, 3.0, -5.0)

Kontrol IAPF



3D Environment
Obs: scheme2 - Wind: NONE - Contr: iapf
Formation: 1 - Orient: NED - Time: 0.10s

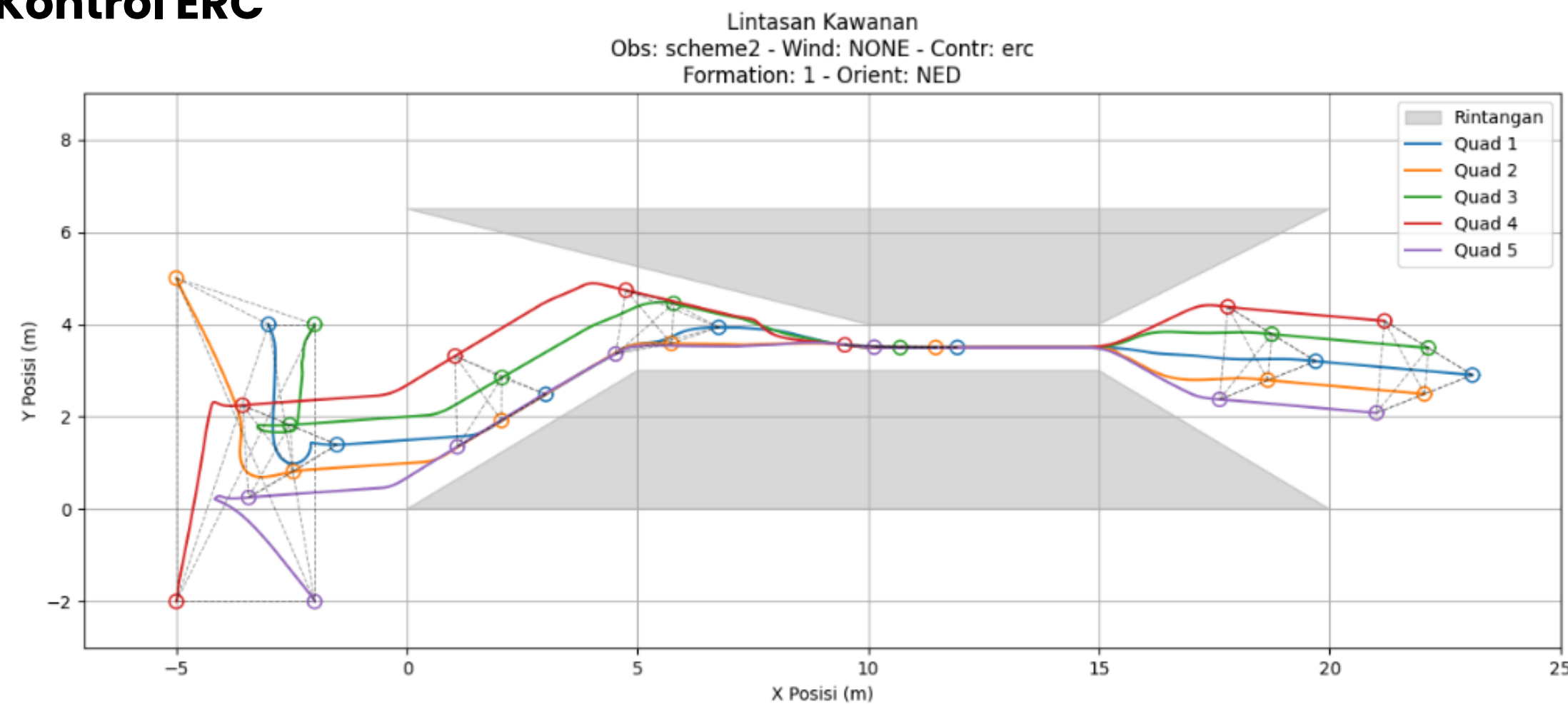


Misi gagal → **Kontrol ERC**

Skenario 2: Uji Adaptasi Pada Celah Sempit

Posisi awal tersebar
Titik tujuan akhir pada (22.0, 3.0, -5.0)

Kontrol ERC

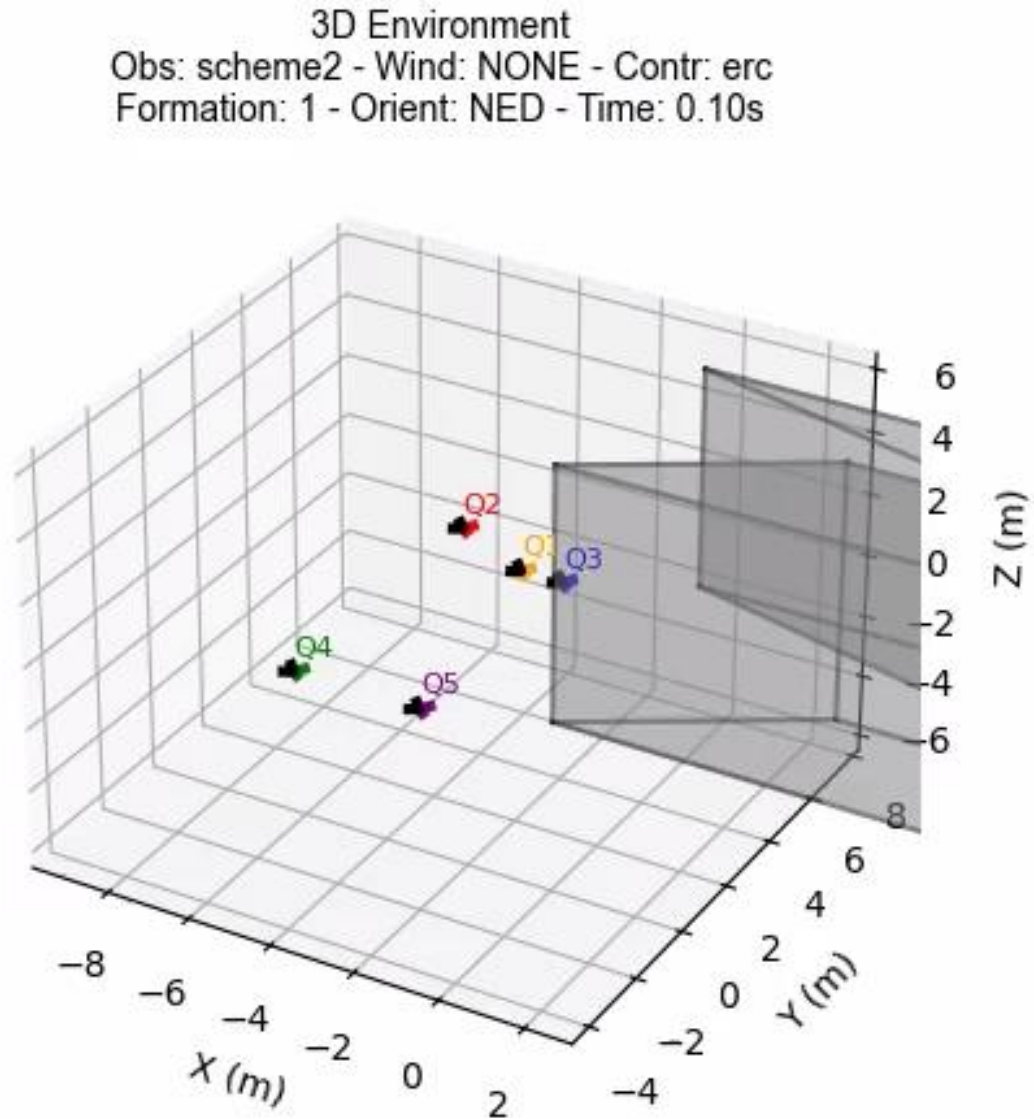
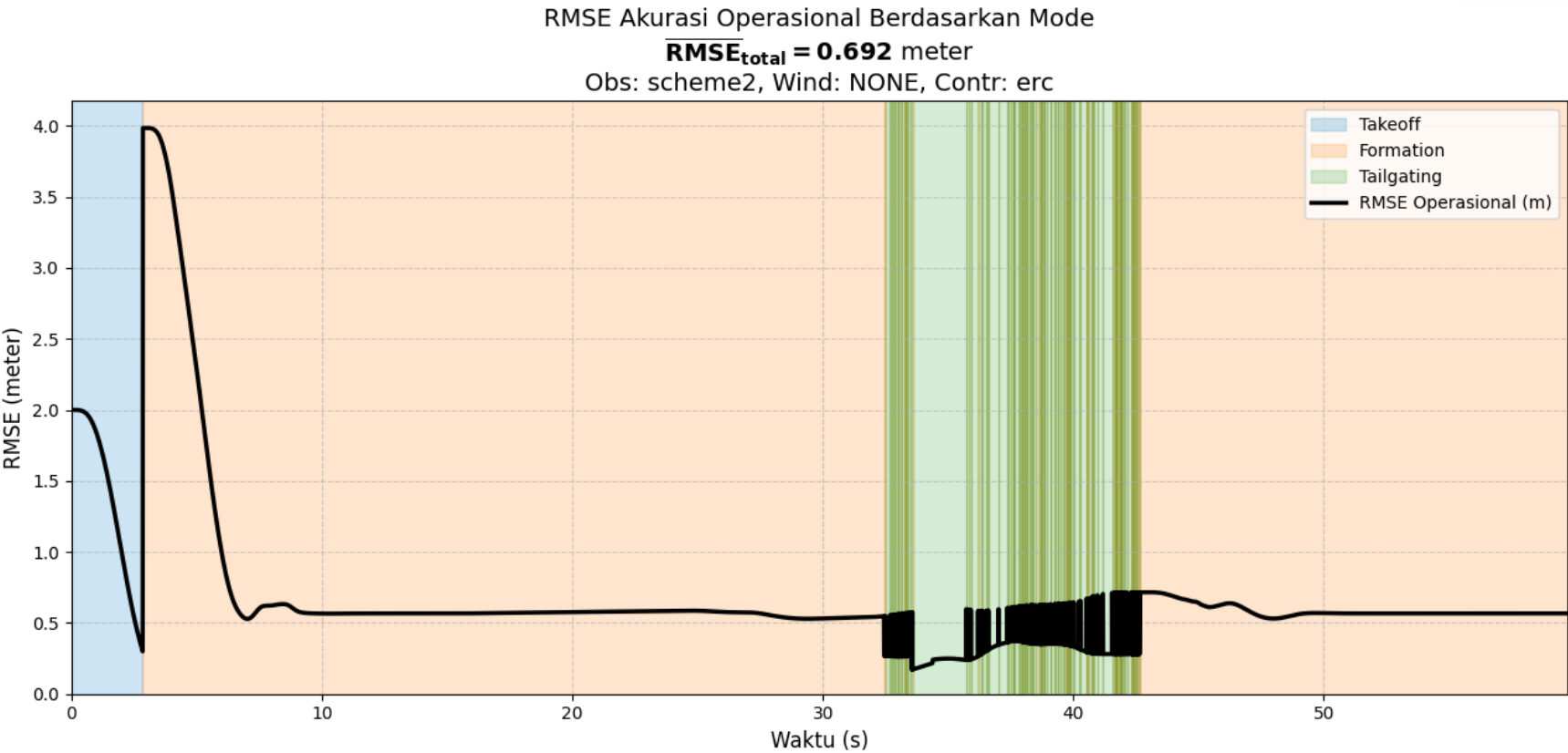
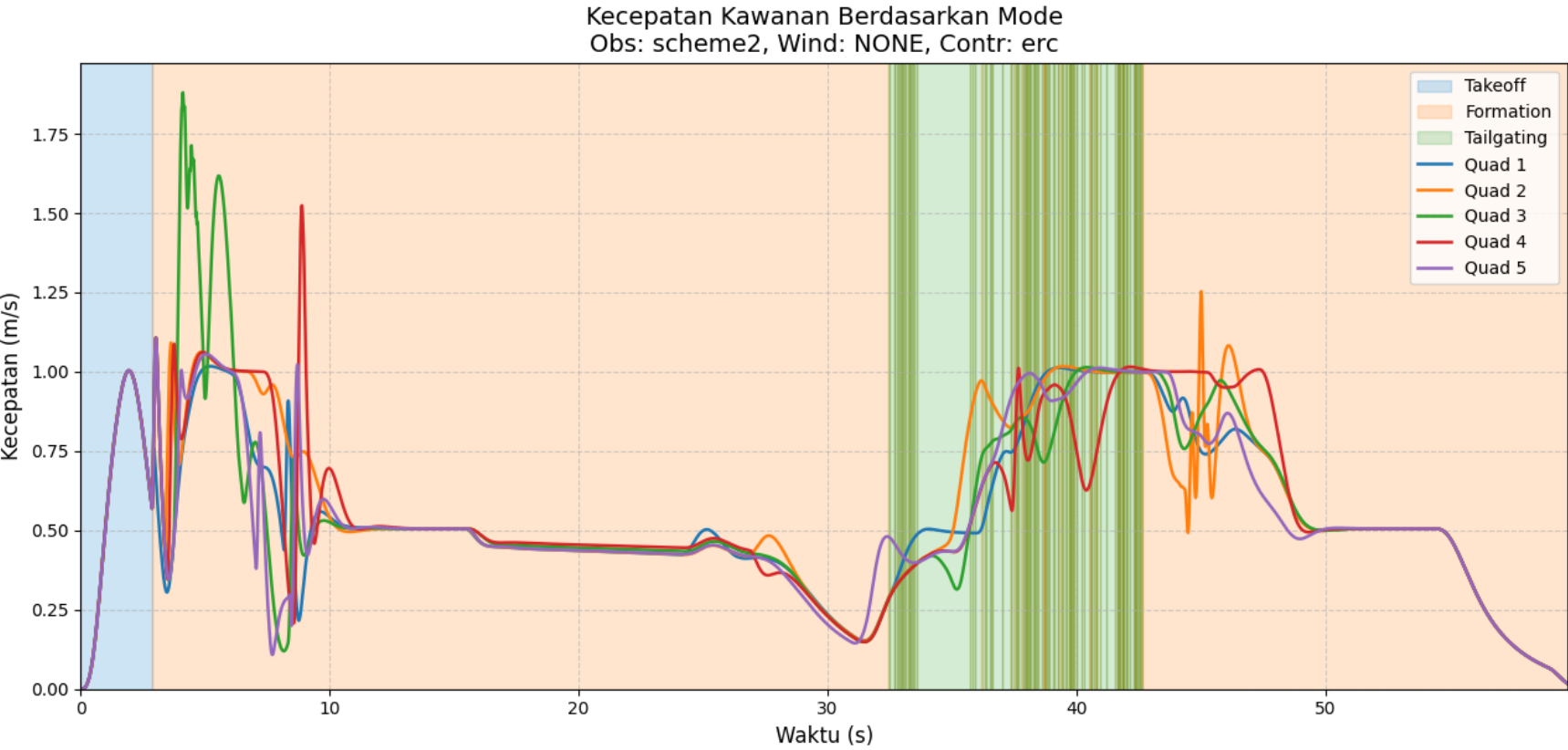


Misi selesai dalam
59.735 detik

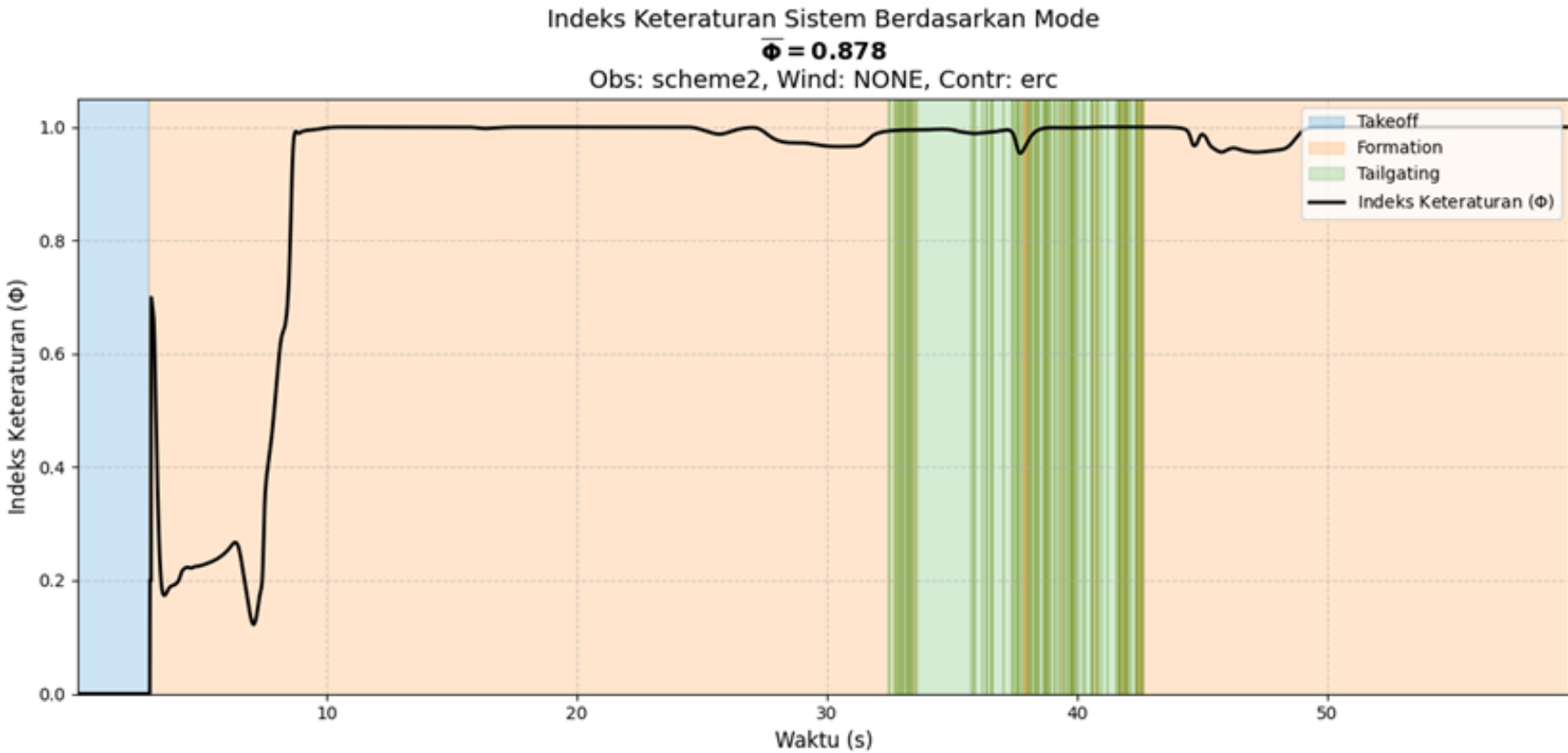
Parameter

Verifikasi
SimulatorPengujian
Skenario

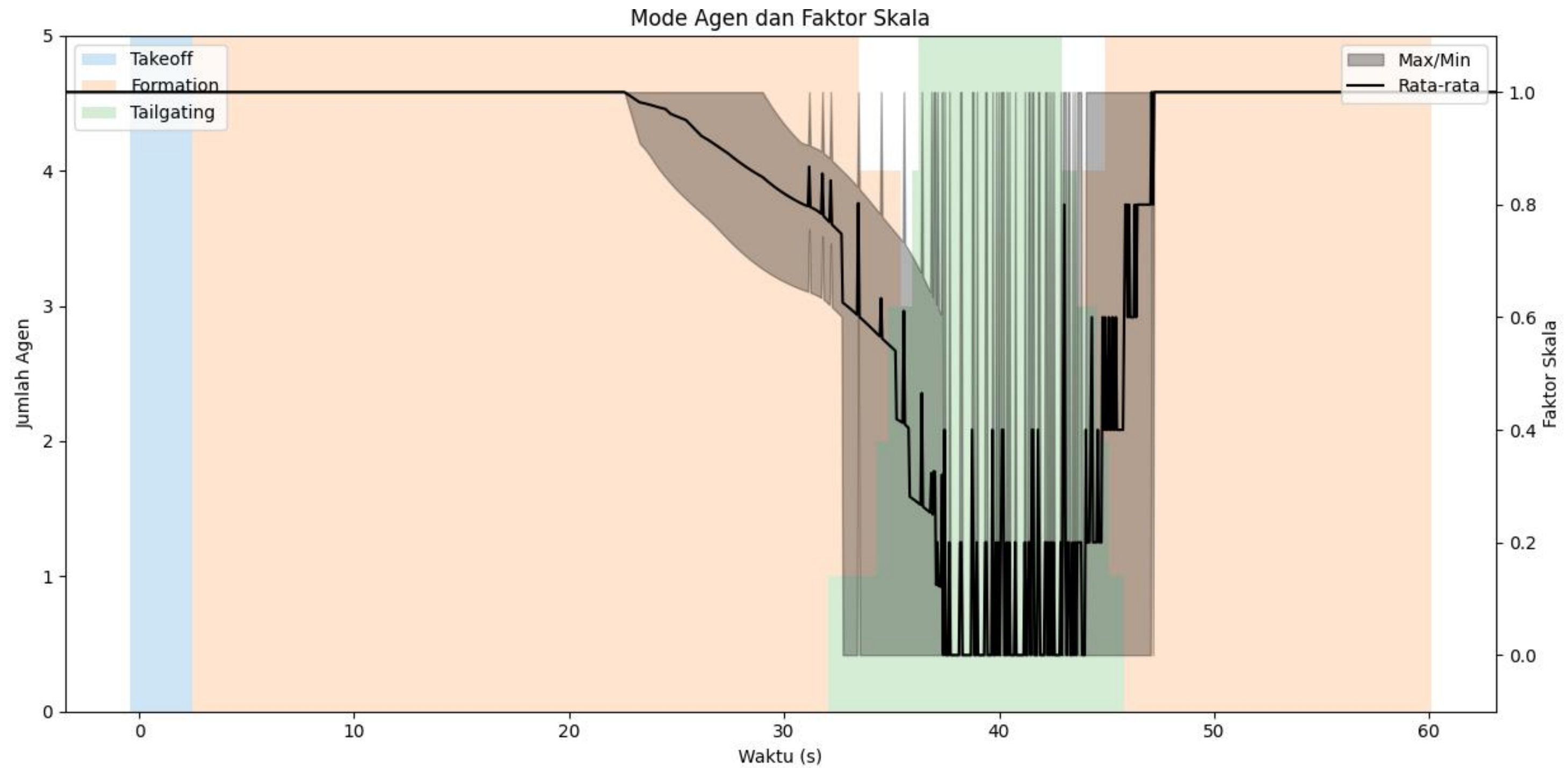
Skenario 2: Uji Adaptasi Pada Celah Sempit



Mode Agen & Faktor Skala

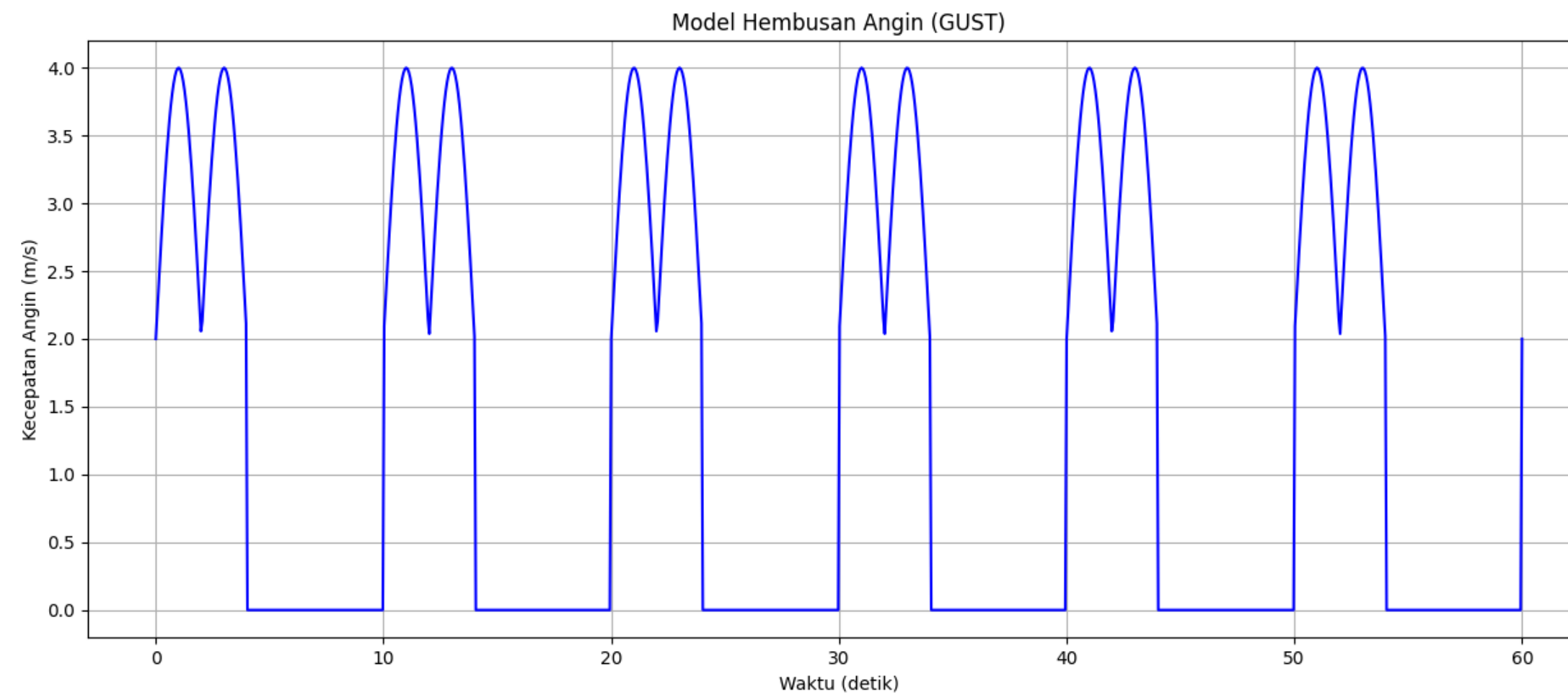


Mode Agen & Faktor Skala



Skenario 3: Uji Ketahanan Gangguan Angin

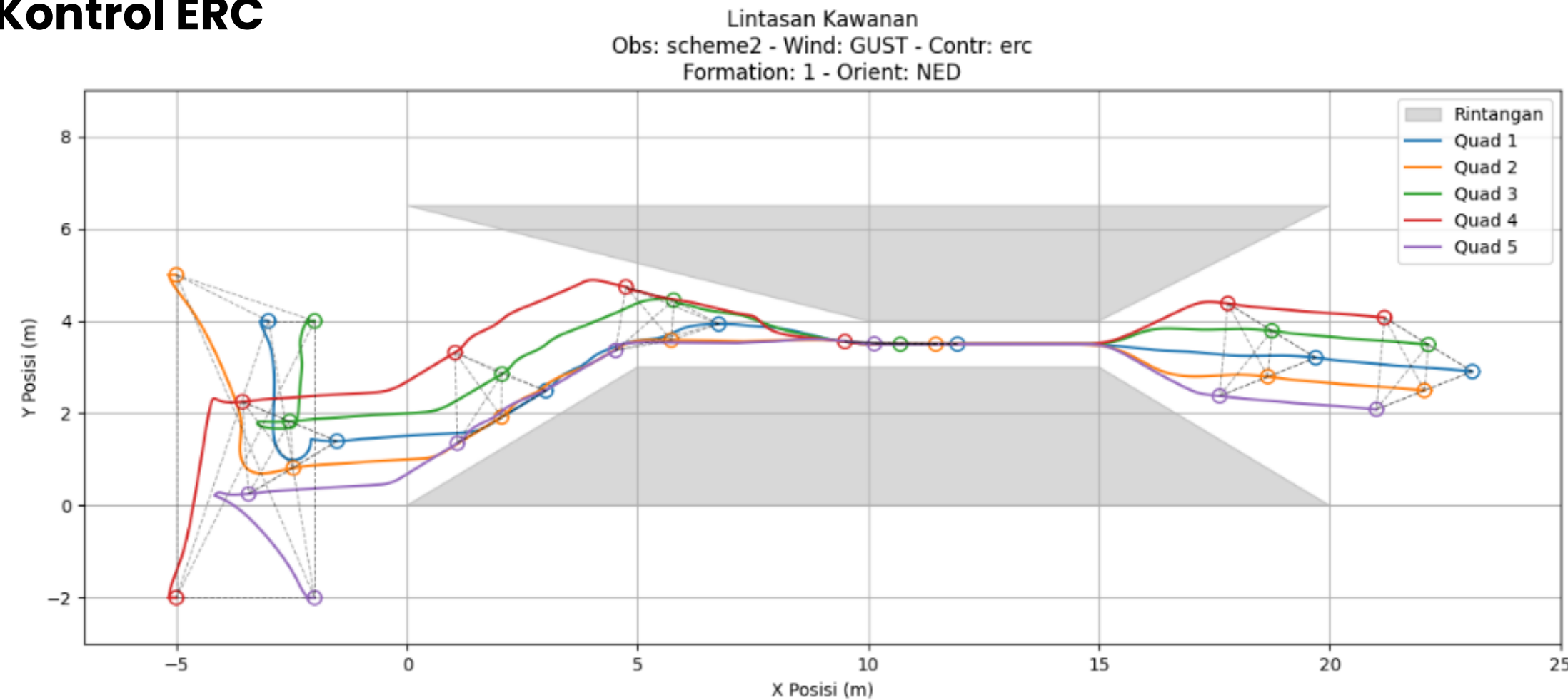
Parameter

Verifikasi
SimulatorPengujian
Skenario

Skenario 3: Uji Ketahanan Gangguan Angin

Posisi awal tersebar
Titik tujuan akhir pada (22.0, 3.0, -5.0)

Kontrol ERC



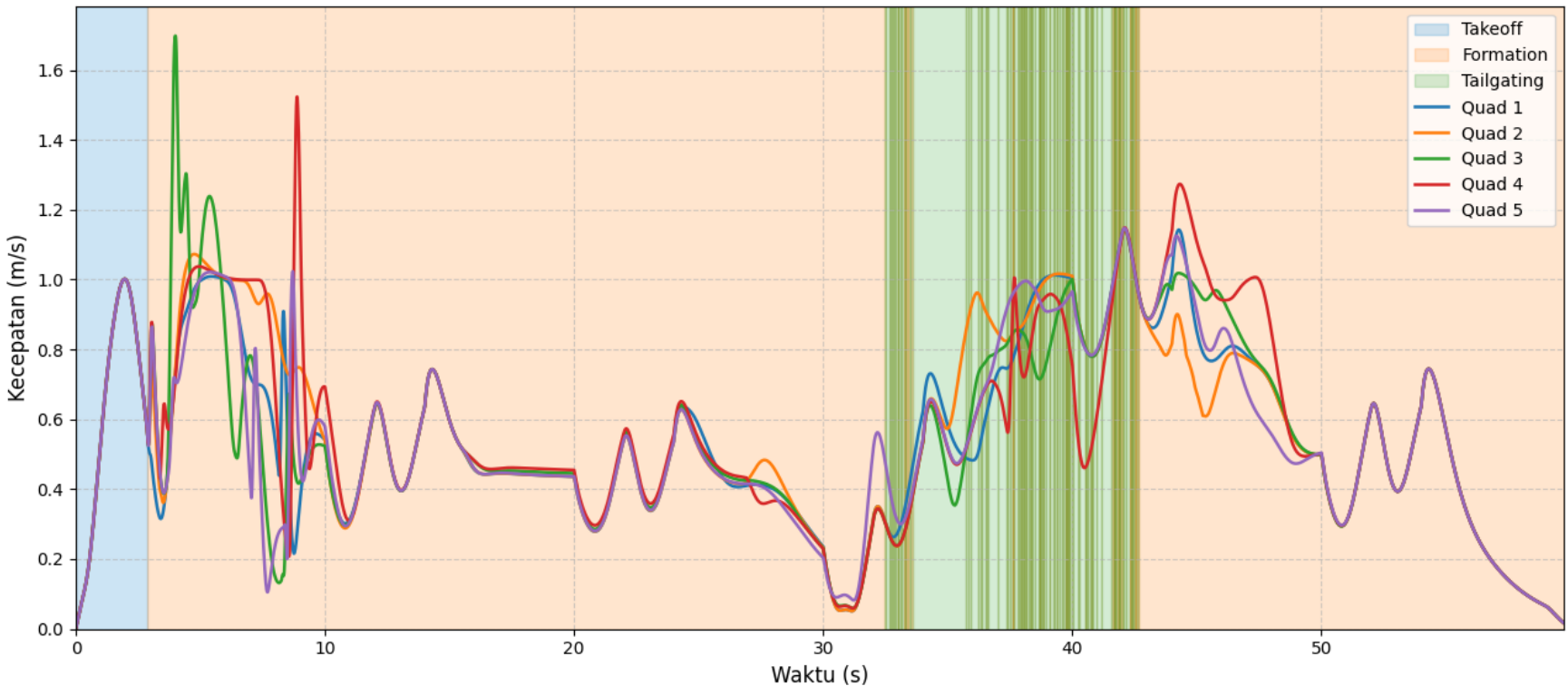
Misi selesai dalam
59.735 detik

Parameter

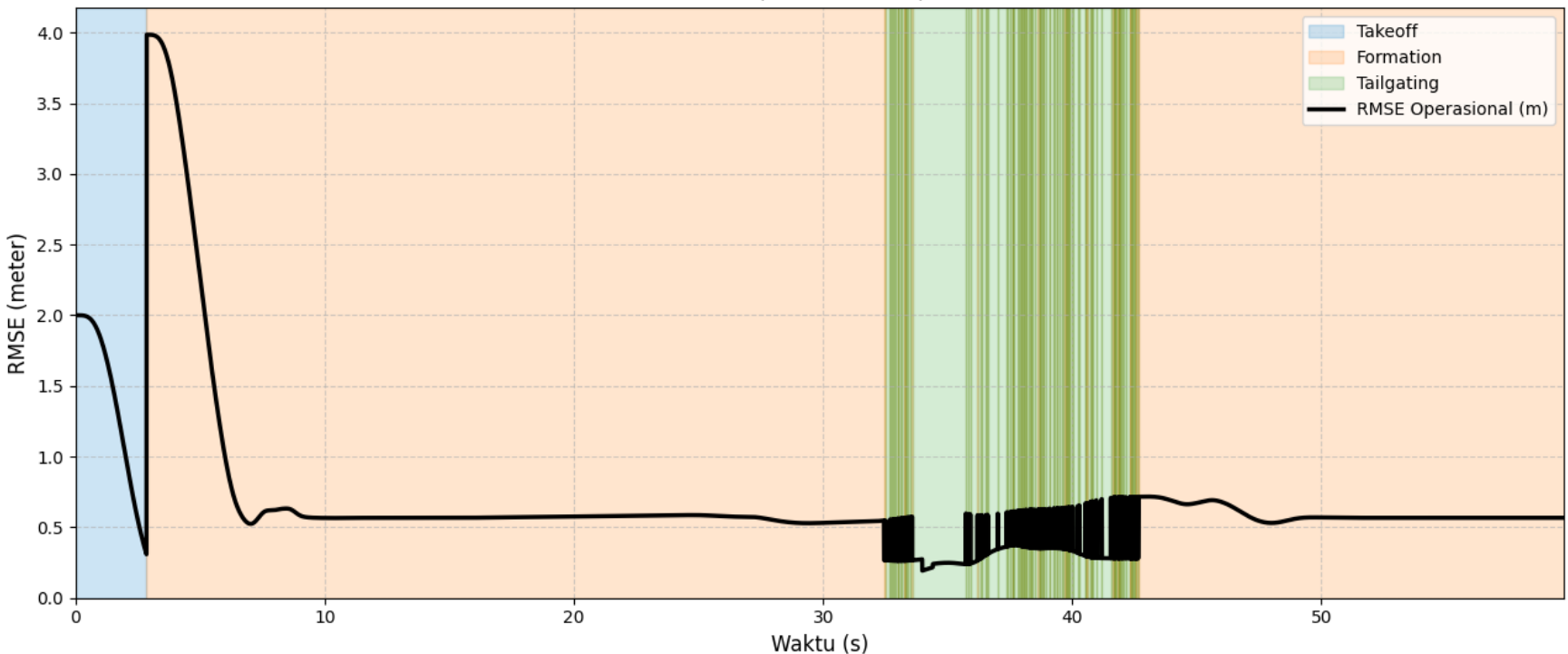
Verifikasi
SimulatorPengujian
Skenario

Skenario 3: Uji Ketahanan Gangguan Angin

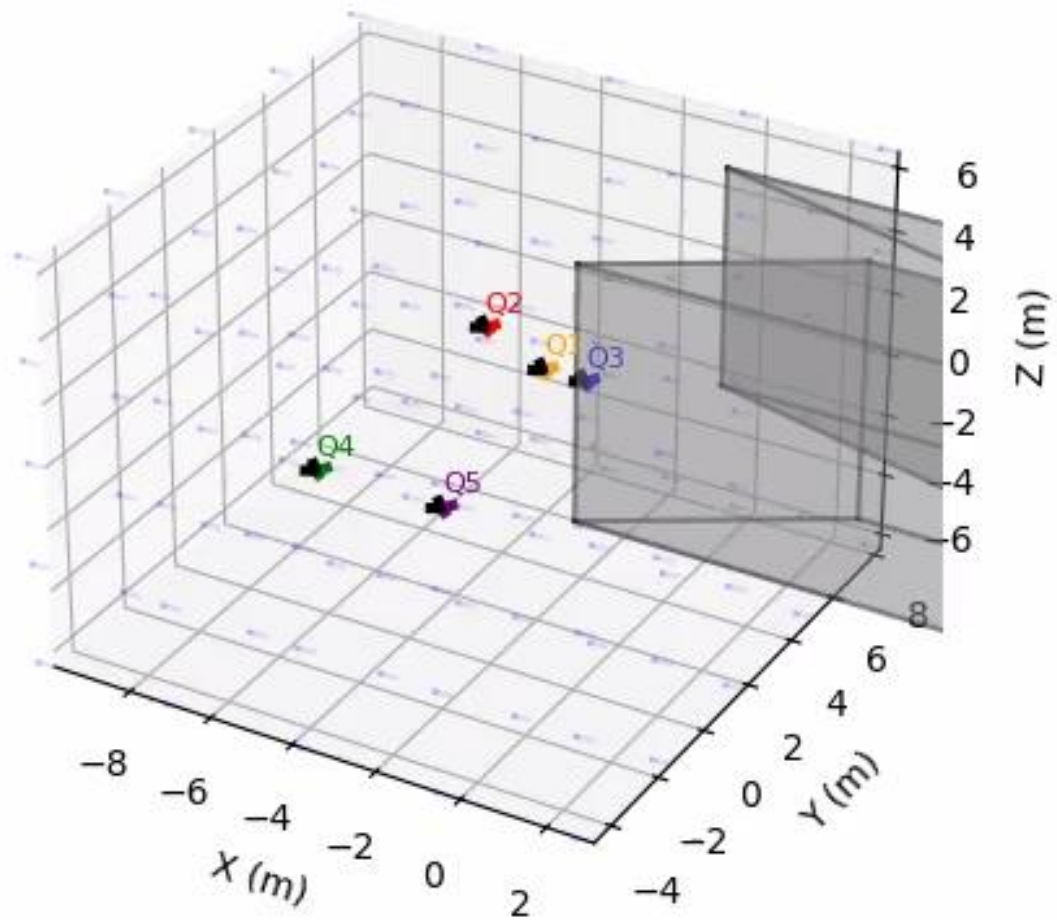
Kecepatan Kawan Berdasarkan Mode
Obs: scheme2, Wind: GUST, Contr: erc



RMSE Akurasi Operasional Berdasarkan Mode
RMSE_{total} = 0.694 meter
Obs: scheme2, Wind: GUST, Contr: erc

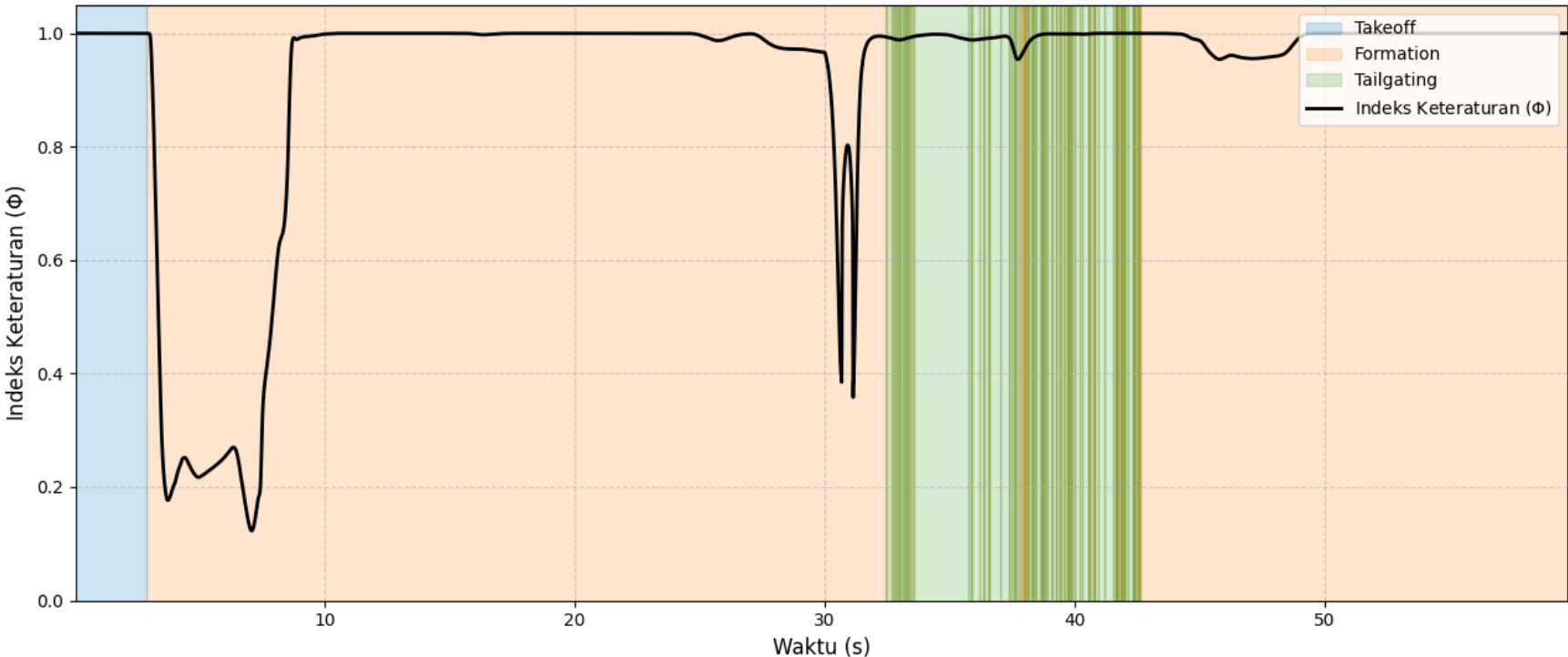


3D Environment
Obs: scheme2 - Wind: GUST - Contr: erc
Formation: 1 - Orient: NED - Time: 0.10s



Usaha Kontrol

Indeks Keteraturan Sistem Berdasarkan Mode
 $\Phi = 0.925$
Obs: scheme2, Wind: GUST, Contr: erc



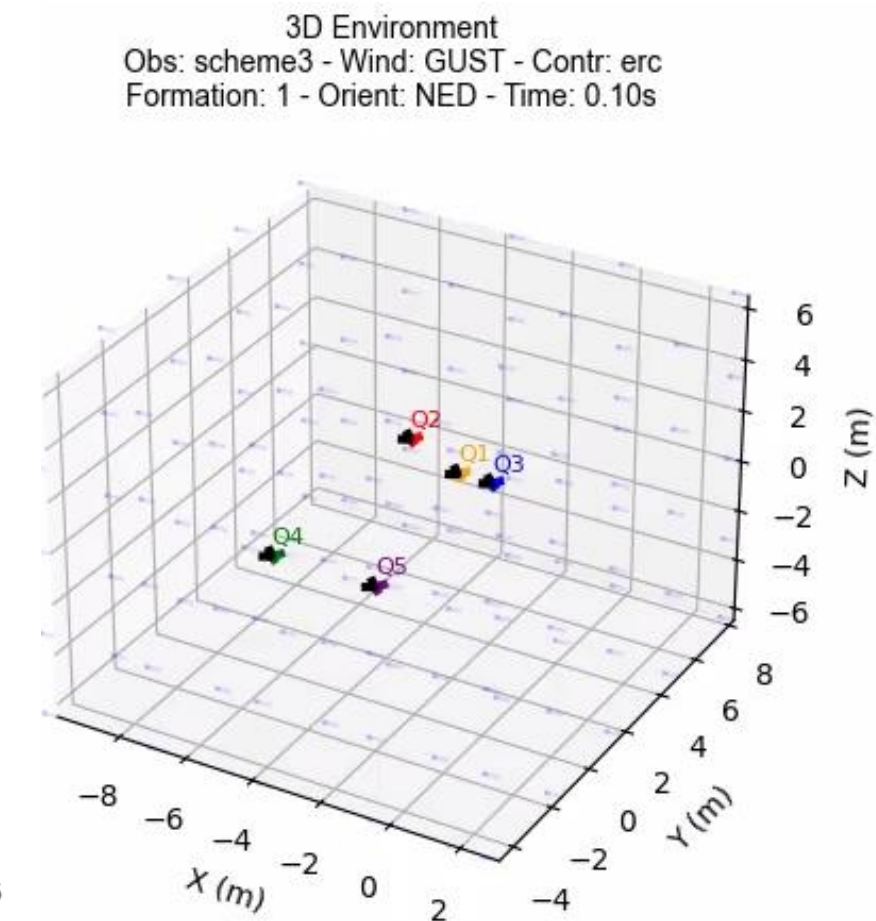
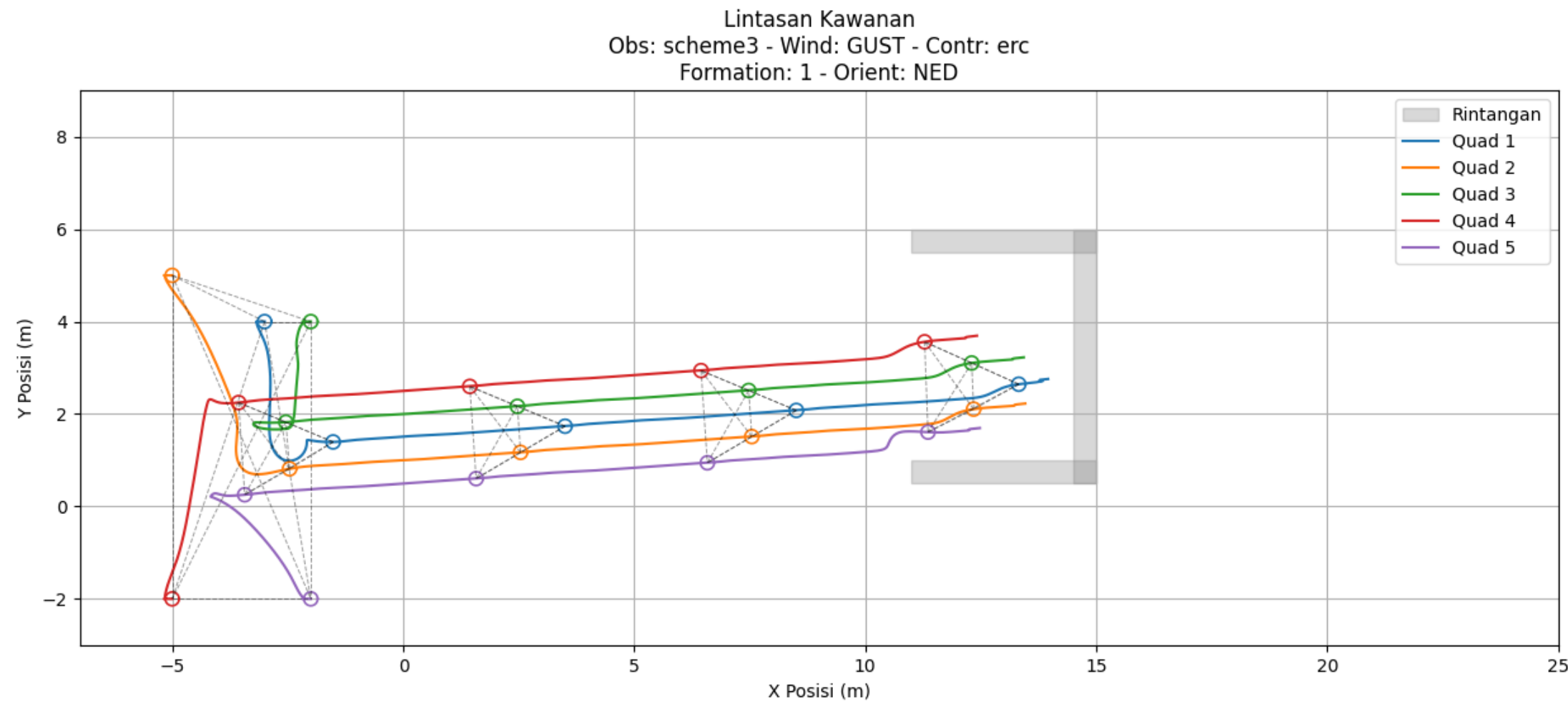
Usaha Kontrol



Skenario 4: Uji Batas Kemampuan

Posisi awal tersebar
Titik tujuan akhir pada (22.0, 3.0, -5.0)

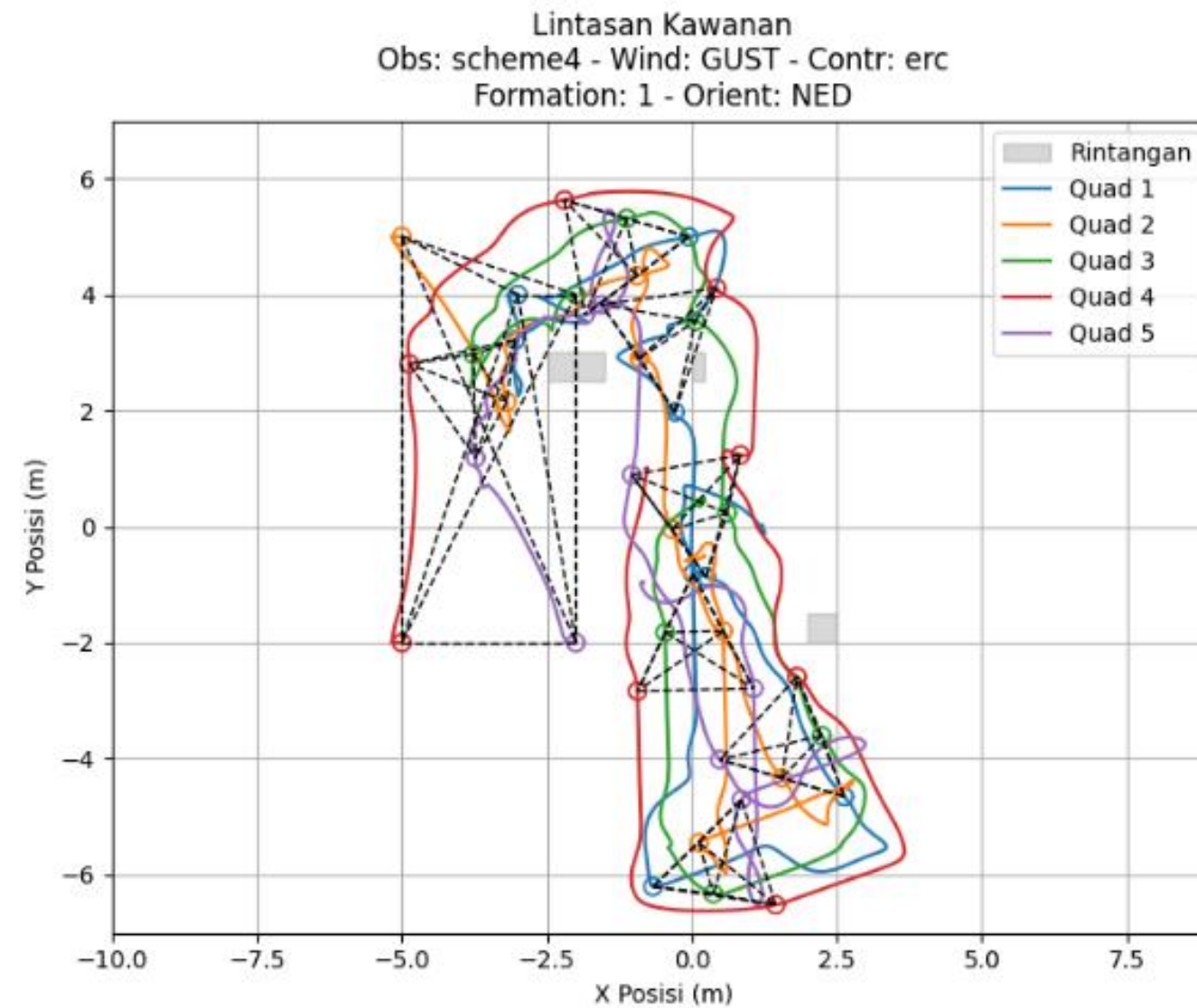
Parameter

Verifikasi
SimulatorPengujian
Skenario

Skenario 5: Uji Navigasi *Waypoint*

Titik-titik waypoint yang ditentukan adalah

[0.0, 5.0, -5.0],
[0.0, 0.0, -5.0],
[3.0, -5.0, -5.0],
[0.0, -6.0, -5.0],
[0.0, 0.0, -5.0]



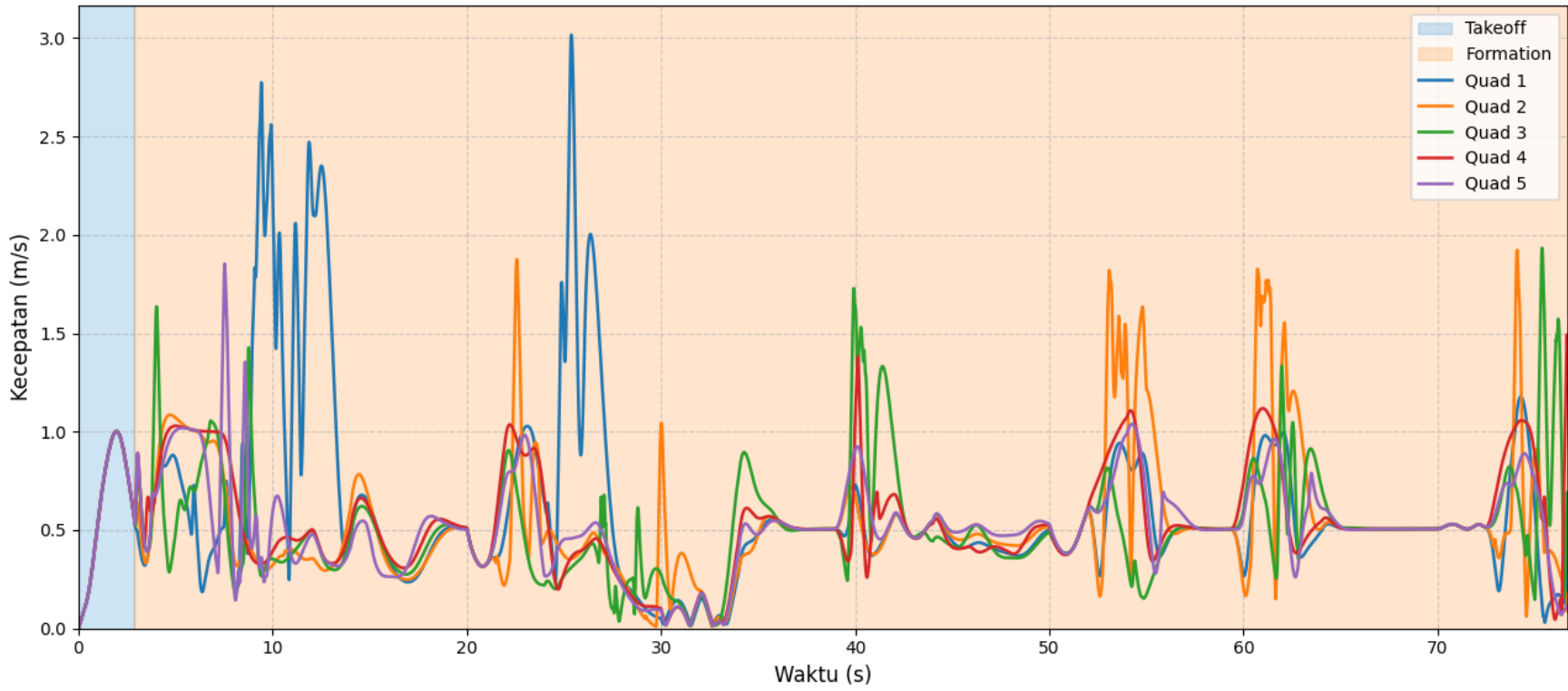
Misi selesai dalam 76.695 detik

Parameter

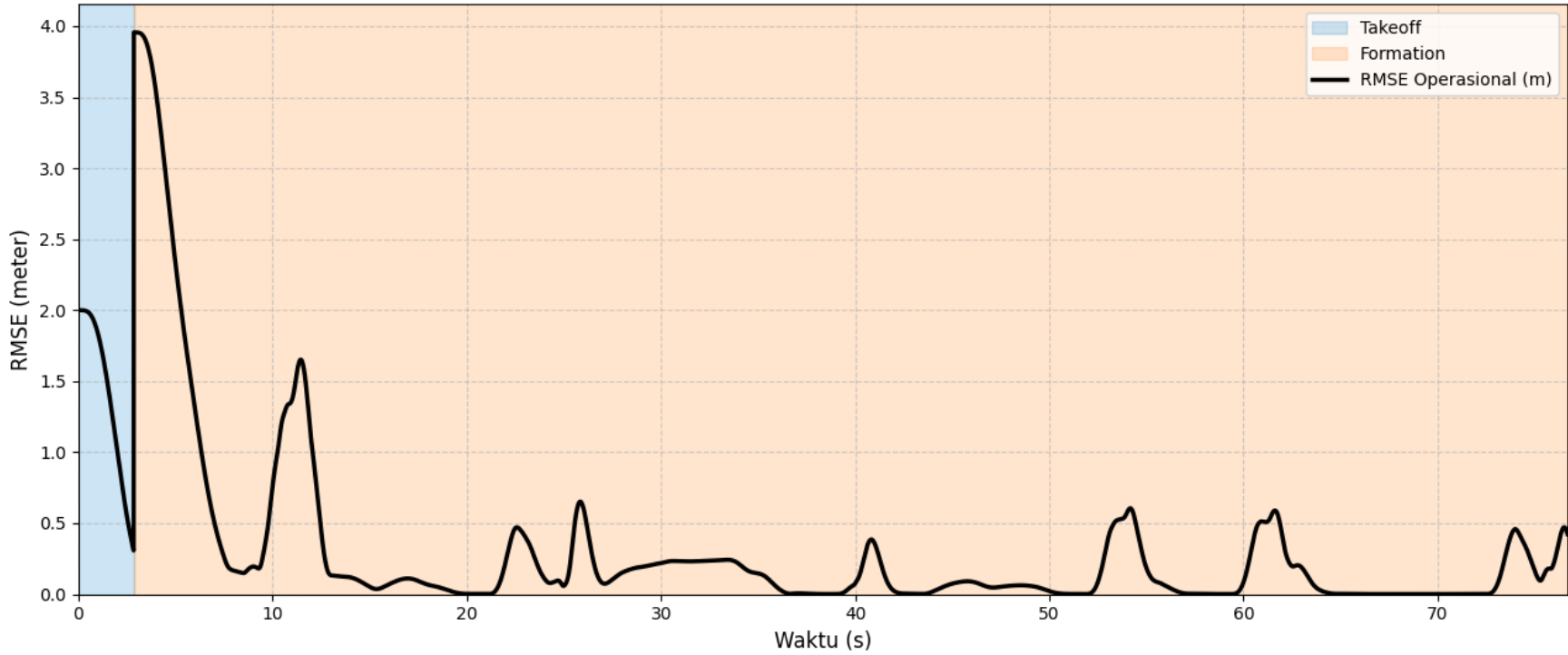
Verifikasi
SimulatorPengujian
Skenario

Skenario 5: Uji Navigasi *Waypoint*

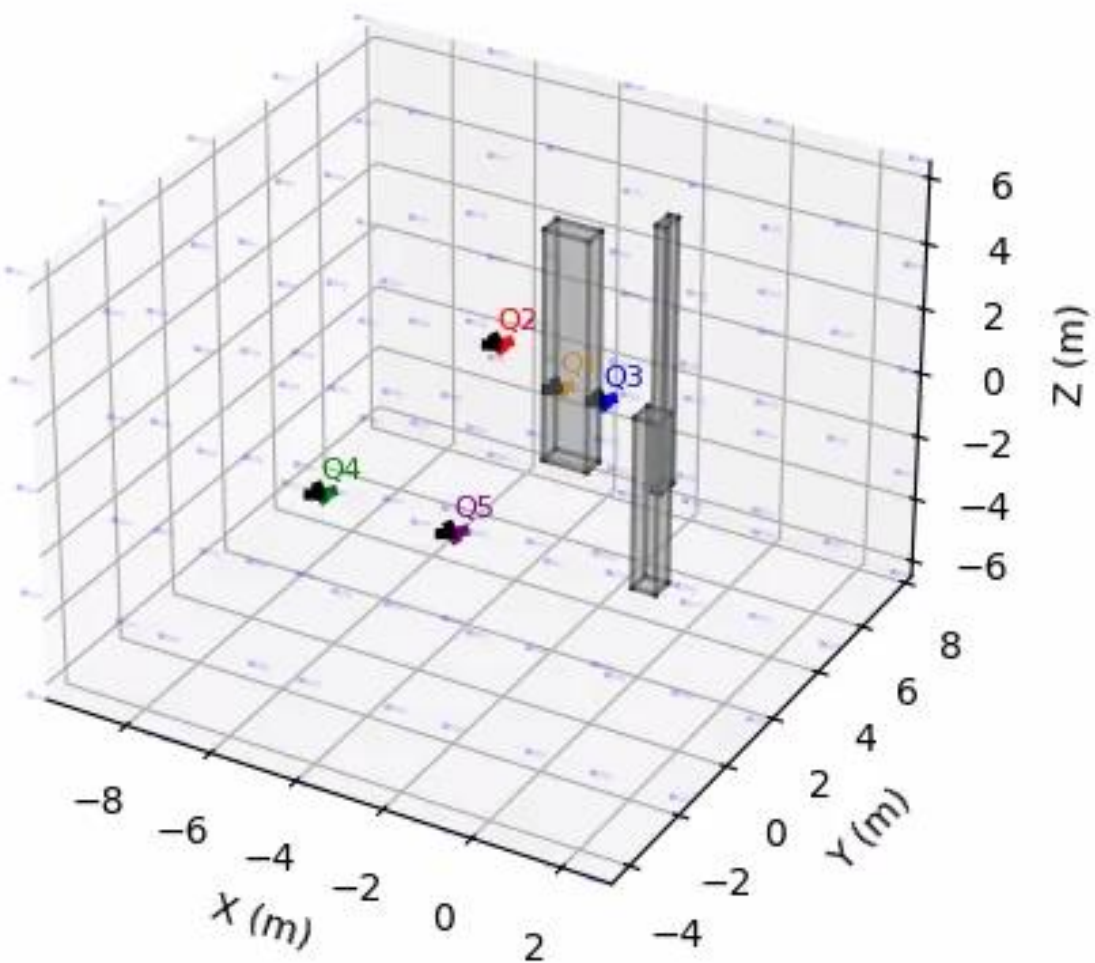
Kecepatan Kawan Berdasarkan Mode
Obs: scheme4, Wind: GUST, Contr: erc



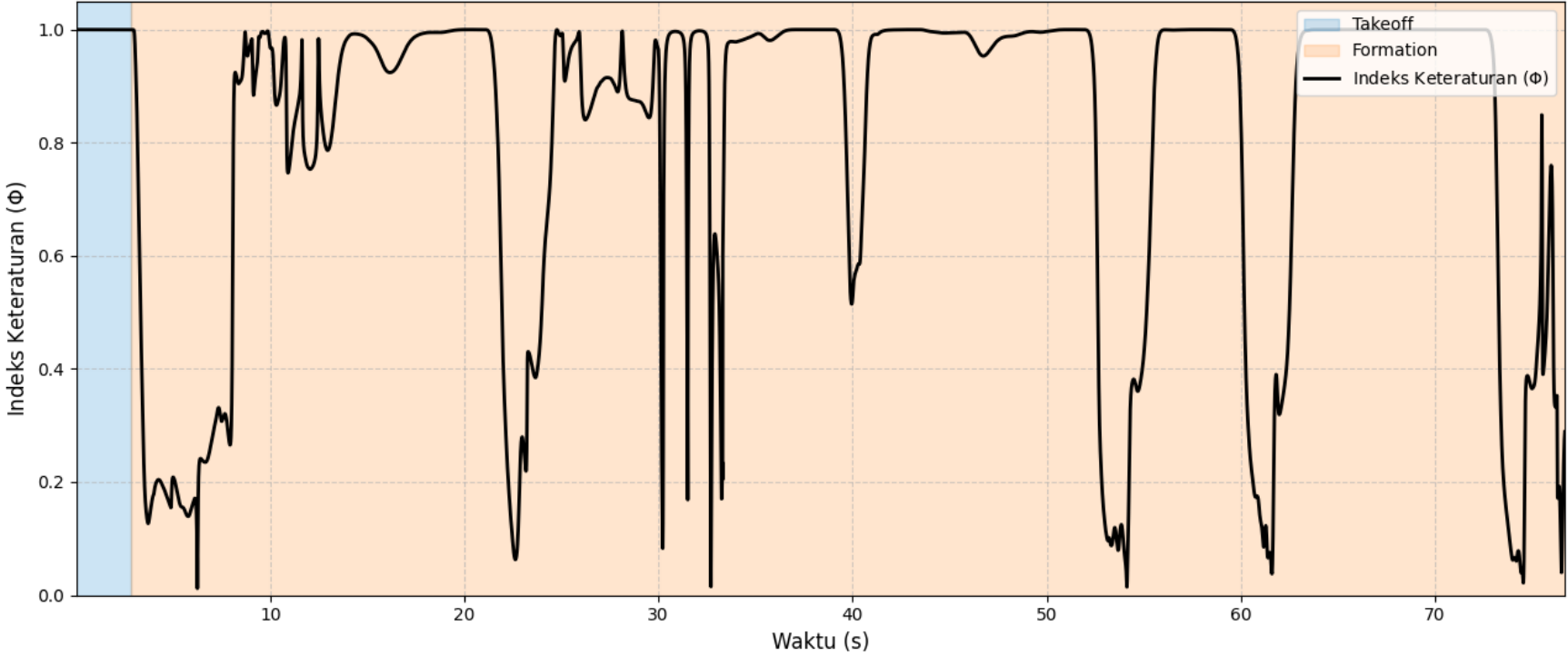
RMSE Akurasi Operasional Berdasarkan Mode
RMSE_{total} = 0.338 meter
Obs: scheme4, Wind: GUST, Contr: erc



3D Environment
Obs: scheme4 - Wind: GUST - Contr: erc
Formation: 1 - Orient: NED - Time: 0.10s



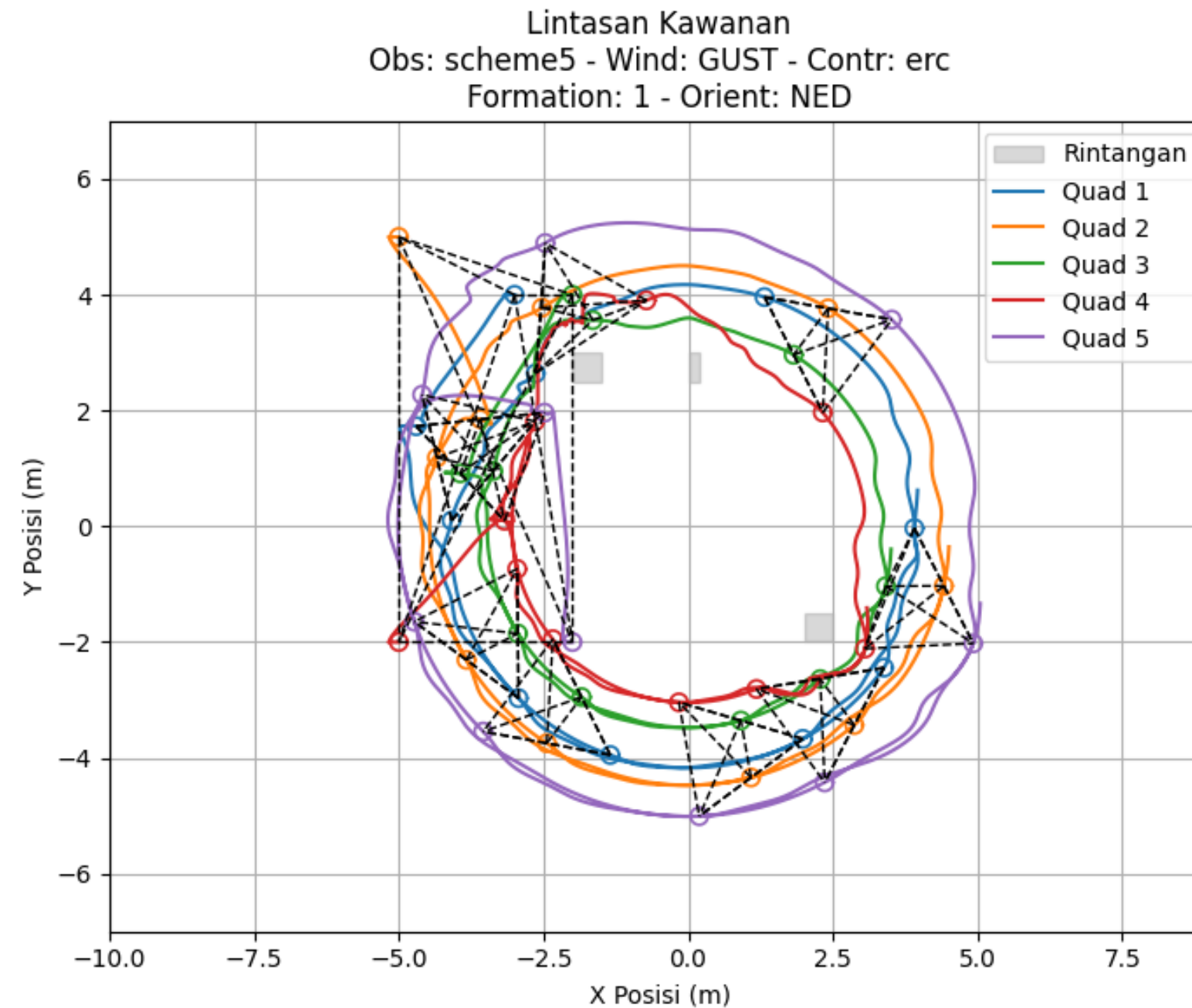
Indeks Keteraturan Sistem Berdasarkan Mode
 $\bar{\Phi} = 0.810$
Obs: scheme4, Wind: GUST, Contr: erc



Skenario 6 : Uji Navigasi Lintasan Melingkar

Melingkari titik pusat
dengan radius 4.0 meter

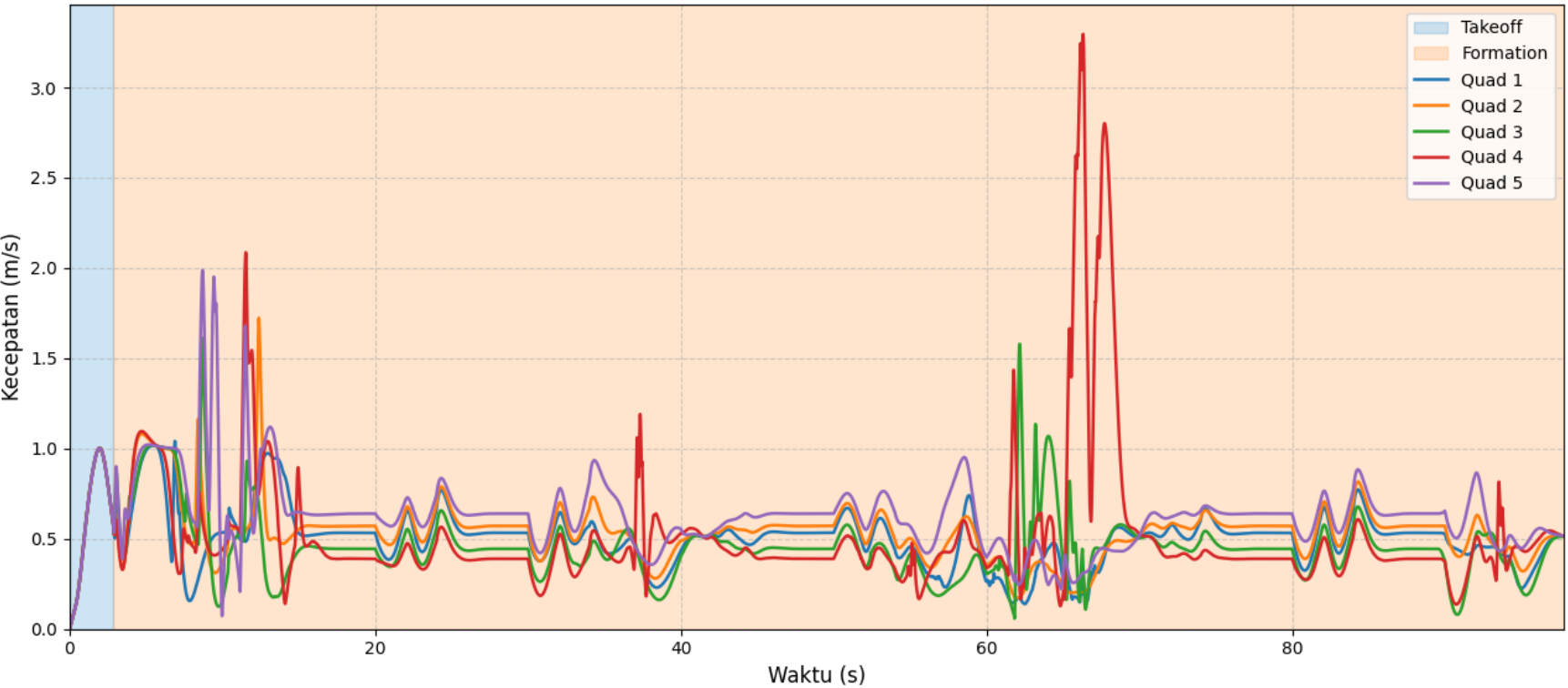
Parameter

Verifikasi
SimulatorPengujian
Skenario

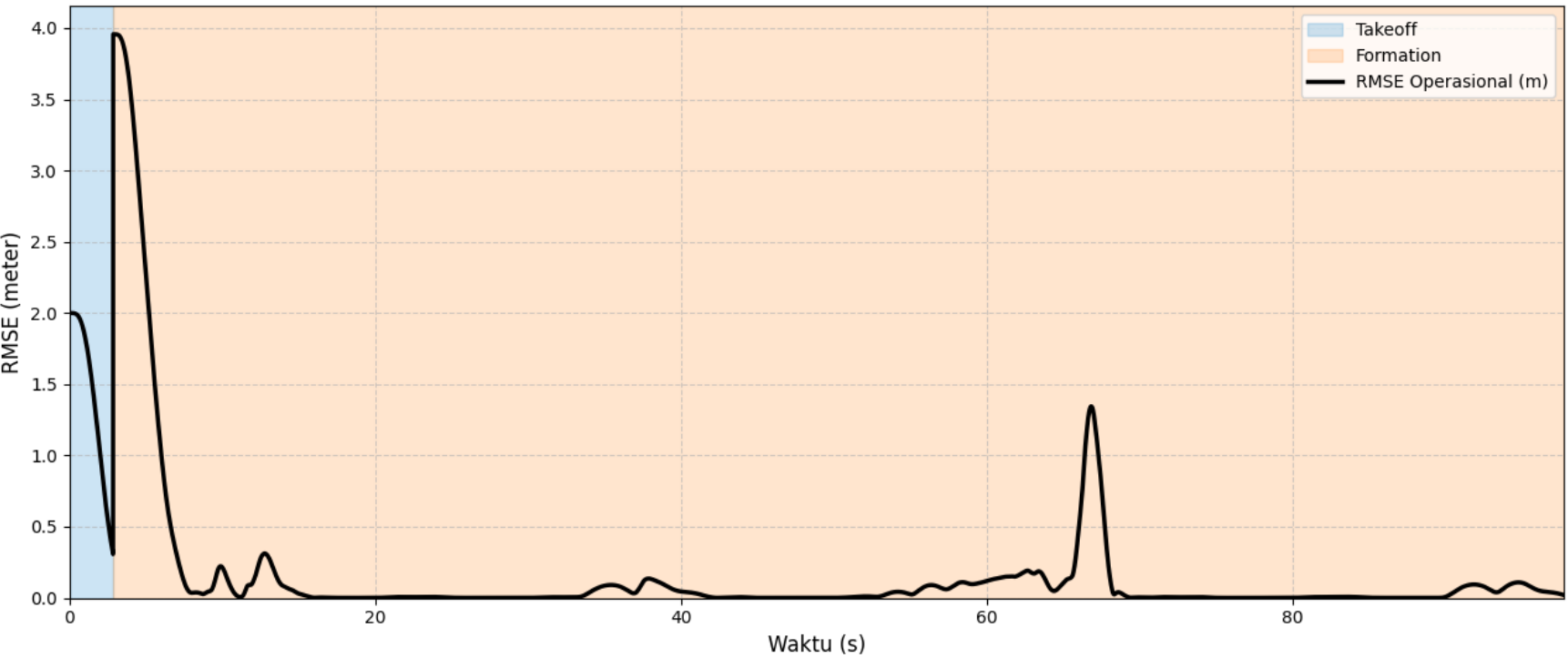
Diselesaikan dalam
97.795 detik

Skenario 6: Uji Navigasi Lintasan Melingkar

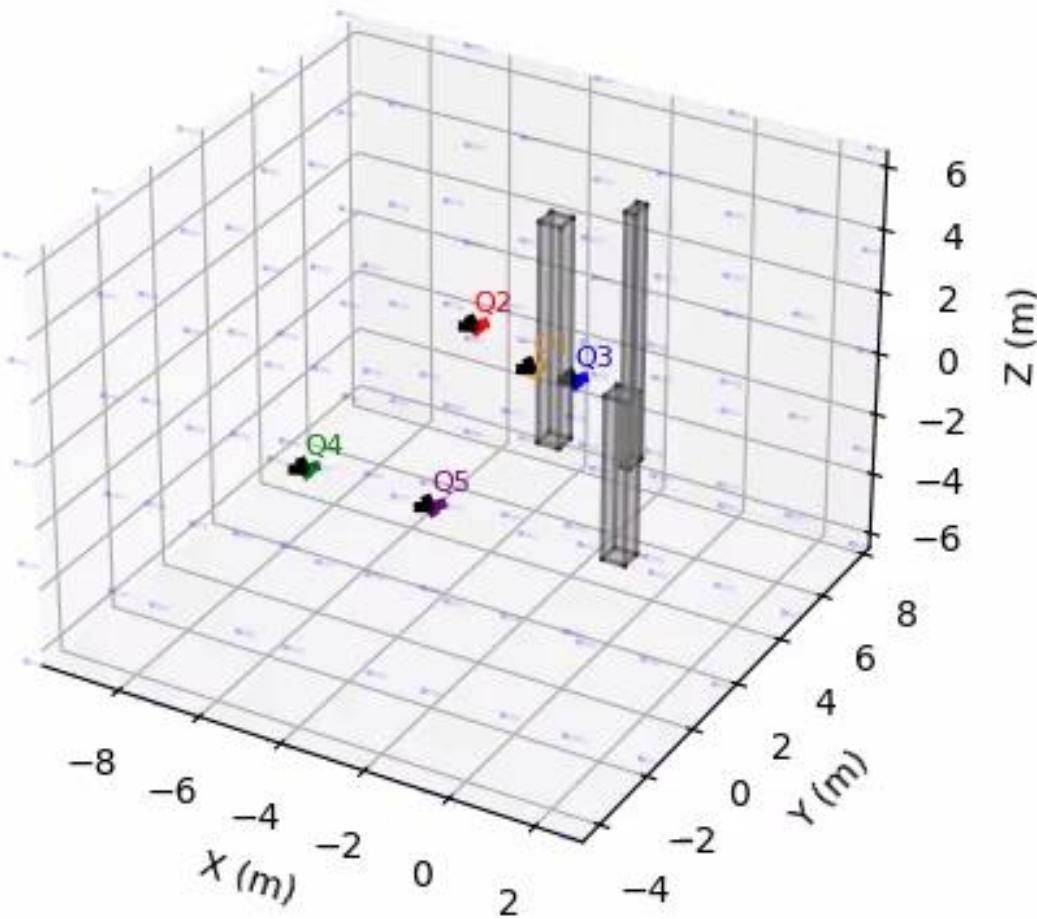
Kecepatan Kawan Berdasarkan Mode
Obs: scheme5, Wind: GUST, Contr: erc



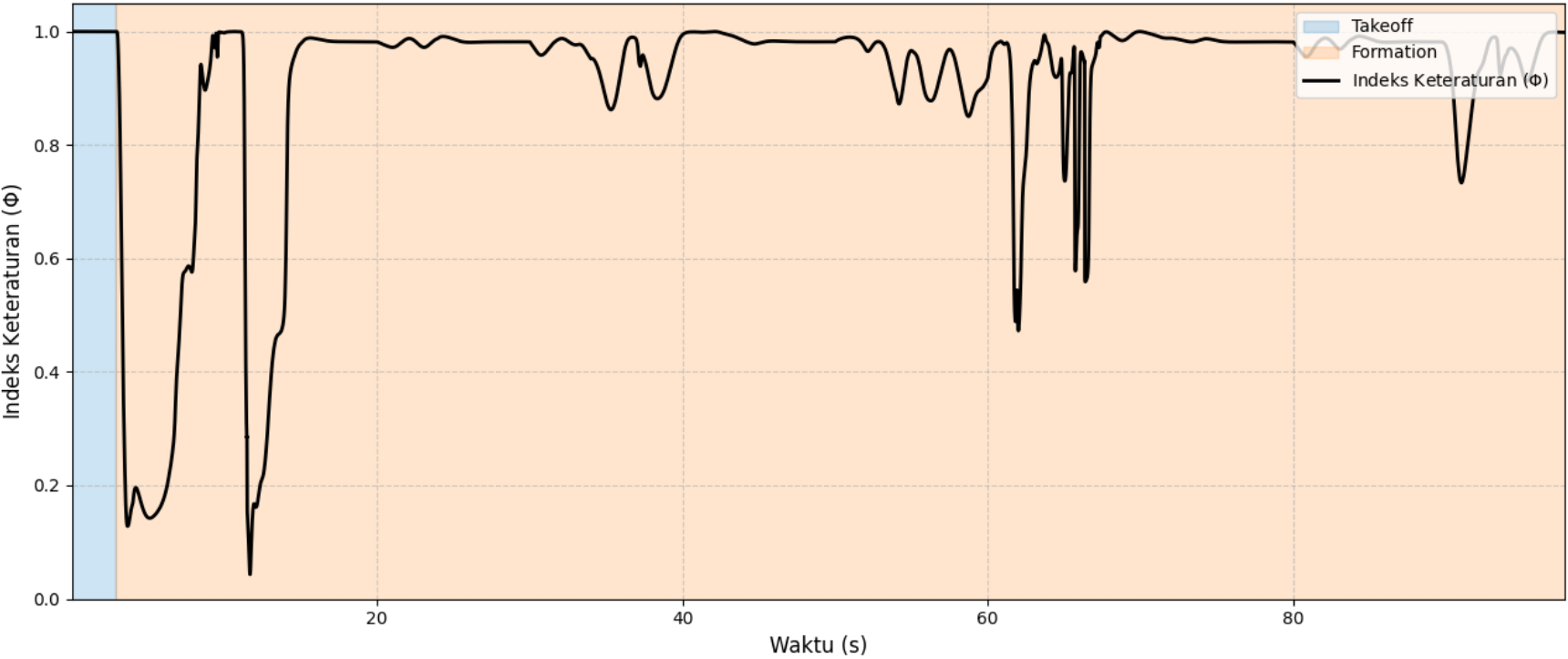
RMSE Akurasi Operasional Berdasarkan Mode
RMSE_{total} = 0.193 meter
Obs: scheme5, Wind: GUST, Contr: erc



3D Environment
Obs: scheme5 - Wind: GUST - Contr: erc
Formation: 1 - Orient: NED - Time: 0.10s



Indeks Keteraturan Sistem Berdasarkan Mode
 $\Phi = 0.908$
Obs: scheme5, Wind: GUST, Contr: erc



1. Platform Simulasi & Kontrol Dasar Berhasil

- Platform simulasi berbasis Python (OOP) dengan kontrol PID berhasil dibangun.
- Mampu menstabilkan 5 agen dari kondisi acak ke formasi dalam < 10 detik.

2. Strategi Adaptif (ERC) Jauh Lebih Unggul

- Strategi ERC berhasil melewati celah sempit, sementara metode statis (**IAPF**) gagal.
- Keberhasilan dicapai dengan adaptasi formasi (menyusut) dan beralih ke mode *tailgating*.
- Misi selesai dalam 59.735 detik dengan menjaga akurasi ($\overline{RMSE}_{total} = 0.692$ m) dan kekohesifan tinggi ($\bar{\Phi} = 0.878$).

3. Sangat Tahan Terhadap Gangguan Angin

- Waktu penyelesaian misi **tidak terpengaruh** oleh hembusan angin (tetap 59.735 detik).
- Sistem secara aktif meningkatkan *effort* kontrol untuk meredam gangguan, sementara akurasi ($\overline{RMSE}_{total} = 0.694$) dan kekohesifan tetap terjaga ($\bar{\Phi} = 0.925$).

4. High-Level Planner Berfungsi Baik untuk Misi Kompleks

- **Navigasi Waypoint** berhasil diselesaikan dalam 76.695 detik dengan akurasi ($\overline{RMSE}_{total} = 0.338$ m) dan kekohesifan tinggi ($\bar{\Phi} = 0.810$).
- **Lintasan Melingkar** berhasil dilacak selama 97.795 detik dengan akurasi sangat tinggi ($\overline{RMSE}_{total} = 0.193$ m) dan koordinasi nyaris sempurna ($\bar{\Phi} = 0.908$).

5. Batasan Sistem Berhasil Diidentifikasi

- Sistem gagal pada skenario **U-Trap** (jebakan topologis).
- Kegagalan terjadi karena mekanisme adaptasi ERC tidak terpicu saat terjebak dalam lokal minimum.

1. Mengatasi Jebakan Topologis (*U-Trap*)

2. Navigasi dengan Rintangan Dinamis

3. Manuver Reaktif 3D

4. Implementasi Formasi 3D

5. Menggunakan Kontroler Adaptif

6. Simulasi Kondisi Non-Ideal

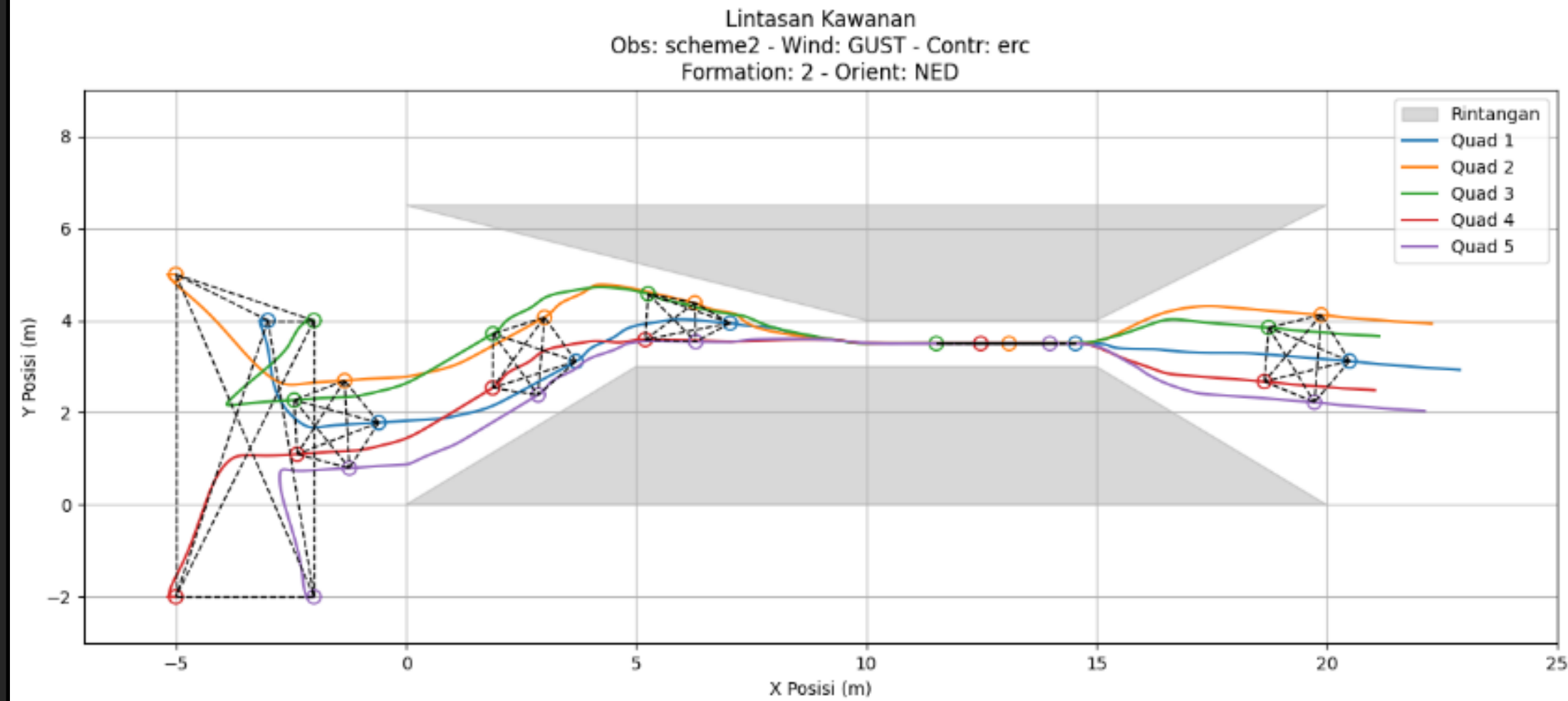
7. Validasi pada Platform Fisik

bit.ly/Vid_13321027

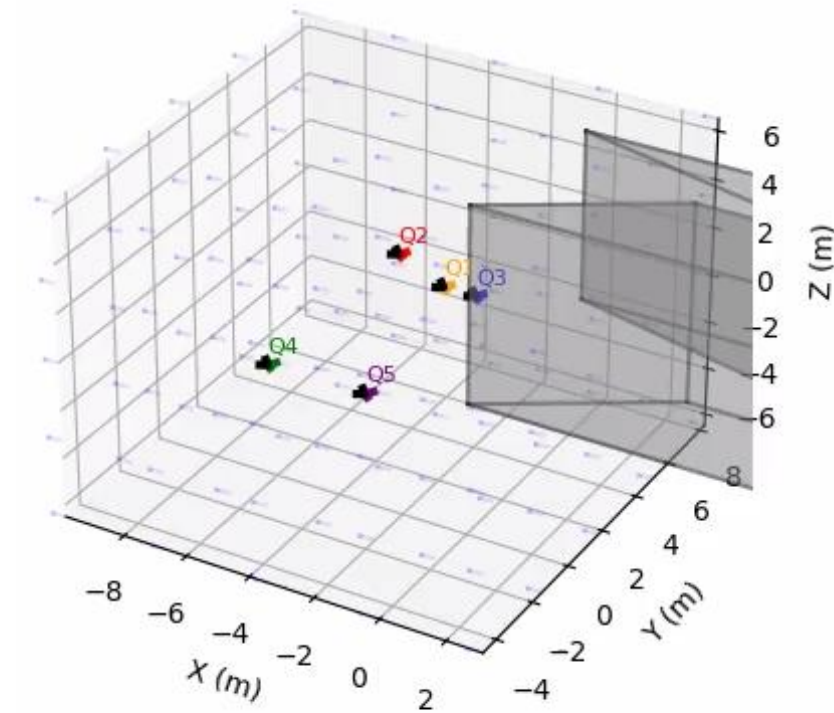


Terima
Kasih

Grafik *High-Level Planner* Poligon

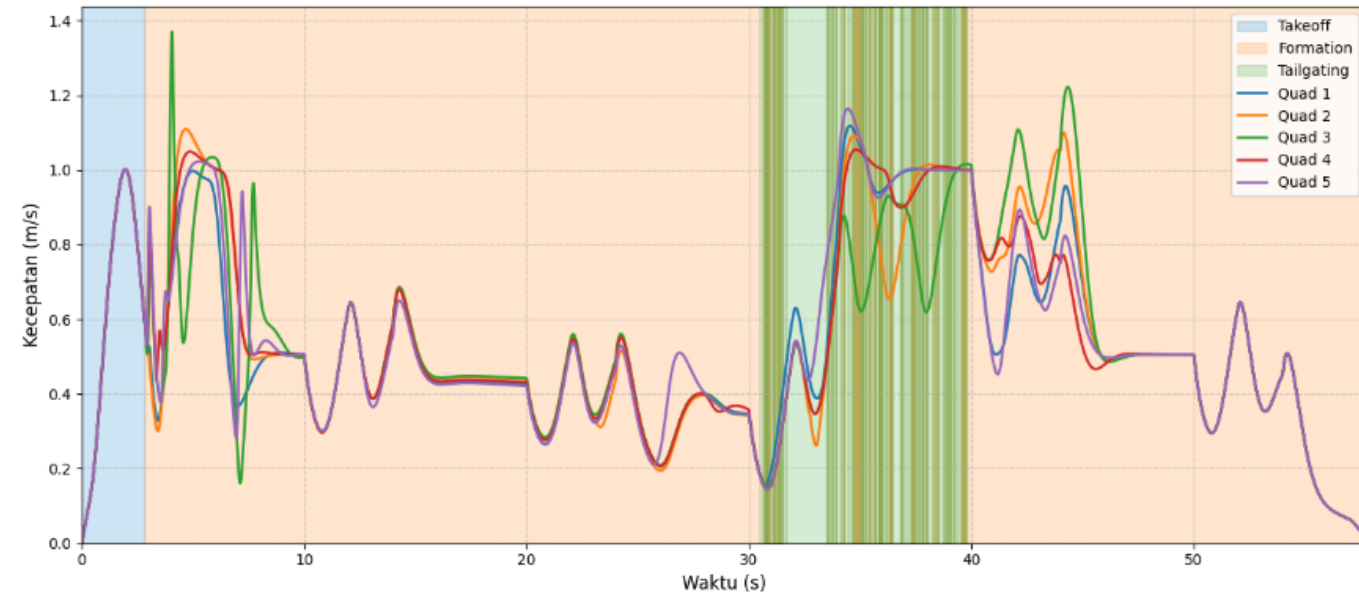


3D Environment
Obs: scheme2 - Wind: GUST - Contr: erc
Formation: 2 - Orient: NED - Time: 0.10s

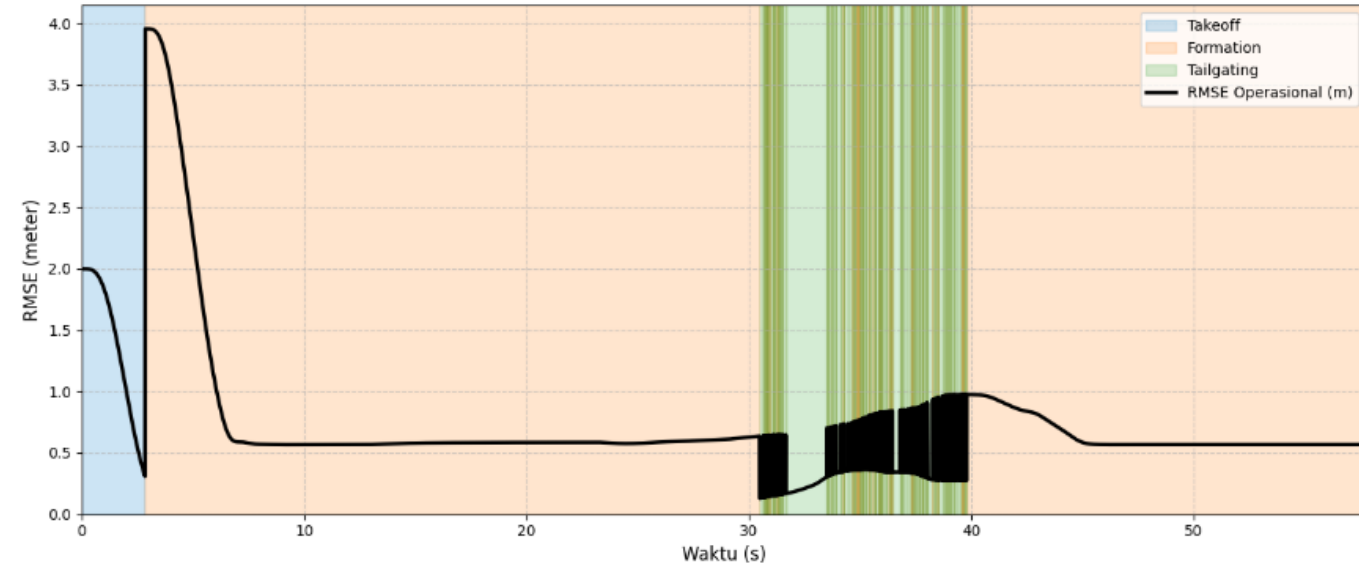


Misi selesai dalam
57.765 detik

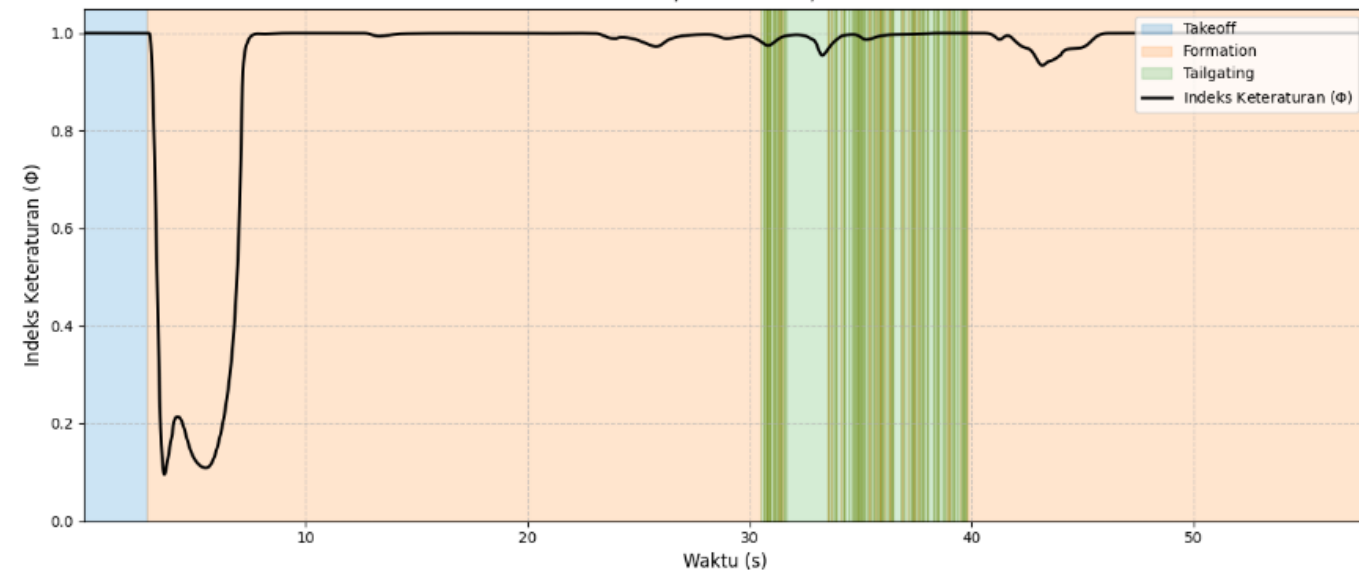
Kecepatan Kawan Berdasarkan Mode
Obs: scheme2, Wind: GUST, Contr: erc



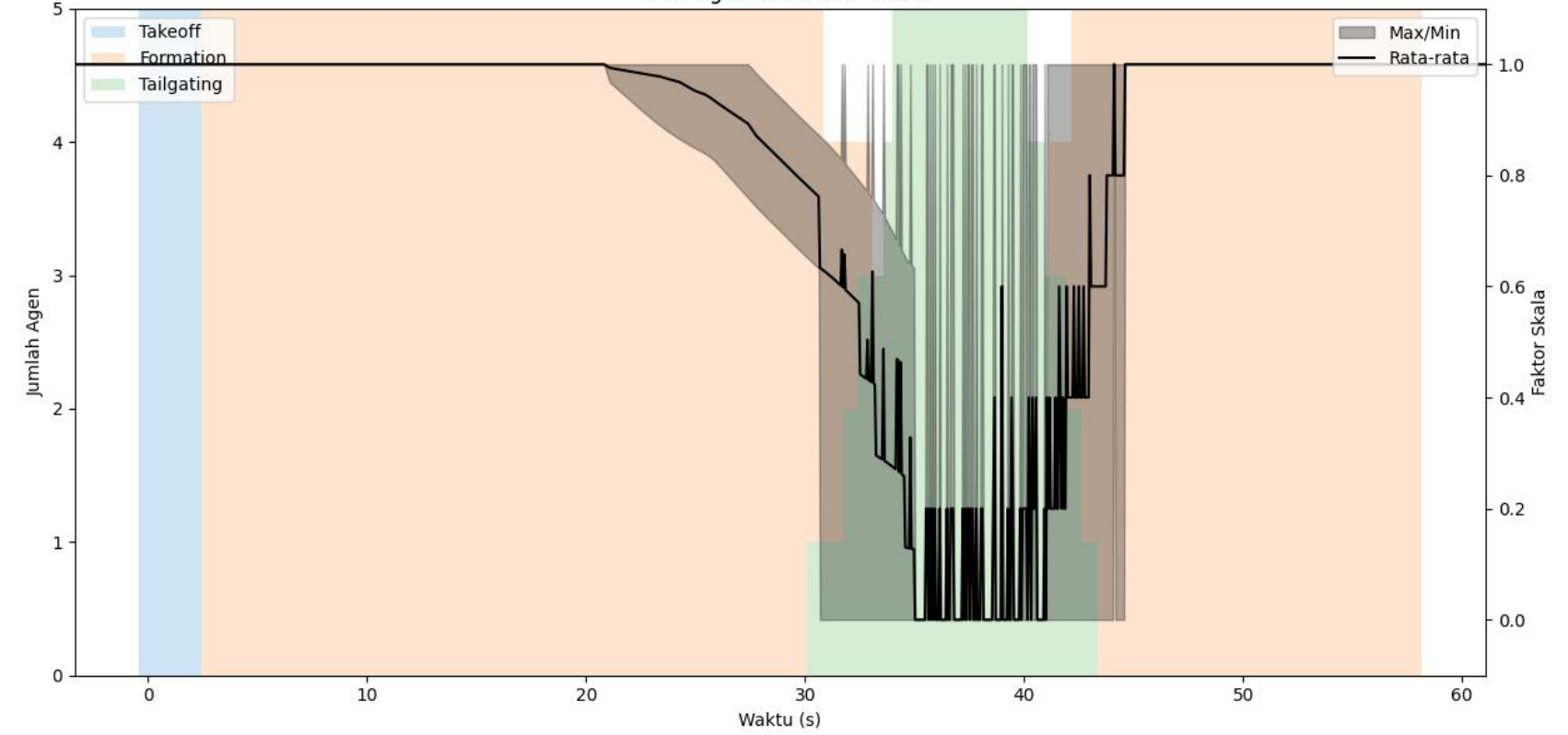
RMSE Akurasi Operasional Berdasarkan Mode
RMSE_{total} = 0.725 meter
Obs: scheme2, Wind: GUST, Contr: erc



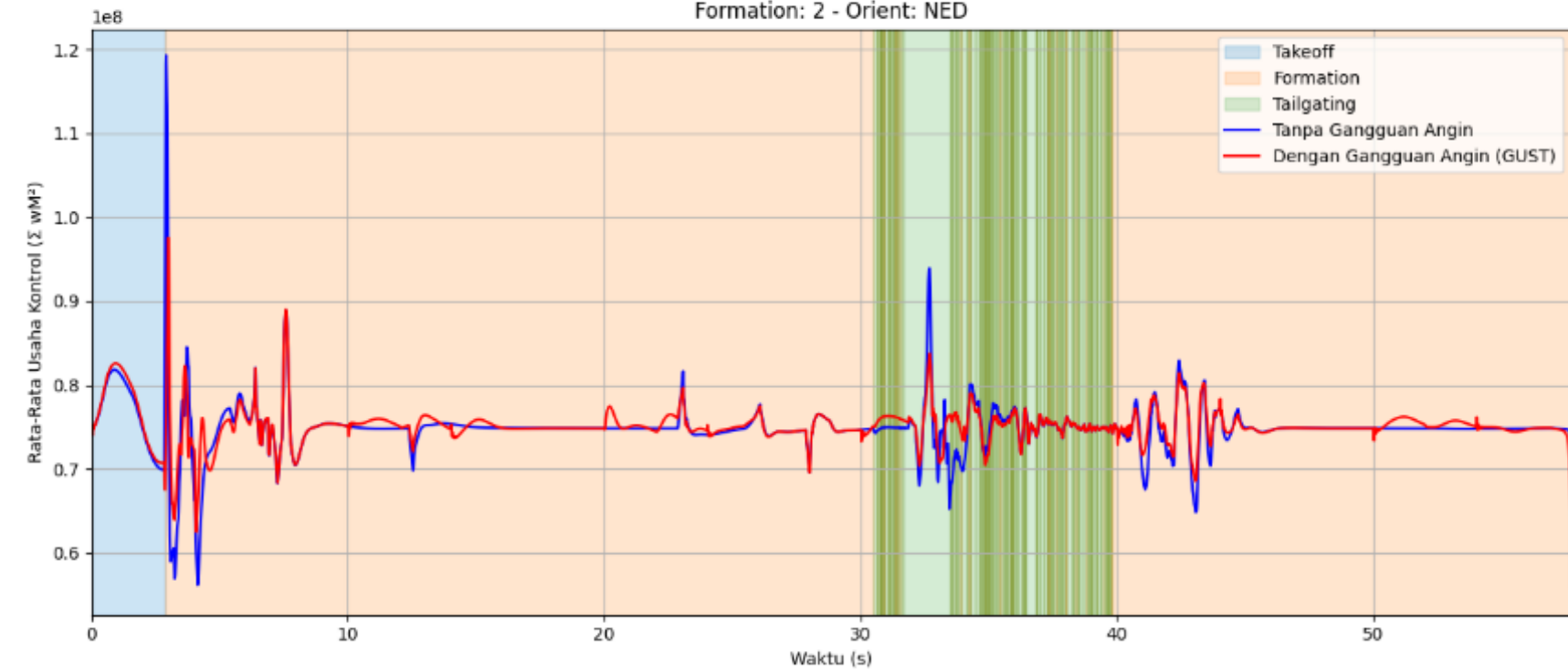
Indeks Keteraturan Sistem Berdasarkan Mode
 $\Phi = 0.941$
Obs: scheme2, Wind: GUST, Contr: erc



Mode Agen dan Faktor Skala



Perbandingan Usaha Kontrol
Obs: scheme2 - Wind: GUST - Contr: erc
Formation: 2 - Orient: NED



**Grafik *Low-Level*
Controller V-shape**
tiap Quadcopter

Untuk **quadcopter 1** saja

Hasilnya mirip untuk quadcopter lainnya

Error Posisi

Posisi Aktual vs Setpoint

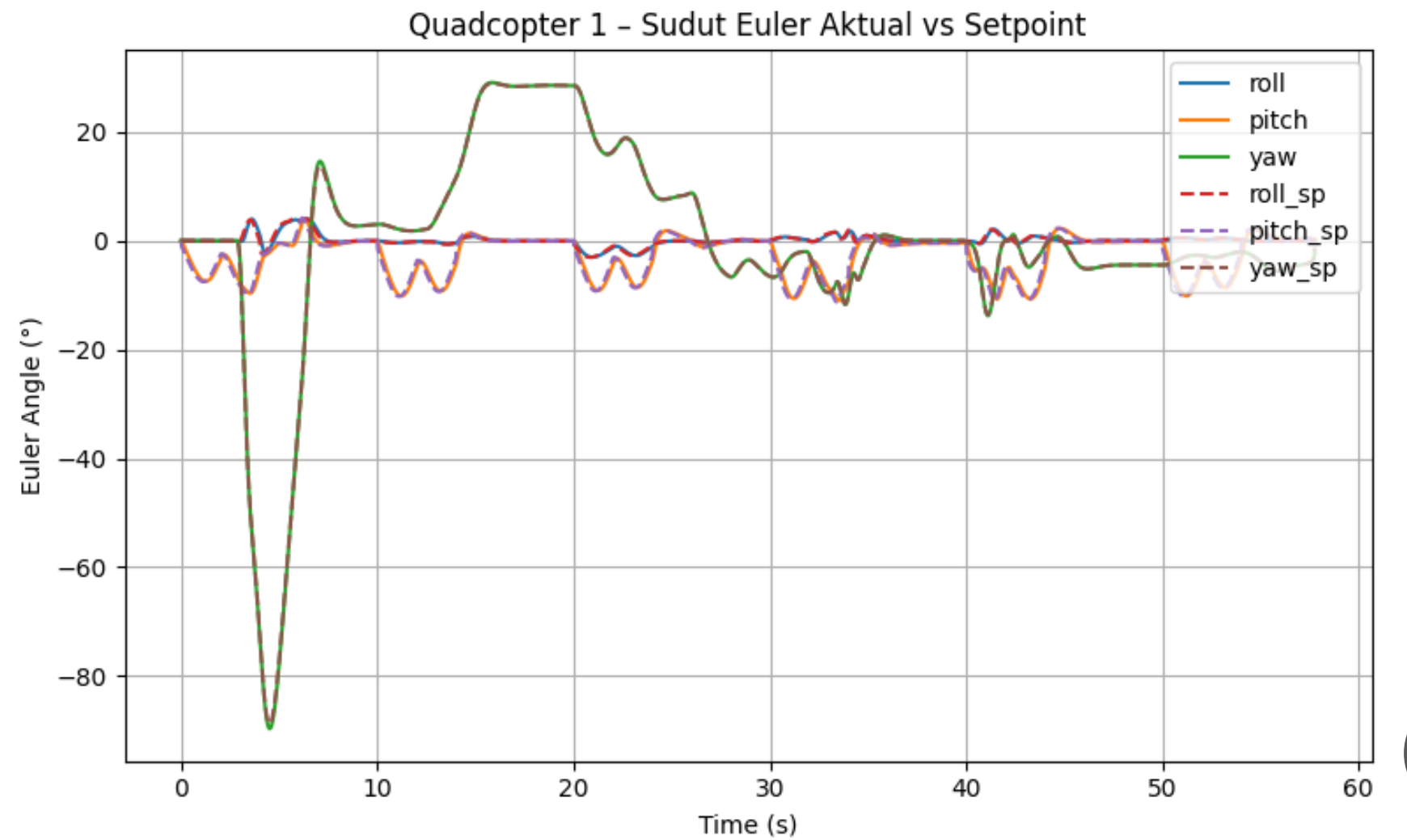
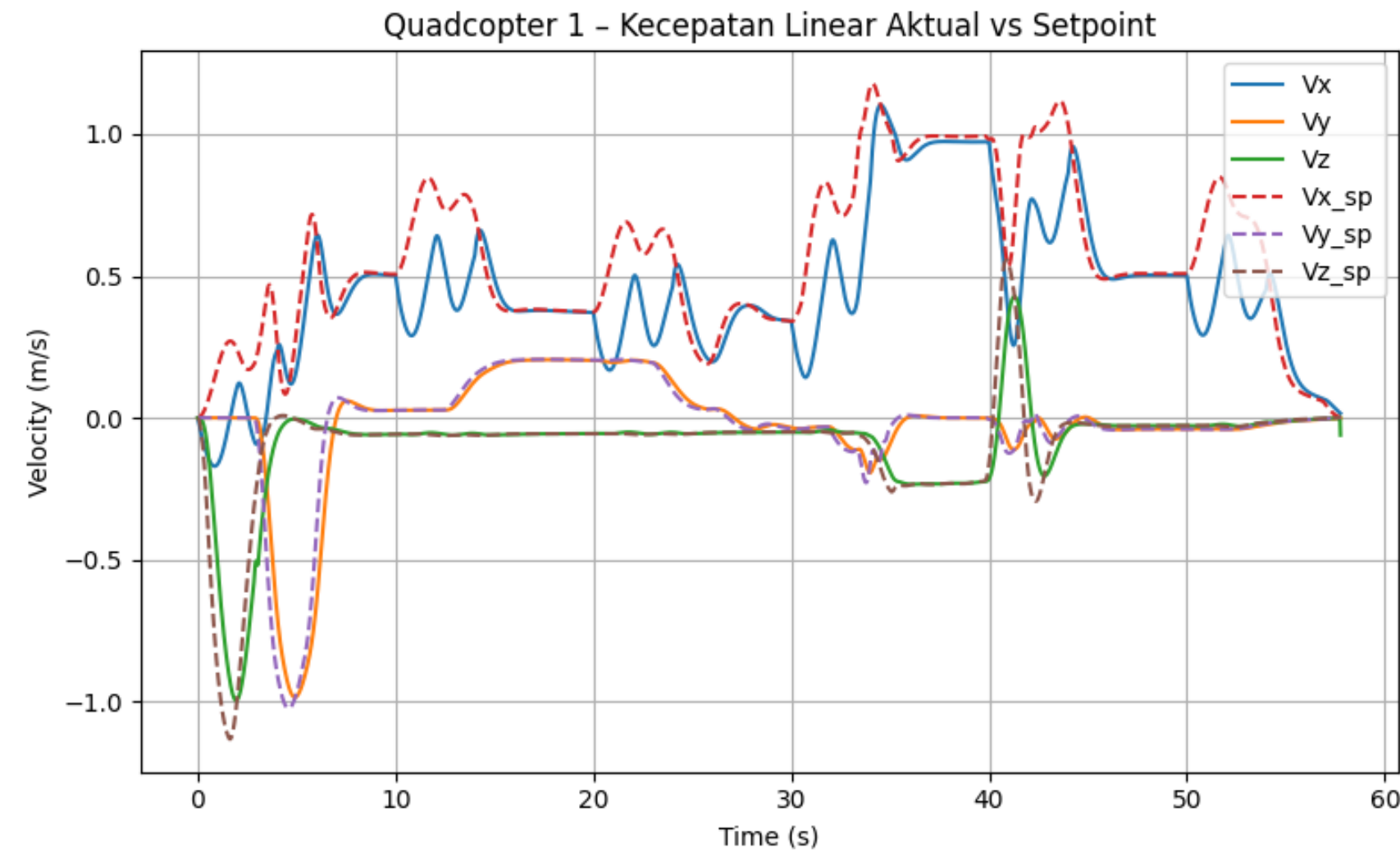
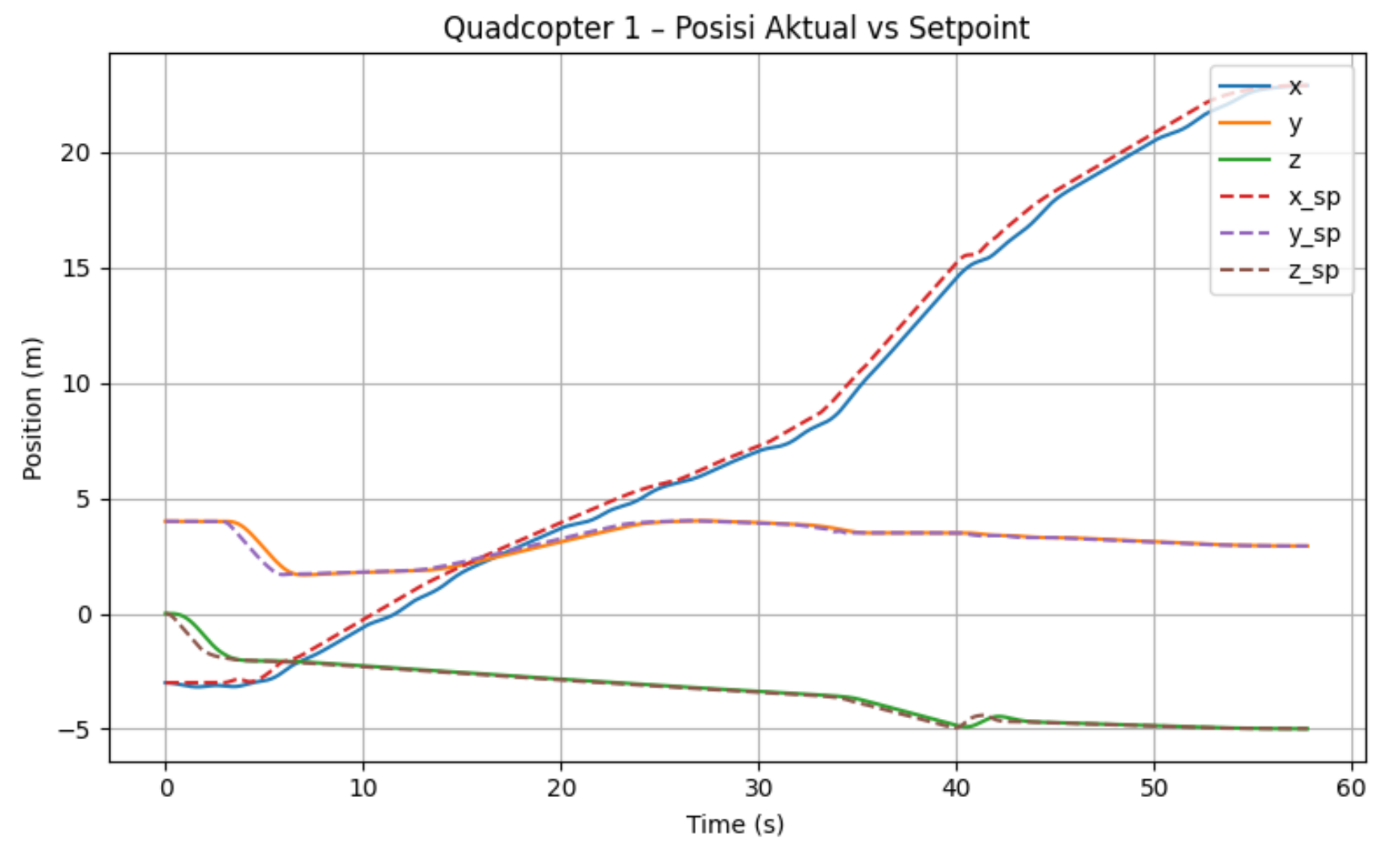
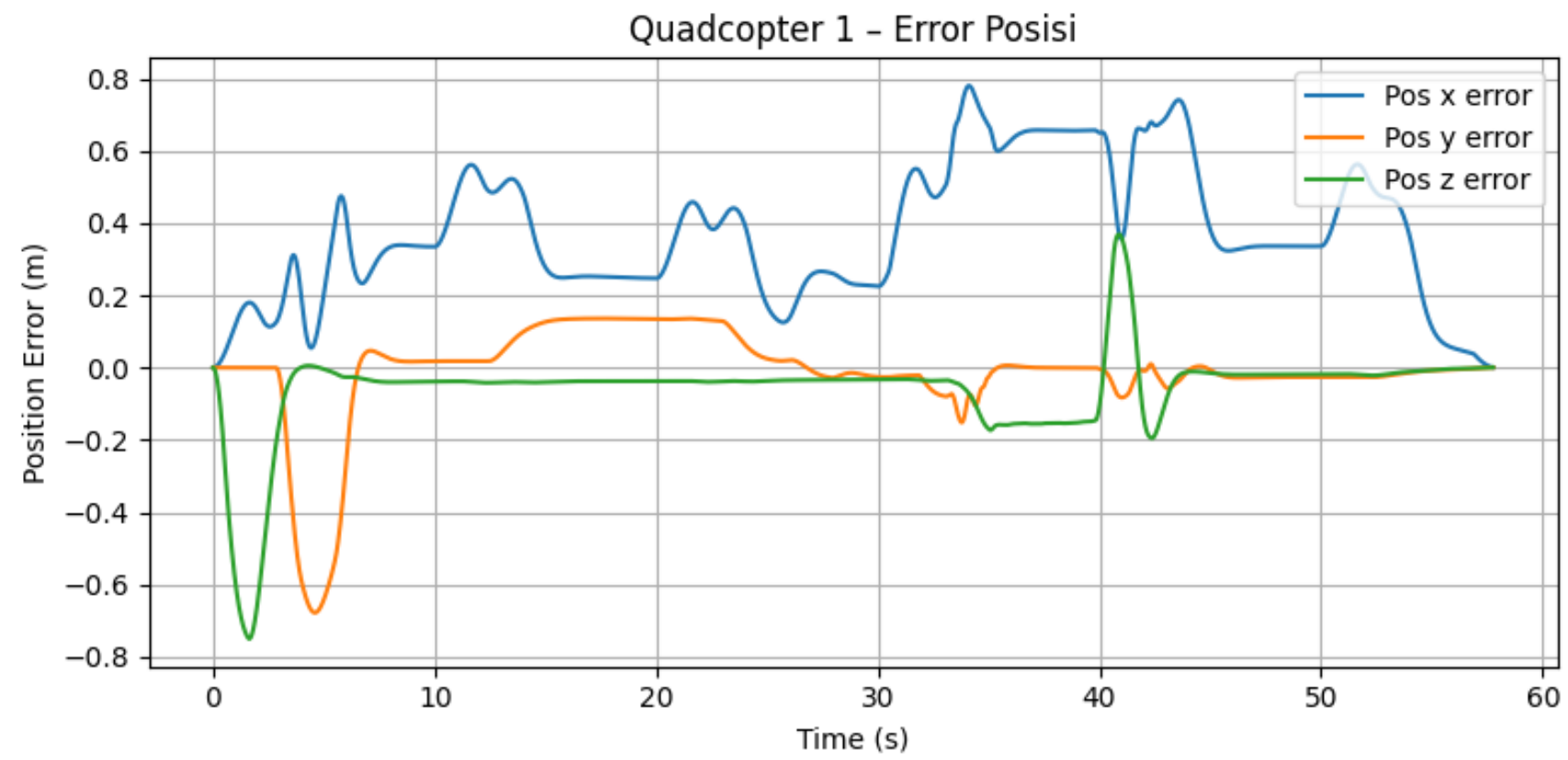
Kecepatan Linear vs Setpoint

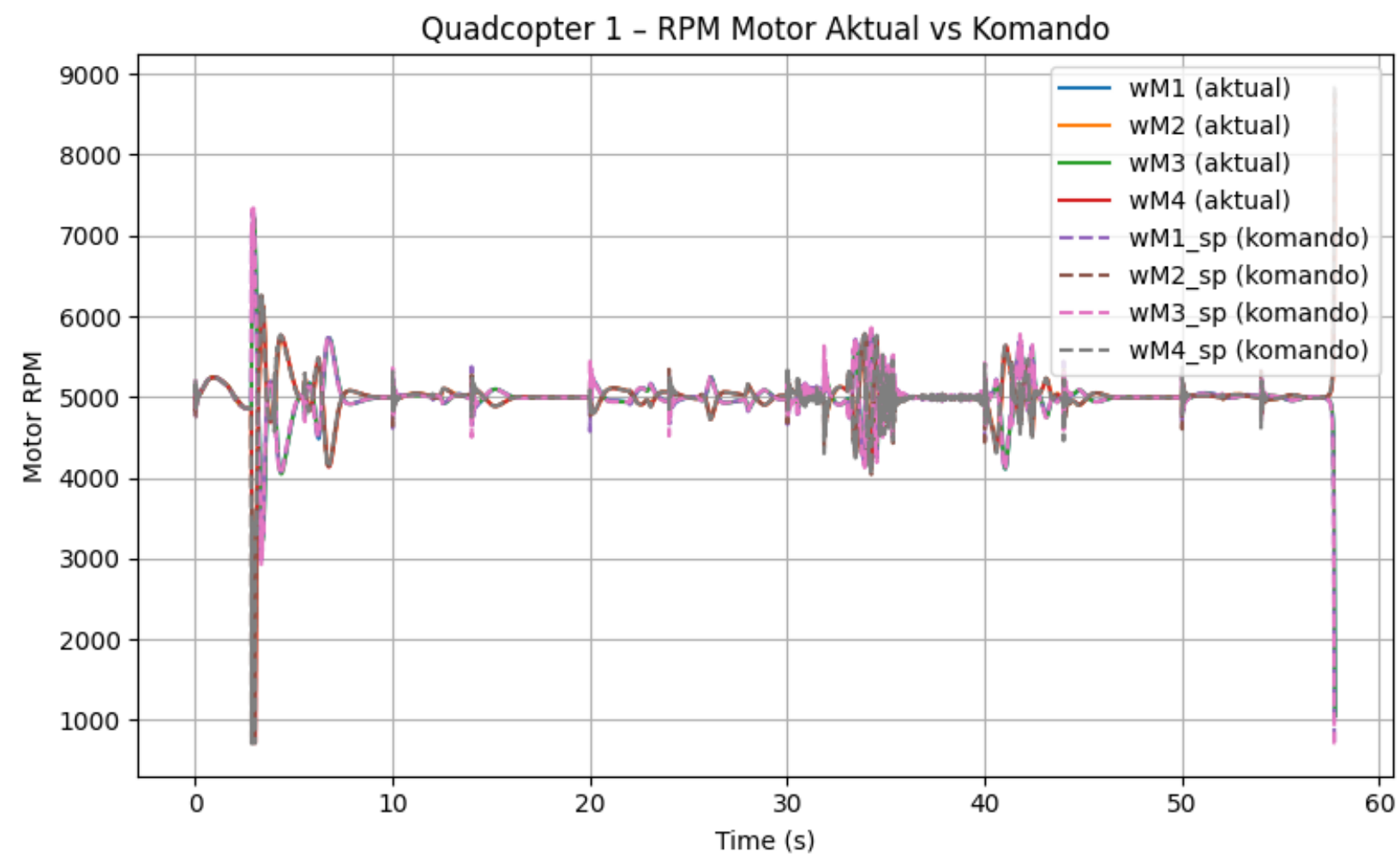
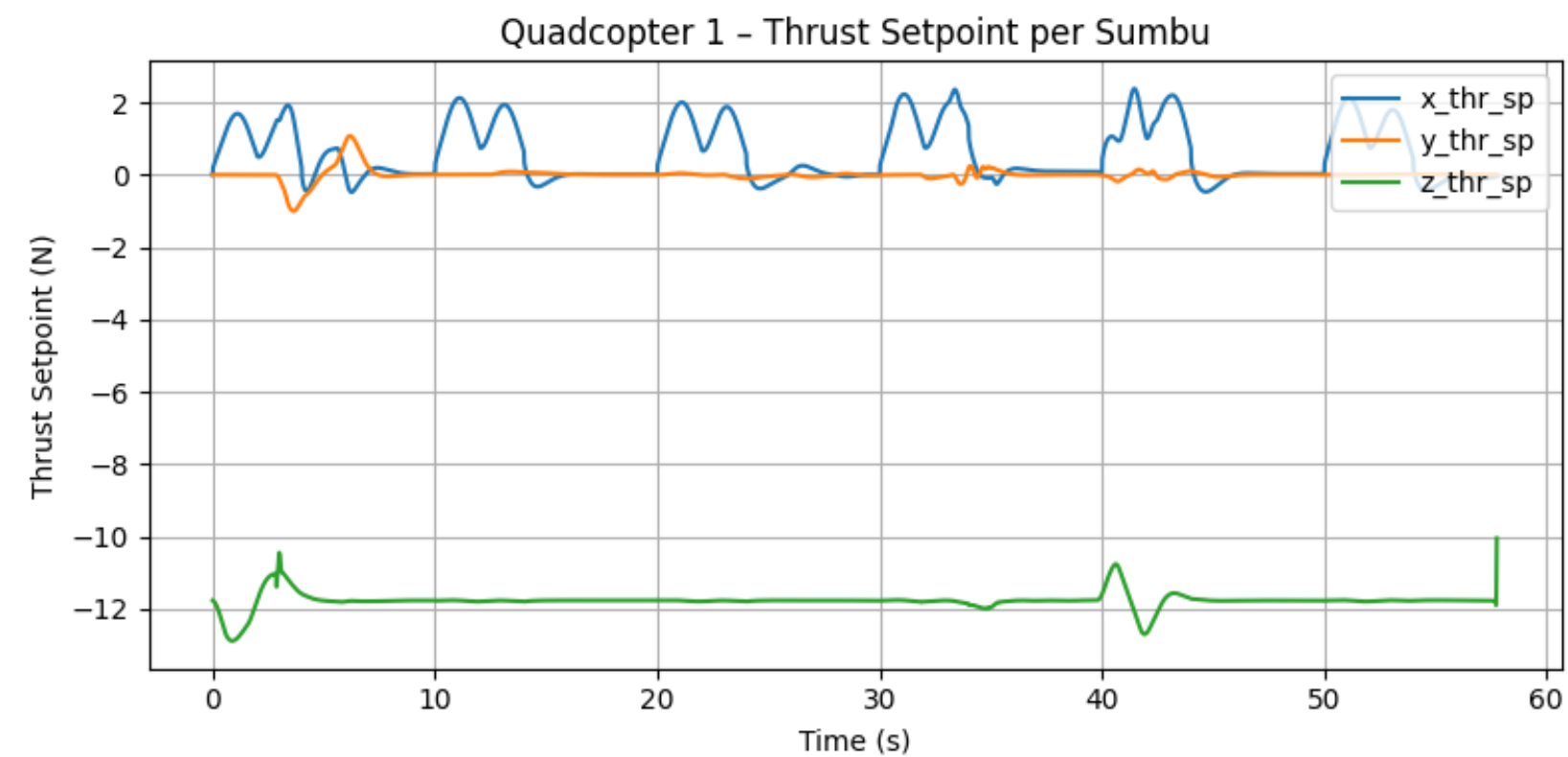
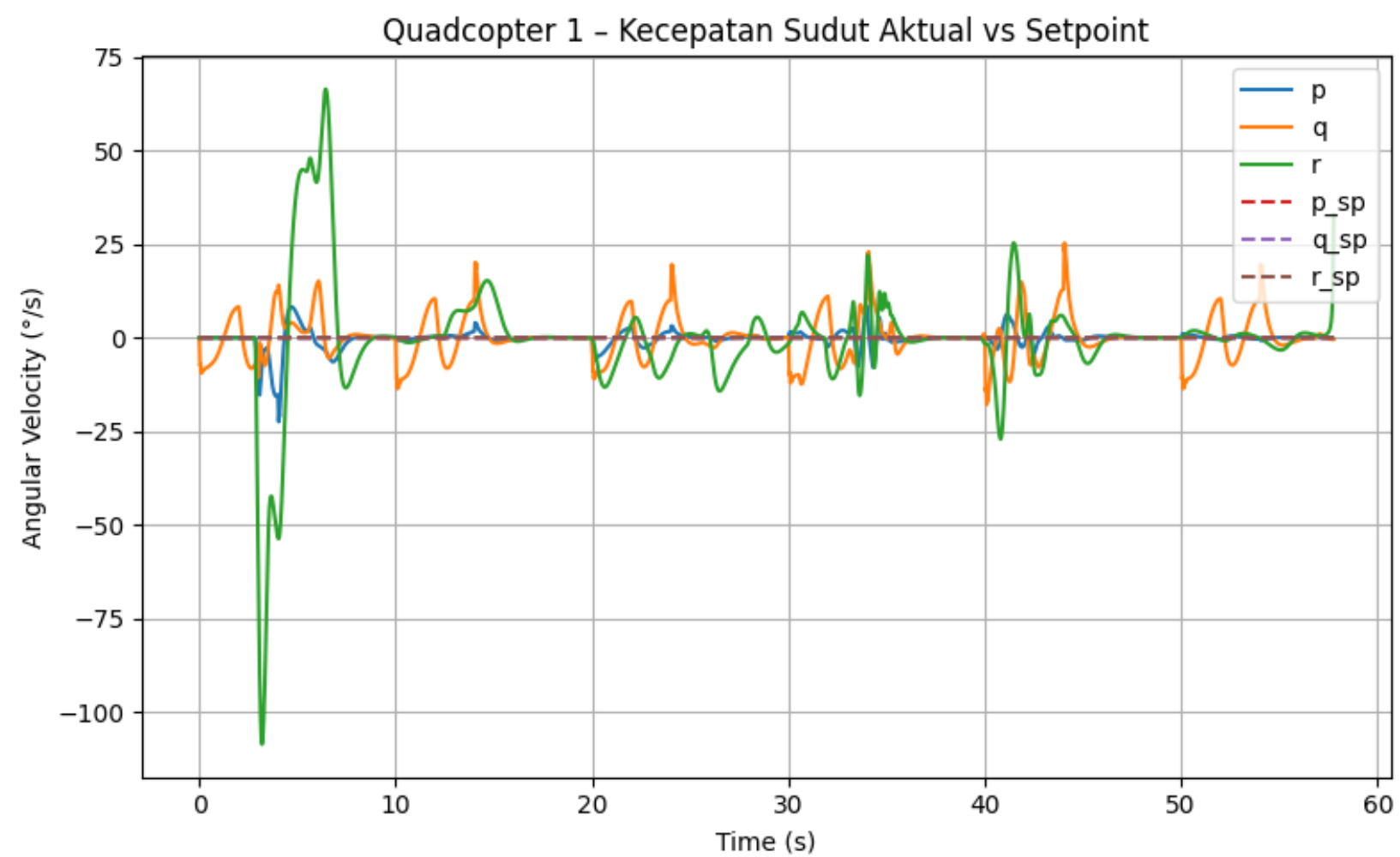
Sudut Euler vs Setpoint

Kecepatan Sudut Aktual vs Setpoint

Thrust Setpoint per Sumbu

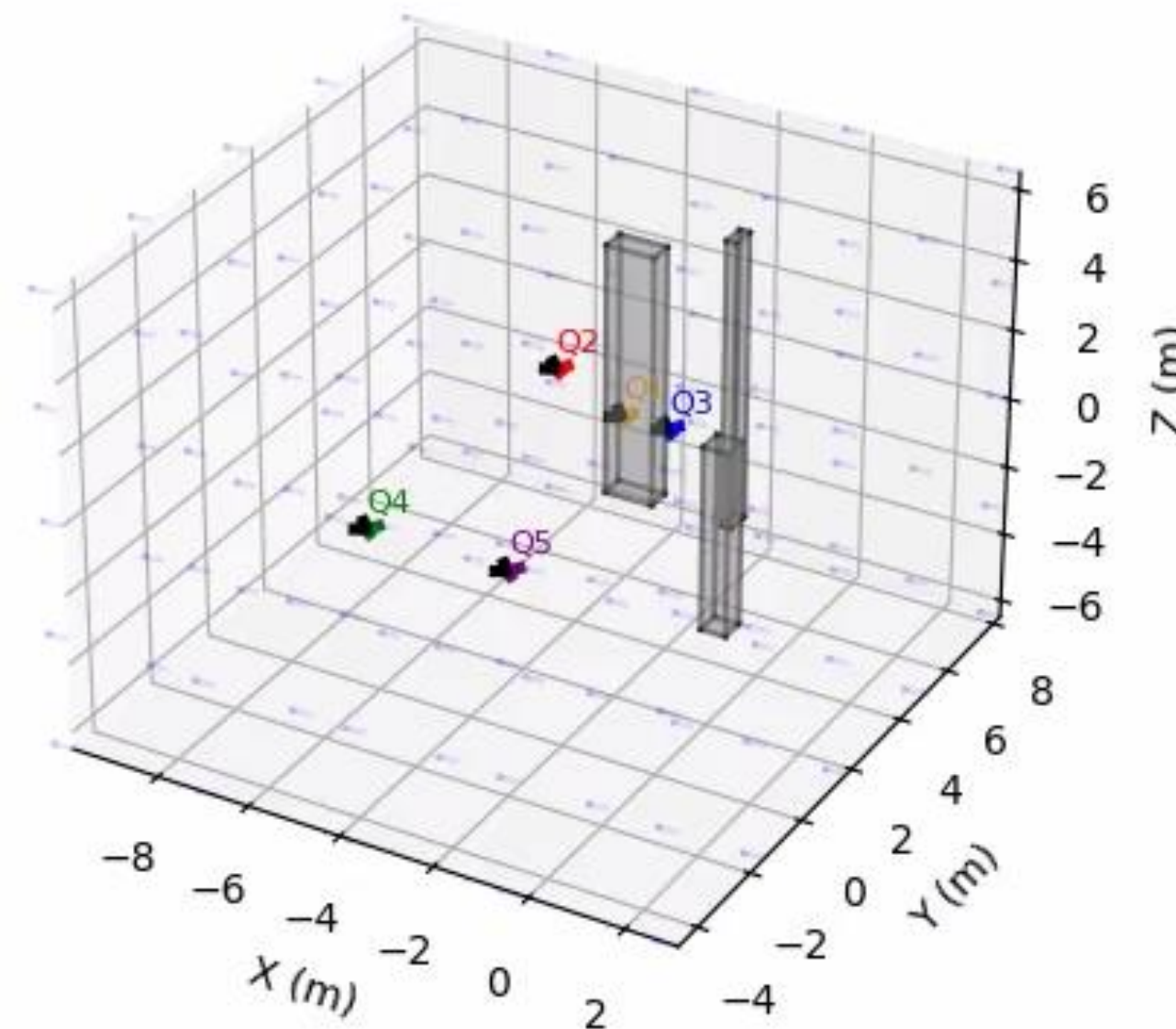
RPM Motor Aktual vs Komando



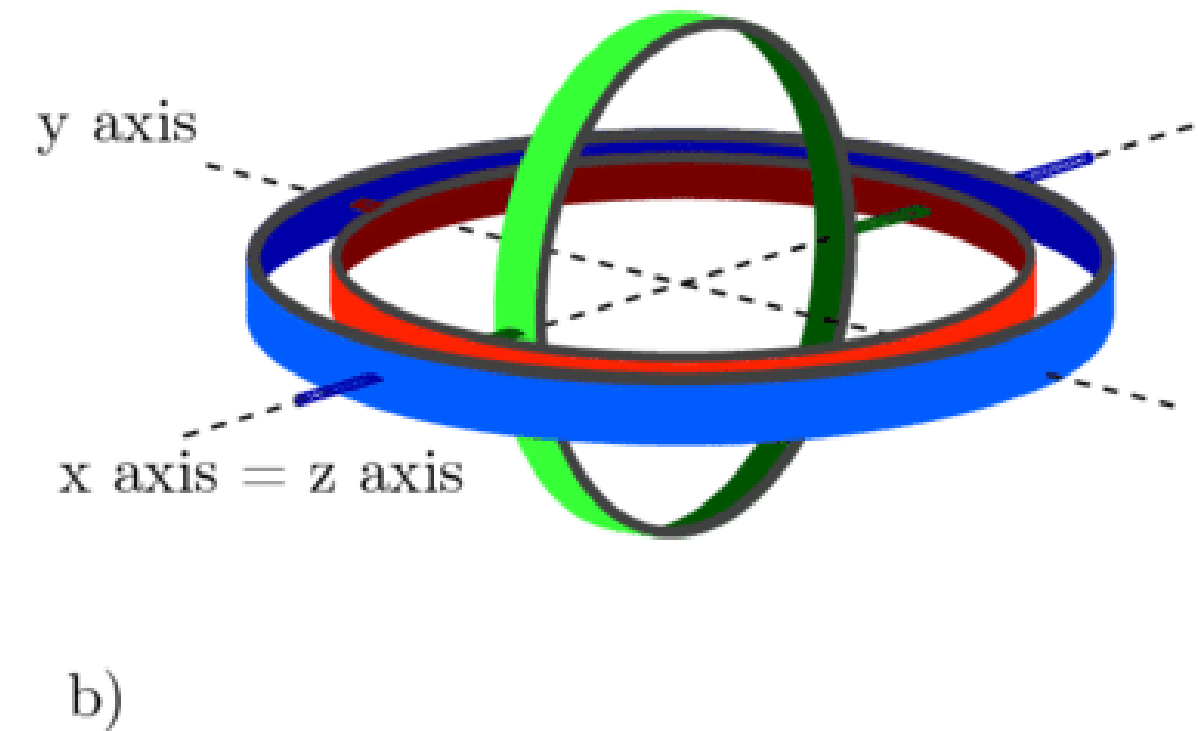
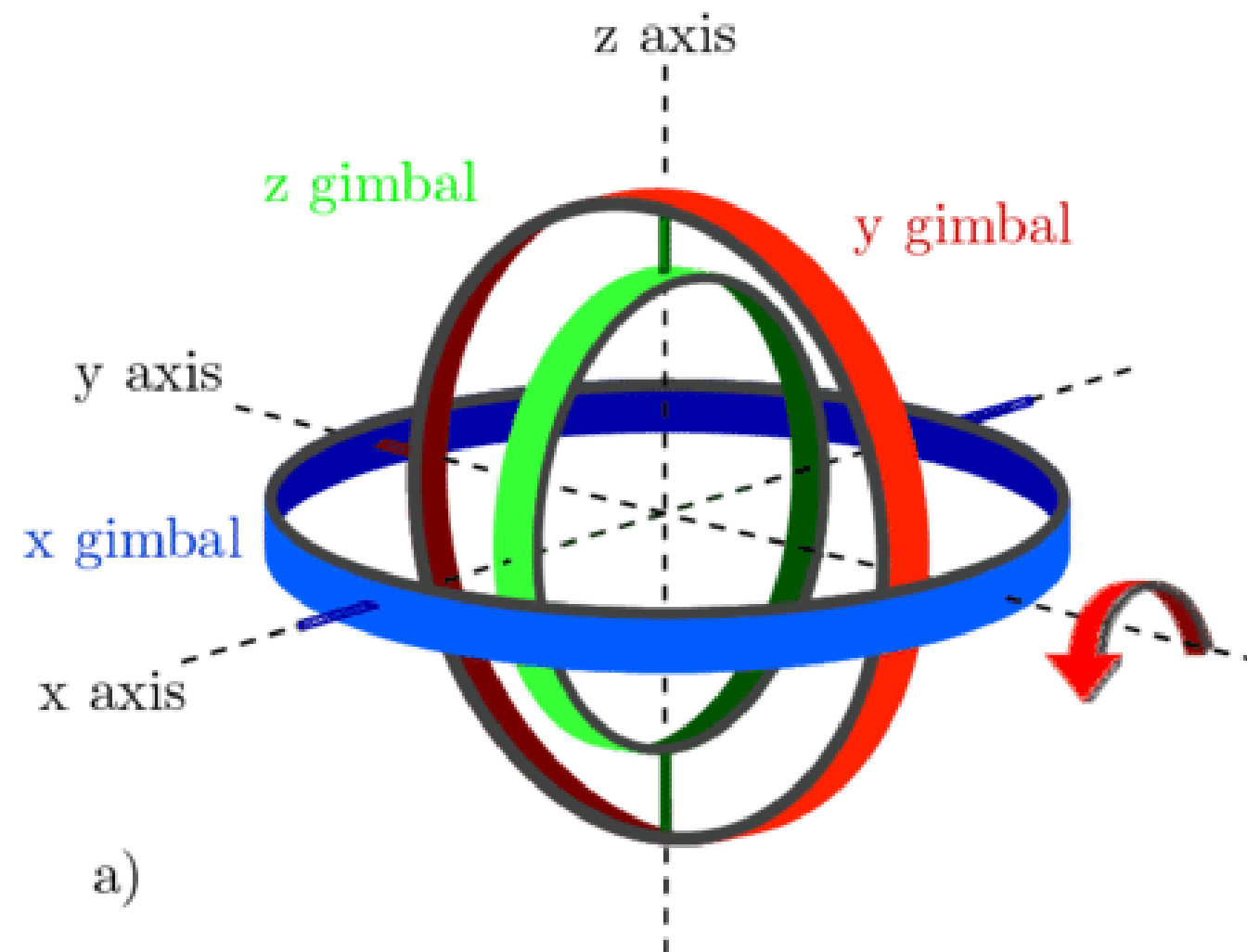


Yaw tetap

3D Environment
Obs: scheme4 - Wind: GUST - Contr: erc
Formation: 1 - Orient: NED - Time: 0.10s

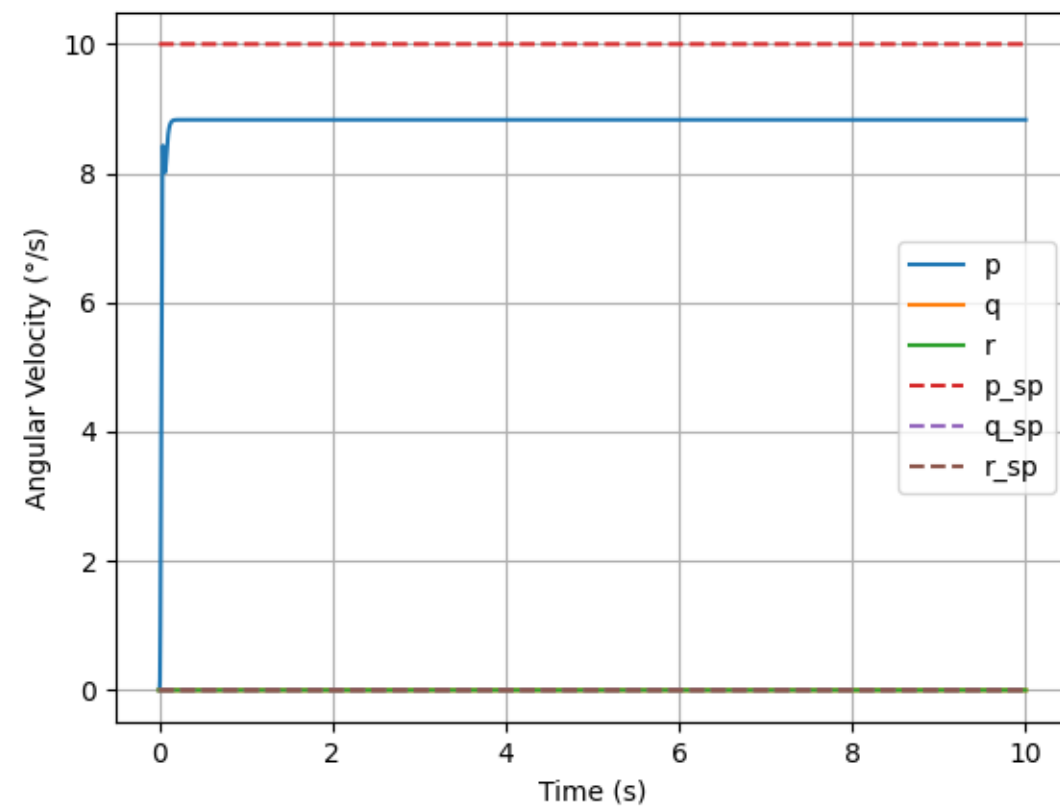


Quaternion



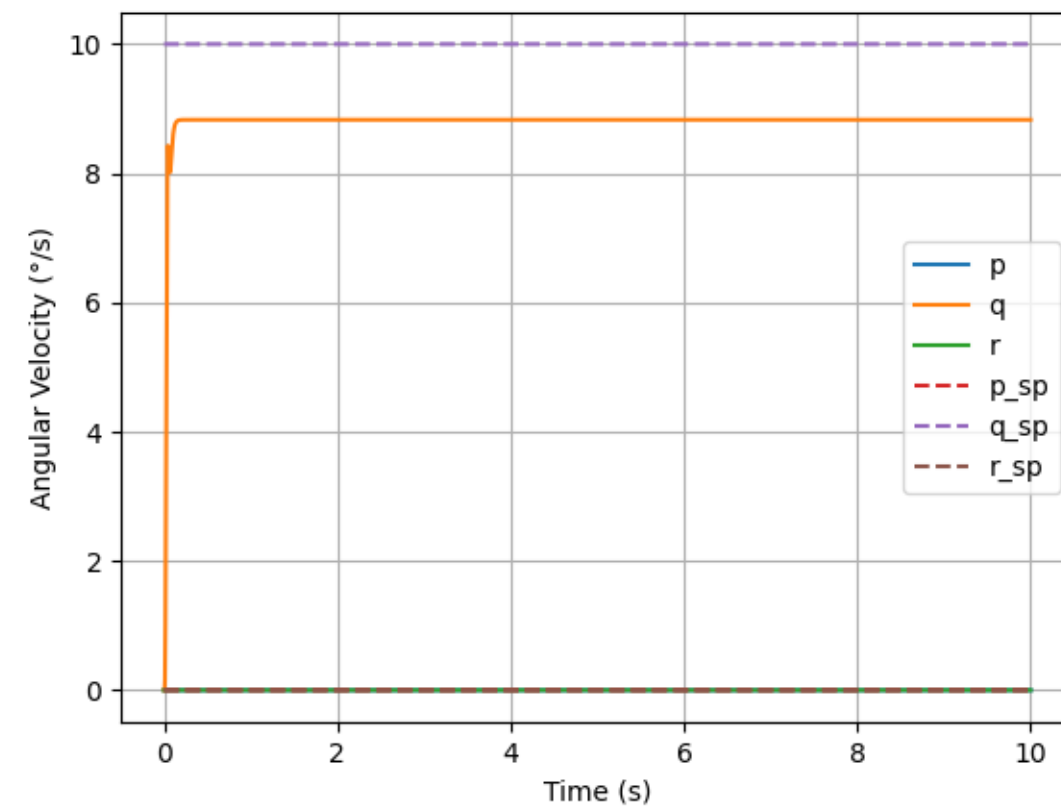
Tuning

**Tuning dari loop terdalam ke loop terluar
(*inside-out tuning*).**



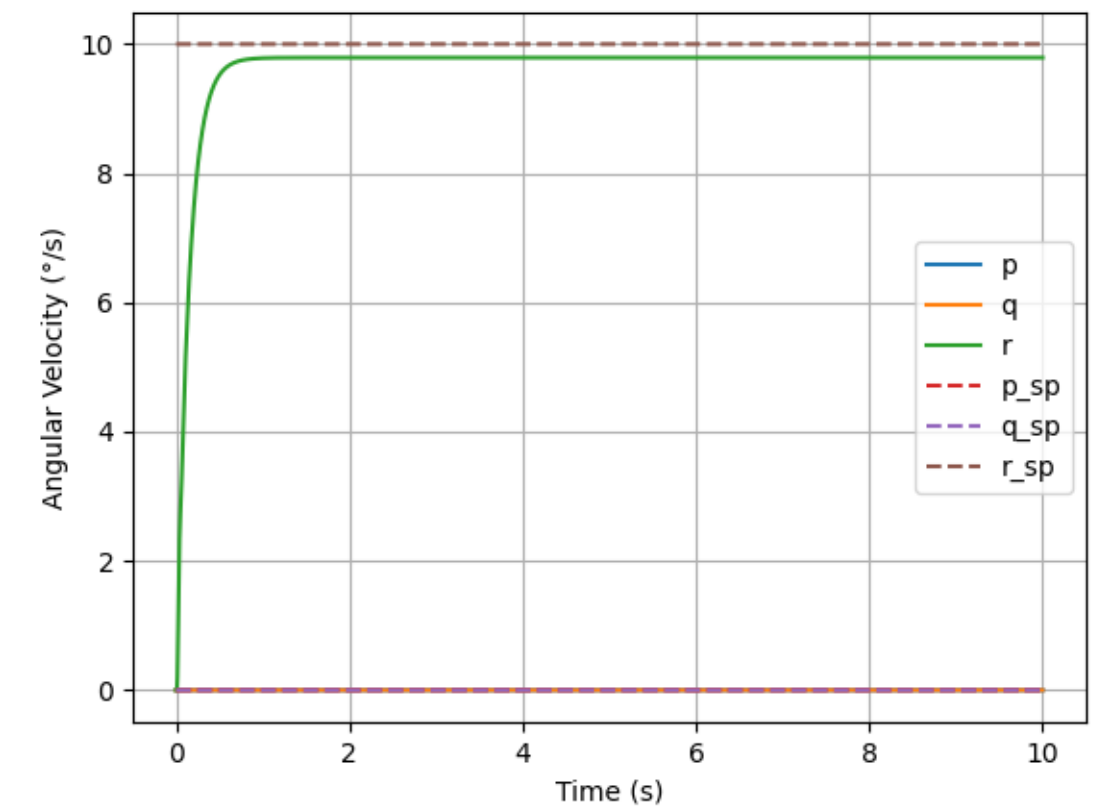
p Analysis:

- Overshoot : -11.72%
- Rise Time : 0.130 s
- Settling Time : nan s
- Peak Time : 0.615 s
- Steady-state err : -1.172



q Analysis:

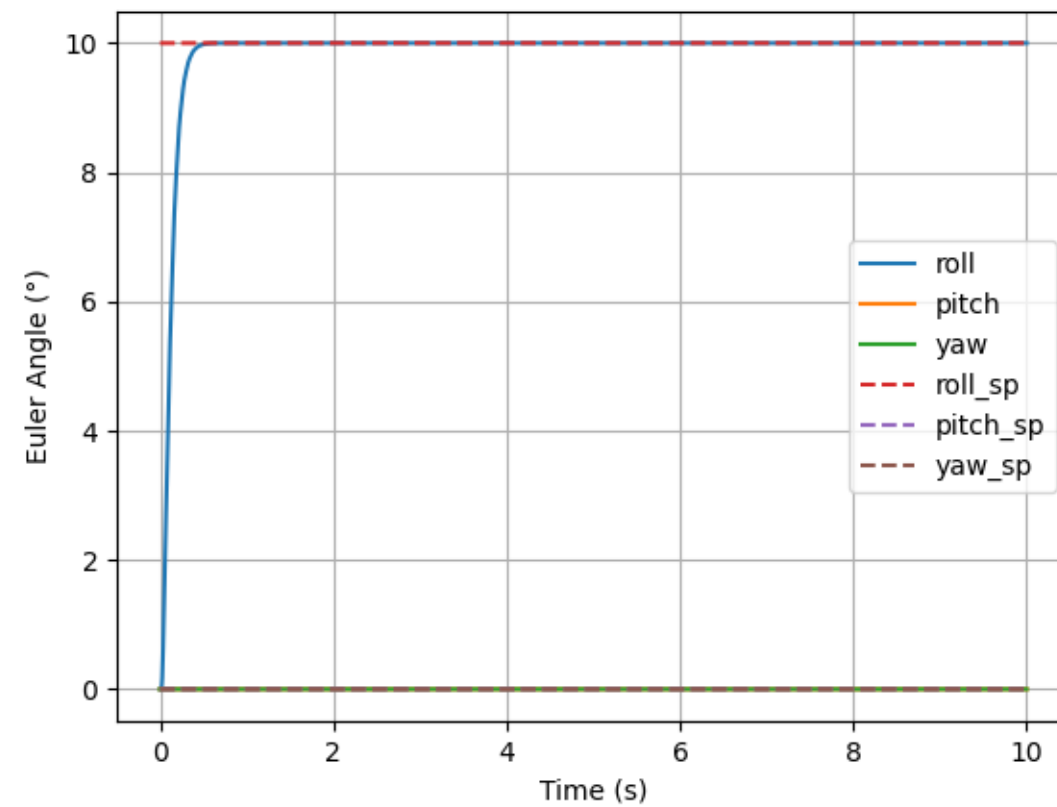
- Overshoot : -11.72%
- Rise Time : 0.130 s
- Settling Time : nan s
- Peak Time : 0.615 s
- Steady-state err : -1.172



r Analysis:

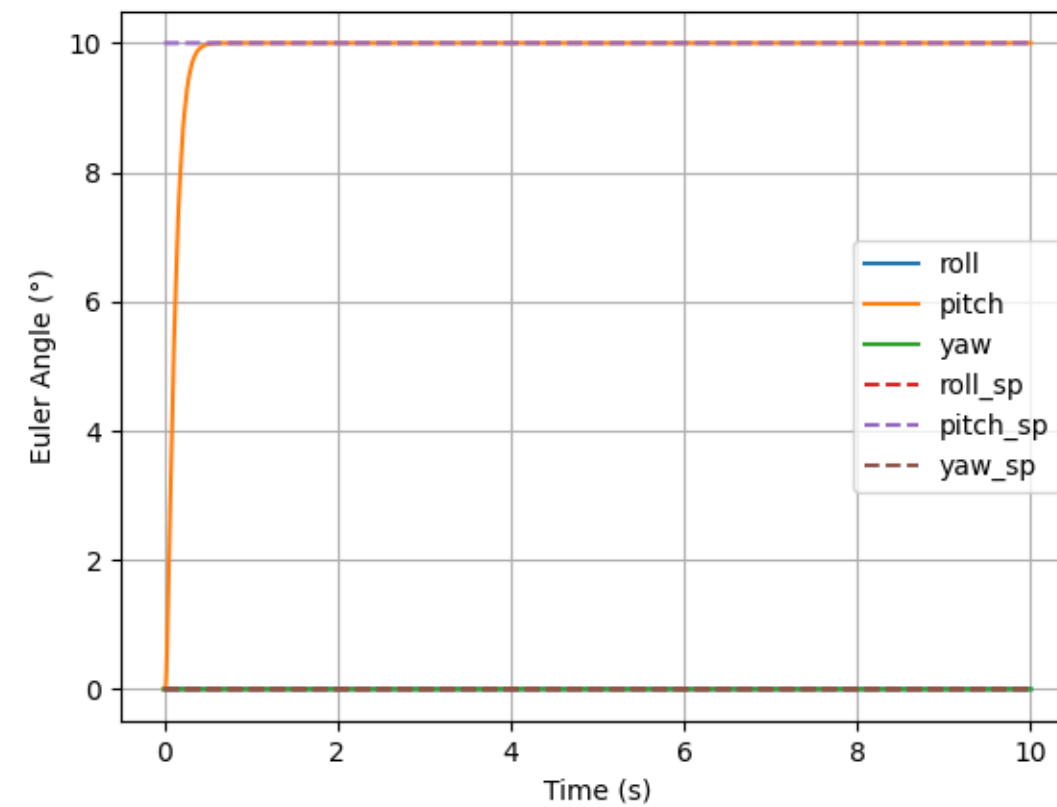
- Overshoot : -2.10%
- Rise Time : 0.300 s
- Settling Time : nan s
- Peak Time : 4.690 s
- Steady-state err : -0.210

Tujuan utama dari penalaan rate controller ini adalah untuk mendapatkan respons yang cepat dan stabil, dan ketiga grafik ini menunjukkan tujuan tersebut berhasil dicapai. Respons pada semua sumbu (roll, pitch, yaw) naik dengan cepat tanpa overshoot atau osilasi. Adanya sedikit steady-state error yang konsisten di semua sumbu adalah karakteristik yang wajar dari kontroler PD yang digunakan. Tanpa adanya suku Integral (I), kontroler memerlukan sedikit error untuk terus menghasilkan torsi yang dibutuhkan untuk melawan hambatan aerodinamis saat berputar. Hasil ini menunjukkan sebuah trade-off yang baik, di mana kecepatan dan stabilitas respons lebih diutamakan, menjadikan fondasi kontrol ini sangat solid.



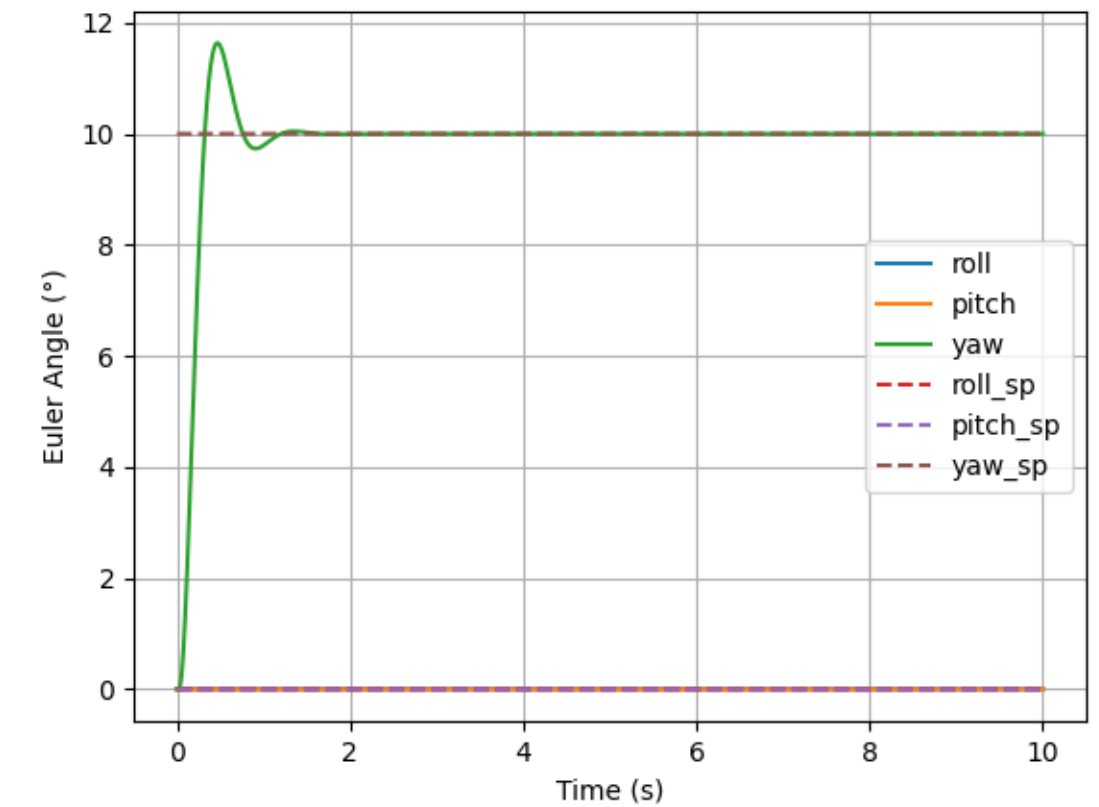
phi Analysis:

- Overshoot	: 0.00%
- Rise Time	: 0.190 s
- Settling Time	: 0.355 s
- Peak Time	: 0.985 s
- Steady-state err	: 0.000



theta Analysis:

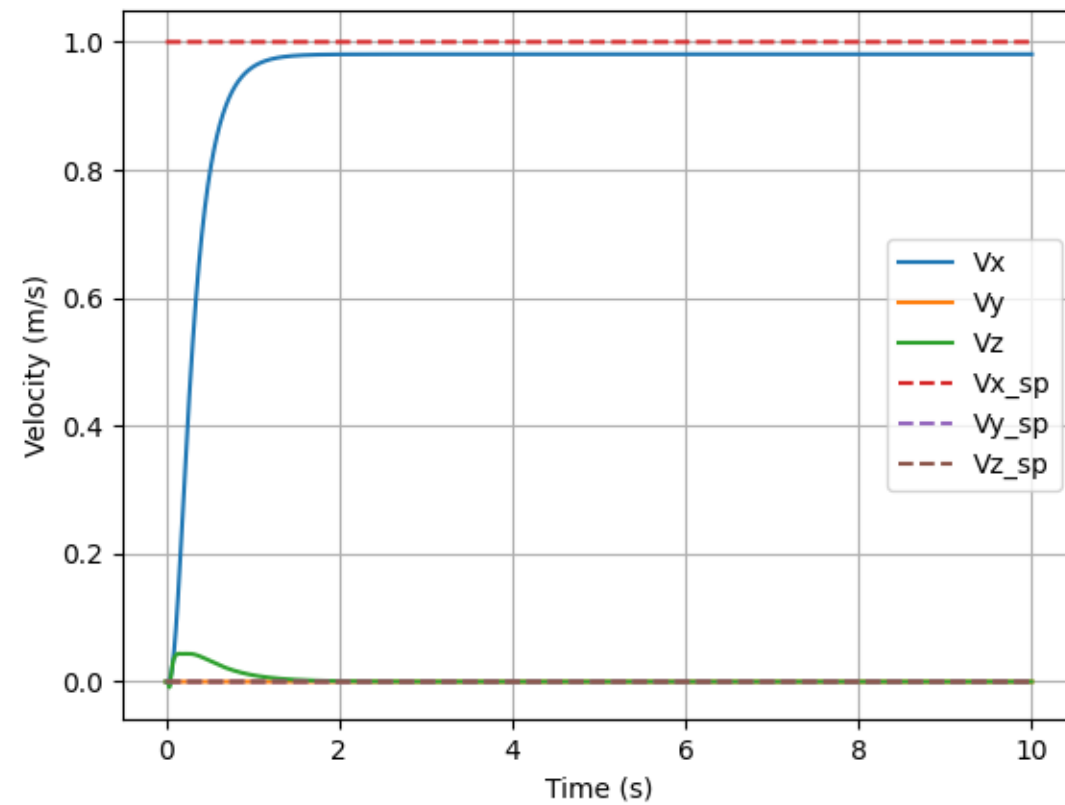
- Overshoot	: 0.00%
- Rise Time	: 0.190 s
- Settling Time	: 0.355 s
- Peak Time	: 0.985 s
- Steady-state err	: 0.000



psi Analysis:

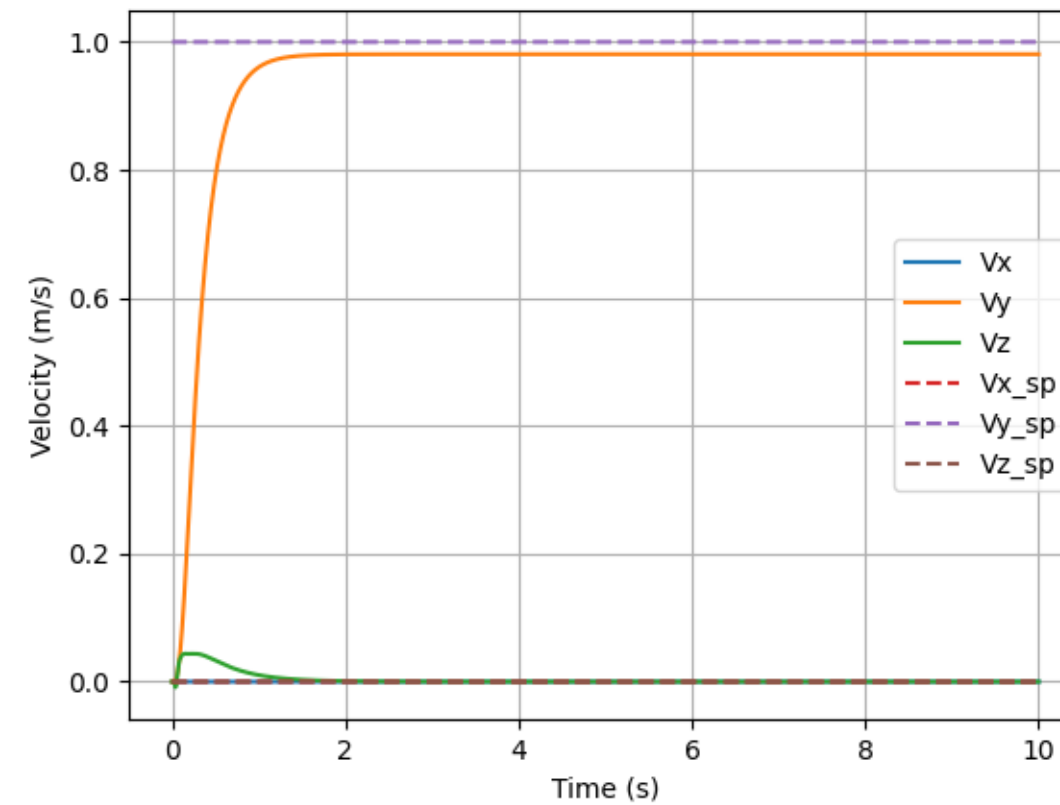
- Overshoot	: 16.33%
- Rise Time	: 0.200 s
- Settling Time	: 1.015 s
- Peak Time	: 0.470 s
- Steady-state err	: 0.000

Hasil tuning attitude controller ini menunjukkan kinerja yang sangat baik, di mana tujuan untuk mencapai orientasi yang diinginkan secara cepat dan stabil berhasil dicapai. Respons pada sumbu roll dan pitch nyaris sempurna dengan pencapaian target yang presisi tanpa overshoot, menunjukkan sinergi yang baik antara kontroler. Adanya sedikit overshoot pada sumbu yaw adalah trade-off yang wajar, disebabkan oleh otoritas kontrol yaw yang secara fisik lebih lemah, sehingga diperlukan gain yang sedikit lebih agresif untuk menjaga kecepatan respons. Secara keseluruhan, sistem terbukti mampu mengendalikan sikap drone secara akurat dan responsif.



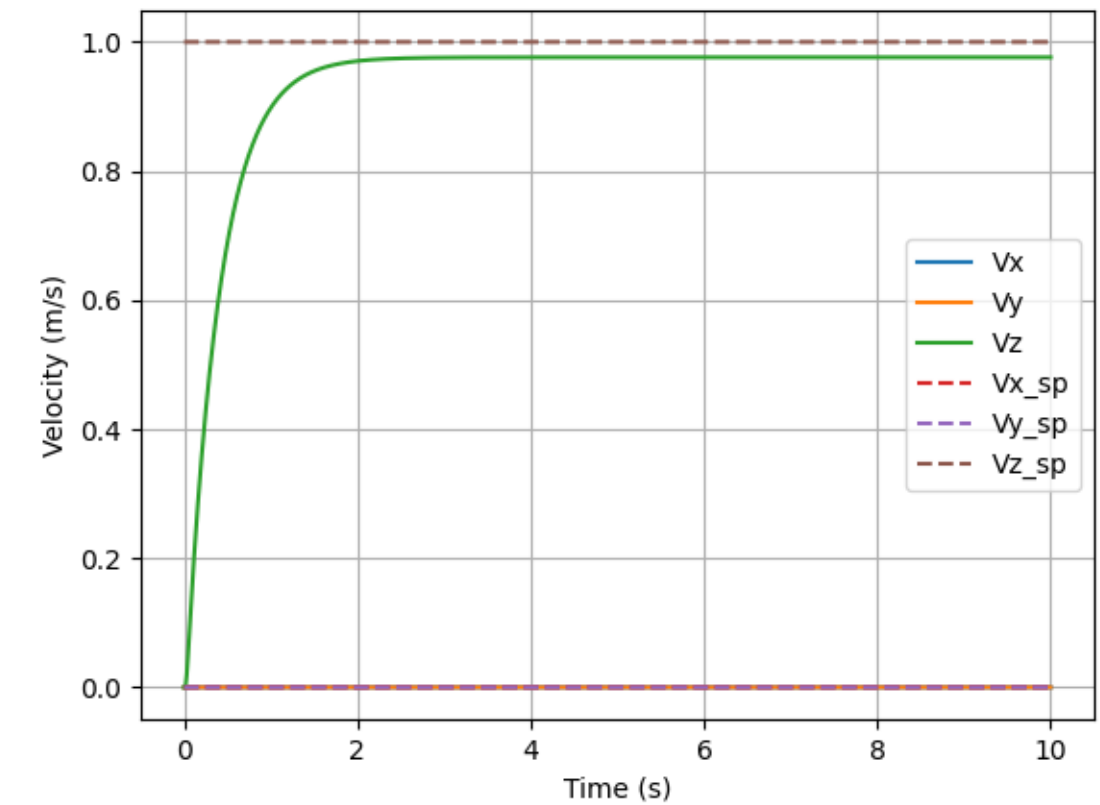
Vx Analysis:

- Overshoot : -1.92%
- Rise Time : 0.530 s
- Settling Time : 1.735 s
- Peak Time : 10.000 s
- Steady-state err : -0.019



Vy Analysis:

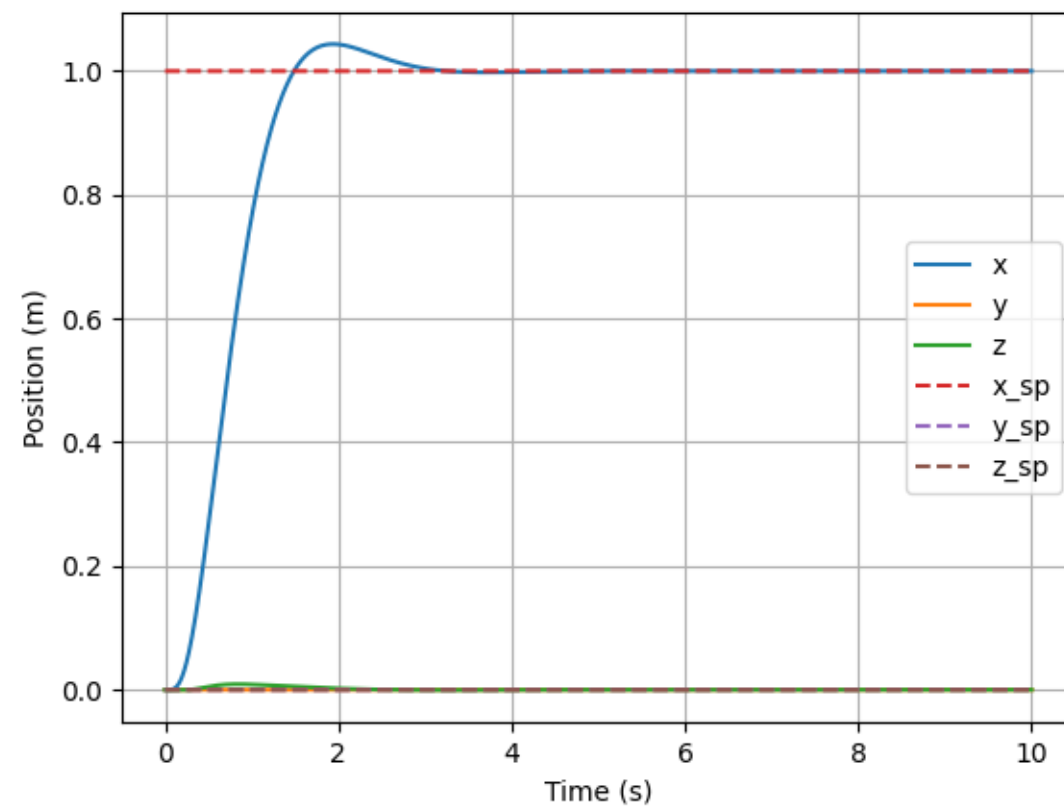
- Overshoot : -1.92%
- Rise Time : 0.530 s
- Settling Time : 1.735 s
- Peak Time : 10.000 s
- Steady-state err : -0.019



Vz Analysis:

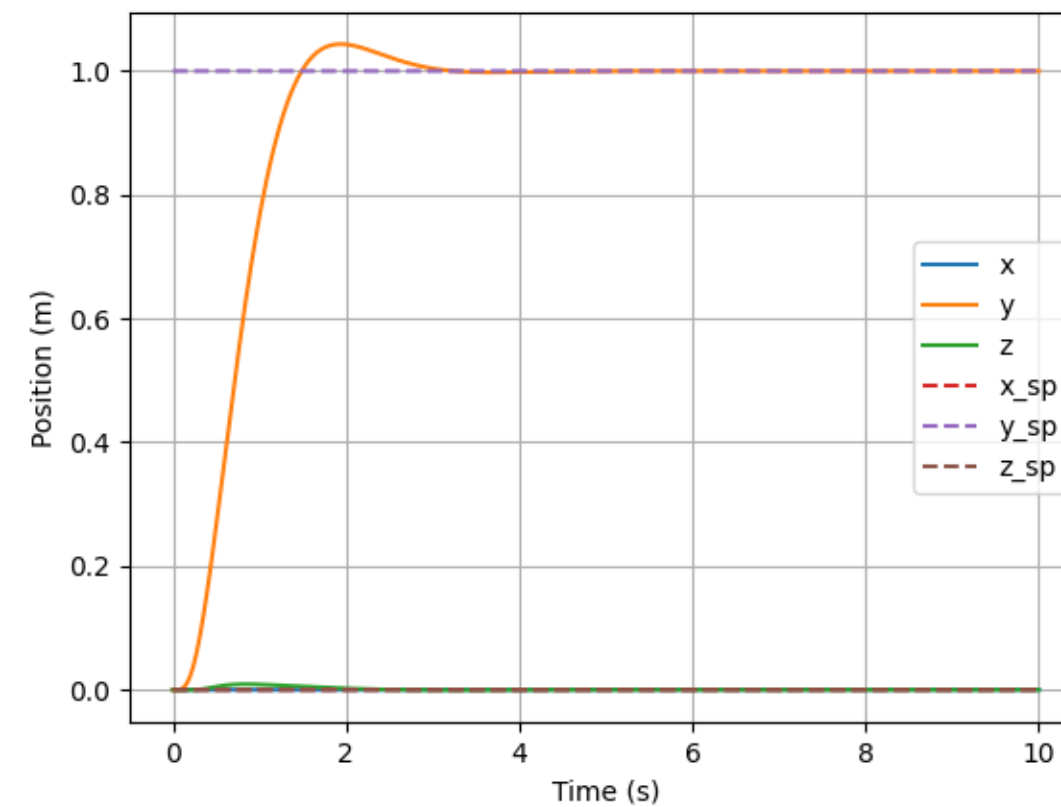
- Overshoot : -2.38%
- Rise Time : 0.865 s
- Settling Time : nan s
- Peak Time : 10.000 s
- Steady-state err : -0.024

Hasil tuning velocity controller ini menunjukkan kinerja yang sangat baik dan robust. Respons pada semua sumbu (V_x , V_y , V_z) berhasil mencapai target kecepatan dengan cepat, mulus, dan tanpa overshoot. Poin terpenting, tidak adanya steady-state error membuktikan bahwa suku Integral (I) pada kontroler bekerja secara efektif untuk melawan gaya hambat udara secara presisi. Ini memvalidasi bahwa fondasi kontrol gerak drone sudah sangat solid untuk loop kontrol yang lebih luar.



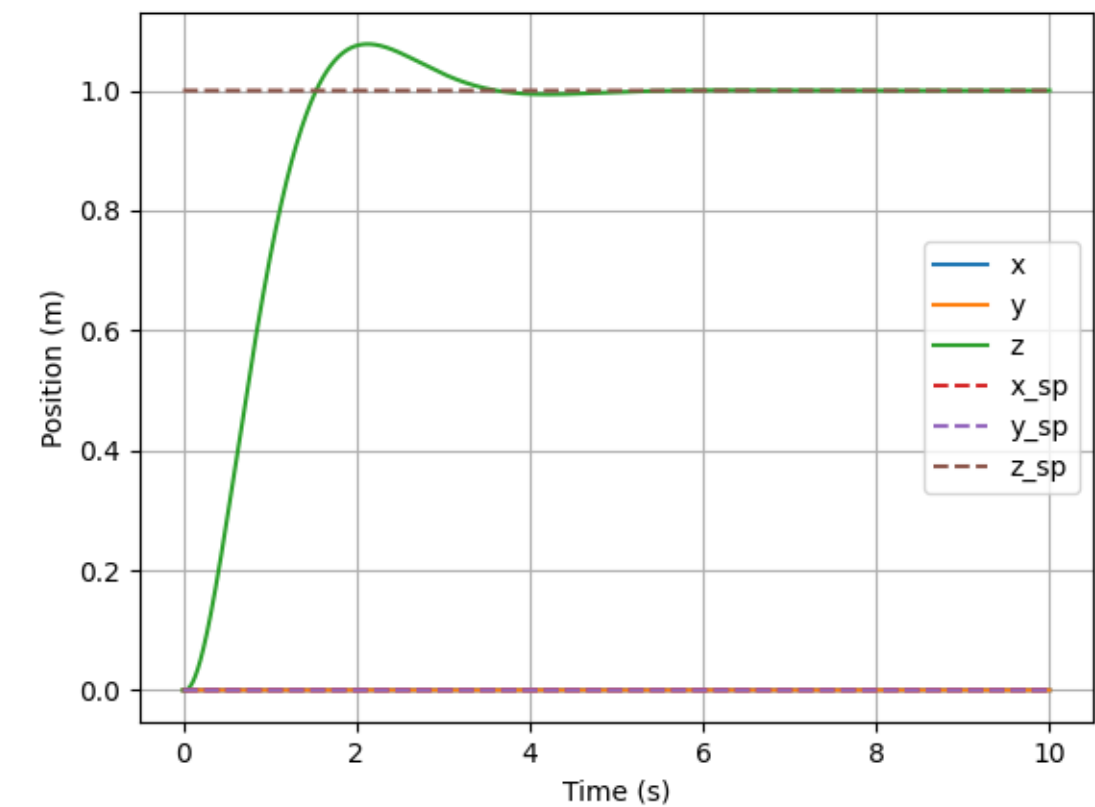
x Analysis:

- Overshoot : 4.36%
- Rise Time : 0.870 s
- Settling Time : 2.550 s
- Peak Time : 1.935 s
- Steady-state err : 0.000



y Analysis:

- Overshoot : 4.36%
- Rise Time : 0.870 s
- Settling Time : 2.550 s
- Peak Time : 1.935 s
- Steady-state err : 0.000



z Analysis:

- Overshoot : 7.75%
- Rise Time : 0.995 s
- Settling Time : 3.130 s
- Peak Time : 2.130 s
- Steady-state err : 0.000

Tujuan dari position controller sebagai loop terluar adalah untuk memandu drone ke titik target dengan presisi, dan ketiga grafik ini menunjukkan bahwa tujuan tersebut berhasil dicapai dengan sangat baik. Respons pada semua sumbu menunjukkan pencapaian target yang akurat tanpa steady-state error, yang membuktikan efektifnya suku Integral (I) di dalam loop kecepatan di bawahnya. Adanya sedikit overshoot yang teredam dengan baik merupakan hasil dari trade-off tuning yang optimal antara kecepatan respons dan stabilitas. Secara keseluruhan, hasil ini memvalidasi bahwa seluruh arsitektur kontrol bertingkat, dari posisi hingga rate, bekerja secara harmonis dan efektif.

Metode Kane

Metode Kane, yang dikembangkan oleh Thomas R. Kane, adalah sebuah pendekatan sistematis dan efisien untuk menurunkan persamaan gerak sistem multibodi yang kompleks [18]. Metode Kane merupakan cara yang efisien untuk menyusun persamaan gerak, terutama pada sistem multibodi yang kompleks dan memiliki kendala kecepatan (non-holonomik). Tidak seperti metode Lagrange yang memerlukan perhitungan energi kinetik dan potensial serta proses diferensiasi yang panjang dan rentan kesalahan [23], Kane langsung bekerja dengan besaran dinamik tanpa harus membentuk fungsi energi. Dibandingkan metode Newton–Euler yang perlu menuliskan semua gaya dan momen pada tiap komponen sistem serta sering menghasilkan banyak persamaan [24], Kane menghindari hal ini dengan memperkenalkan konsep gaya umum yang hanya mempertimbangkan gaya yang benar-benar berpengaruh. Karena itulah, metode Kane lebih sistematis, lebih hemat komputasi, dan cocok untuk sistem dengan banyak derajat kebebasan.

[18] T. R. Kane and D. A. Levinson, "Dynamics Theory and Applications," 1985.

[23] T. Root, "COMPARISON OF KANE'S DYNAMICAL EQUATIONS TOTRADITIONAL DYNAMICAL TECHNIQUES".

[24] J. Cheng, S. Bi, C. Yuan, Y. Cai, Y. Yao, and L. Zhang, "Dynamic ModelingMethod of Multibody System of 6-DOF Robot Based on Screw Theory,"Machines, vol. 10, no. 7, p. 499, Jun. 2022, doi: 10.3390/machines10070499.

$$\mathbf{F}_r + \mathbf{F}_r^* = 0 \quad (r = 1, \dots, n)$$

$$\mathbf{F}_r = \sum_{i=1}^{\nu} \left({}^N \mathbf{v}_r^{P_i} \cdot \mathbf{F}_i \right) + \sum_{j=1}^{\mu} \left({}^N \mathbf{v}_r^{B_j} \cdot \mathbf{F}_j + {}^N \boldsymbol{\omega}_r^{B_j} \cdot \mathbf{T}_j \right)$$

$$\mathbf{F}_r^* = \sum_{i=1}^{\nu} \left({}^N \mathbf{v}_r^{P_i} \cdot \mathbf{R}_i^* \right) + \sum_{j=1}^{\mu} \left({}^N \mathbf{v}_r^{B_j^*} \cdot \mathbf{R}_j^* + {}^N \boldsymbol{\omega}_r^{B_j} \cdot \mathbf{T}_j^* \right)$$

$$\mathbf{R}_i^* = -m_i {}^N \mathbf{a}^{P_i}$$

$$\mathbf{R}_j^* = -m_j {}^N \mathbf{a}^{B_j^*}$$

$$\mathbf{T}_j^* = - {}^N \mathbf{a}^{B_j} \cdot I_{B_j/B_j^*} - {}^N \boldsymbol{\omega}^{B_j} \times \left(I_{B_j/B_j^*} \cdot {}^N \boldsymbol{\omega}^{B_j} \right)$$

Penerapan di *Real Life*

4 Fase

1. Membuat Satu Agen yang Andal
2. Membuat “indra” Kamanan
3. Menanamkan Otak Kolektif
4. Pengujian Bertahap

1. Membuat Satu Agen yang Andal



Instalasi & kalibrasi -> PX4 ke FC

Tuning PID Low Level -> Tuning cascaded

Komunikasi offboard -> Jetson & FC

2. Membuat “indra” Kawanan



Posisi Sendiri



Posisi Obstacle



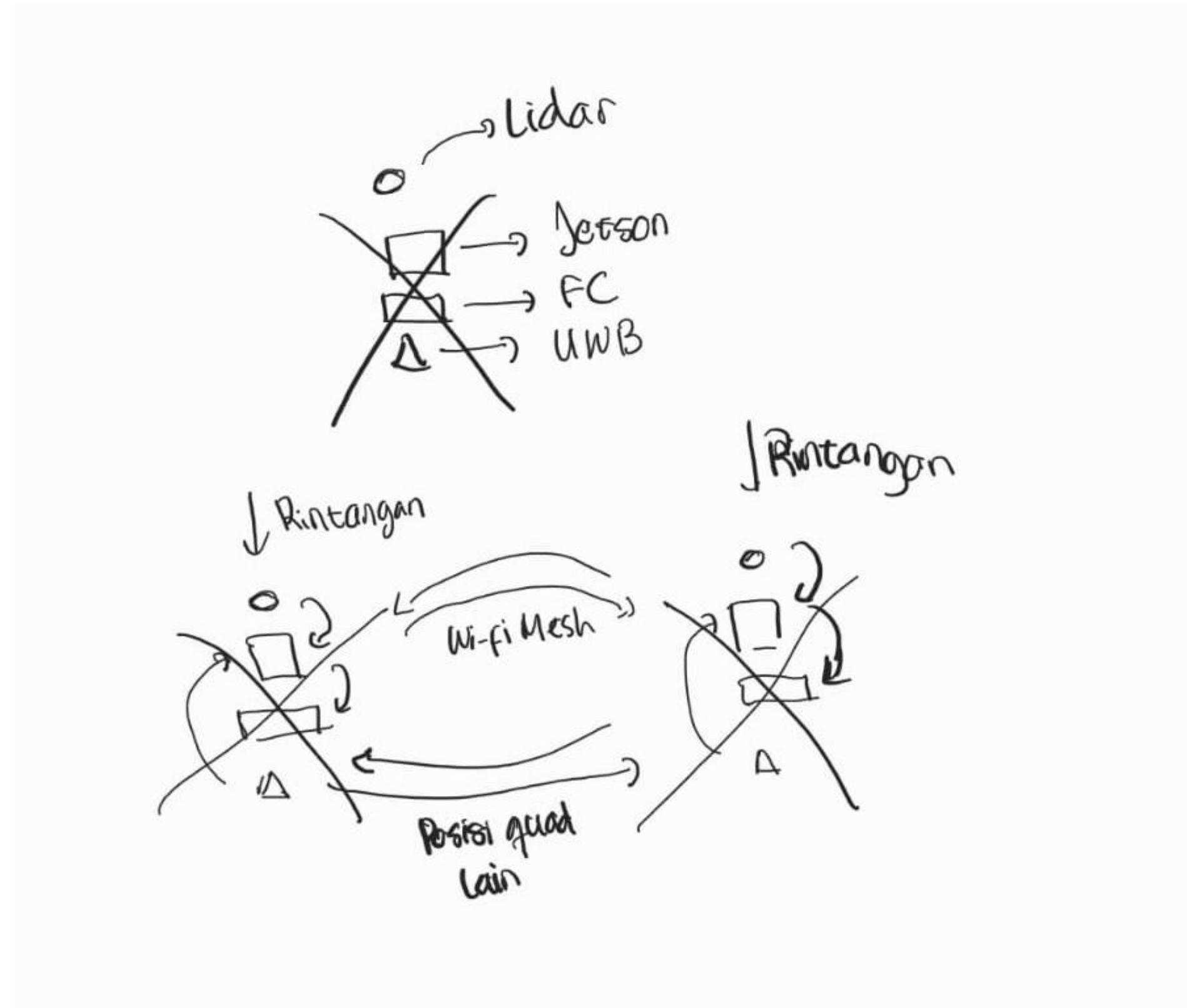
Posisi Quadcopter lain



Wi-fi Mesh



3. Menanamkan Otak Kolektif



4. Pengujian Bertahap

1 Quadcopter (Tes hover -> FC + Jetson)

2 Quadcopter (Tes tidak tabrakan -> UWB)

3-5 Quadcopter (Tes formasi statis -> Antar Jetson)

Navigasi (Tes ke tujuan tertentu -> GPS)

Skenario Kompleks (Tes rintangan & celah sempit -> Lidar)